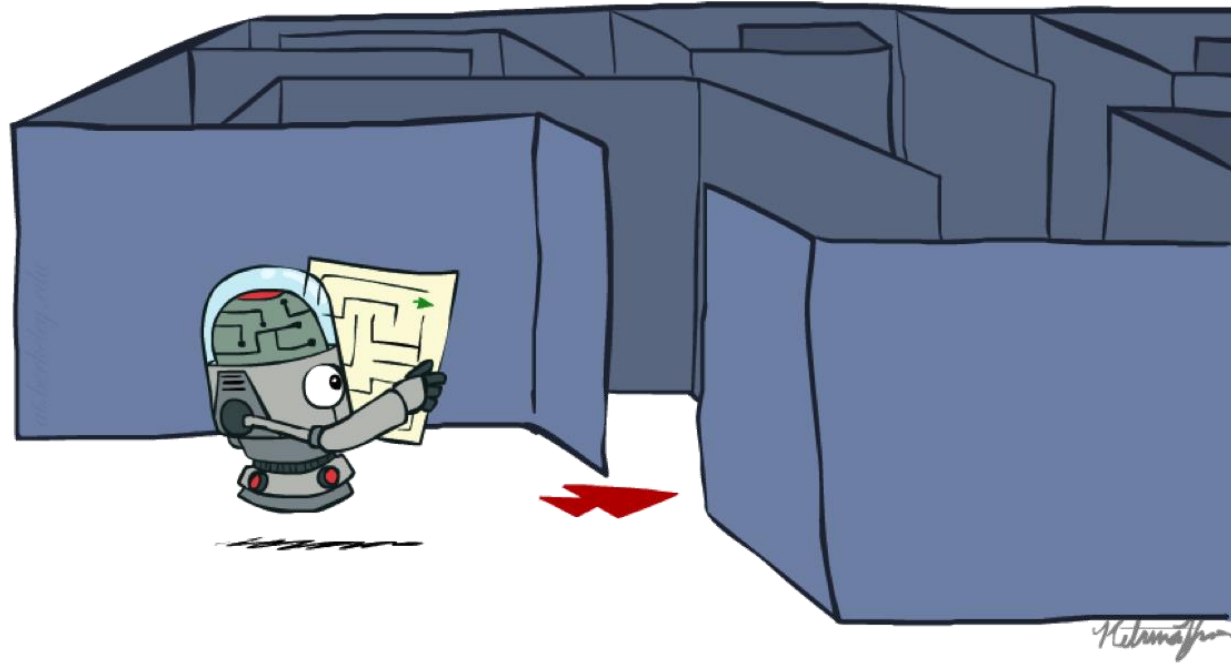


# Artificial Intelligence

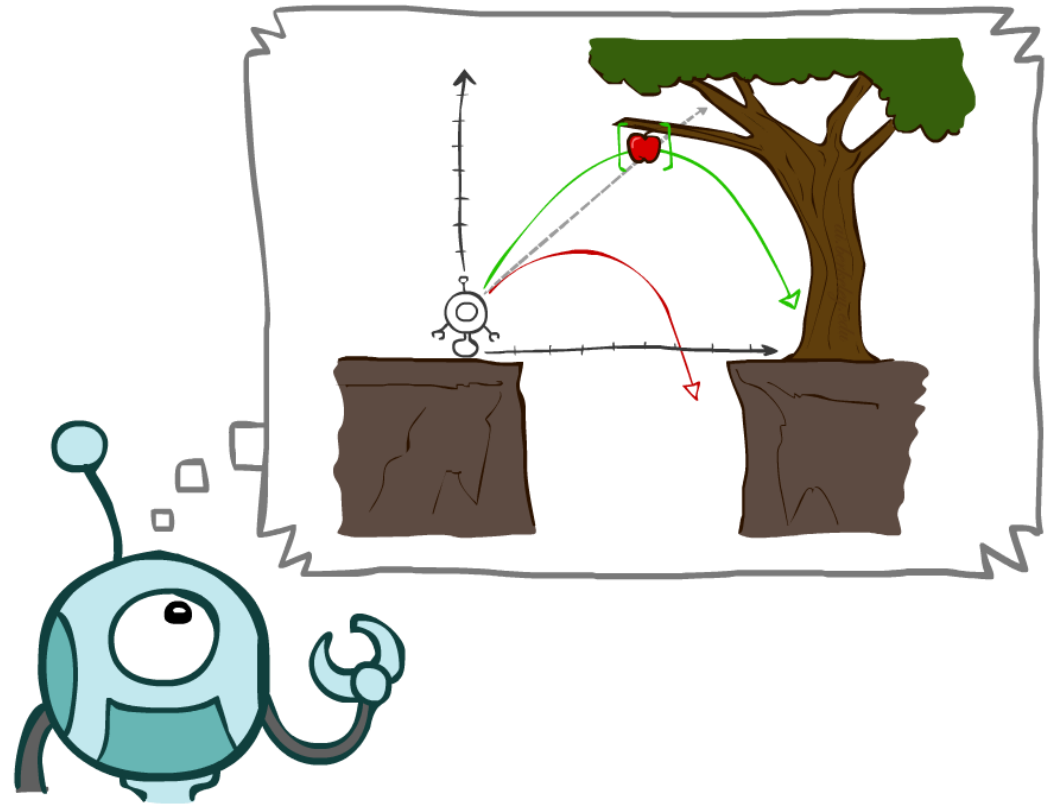
## Search



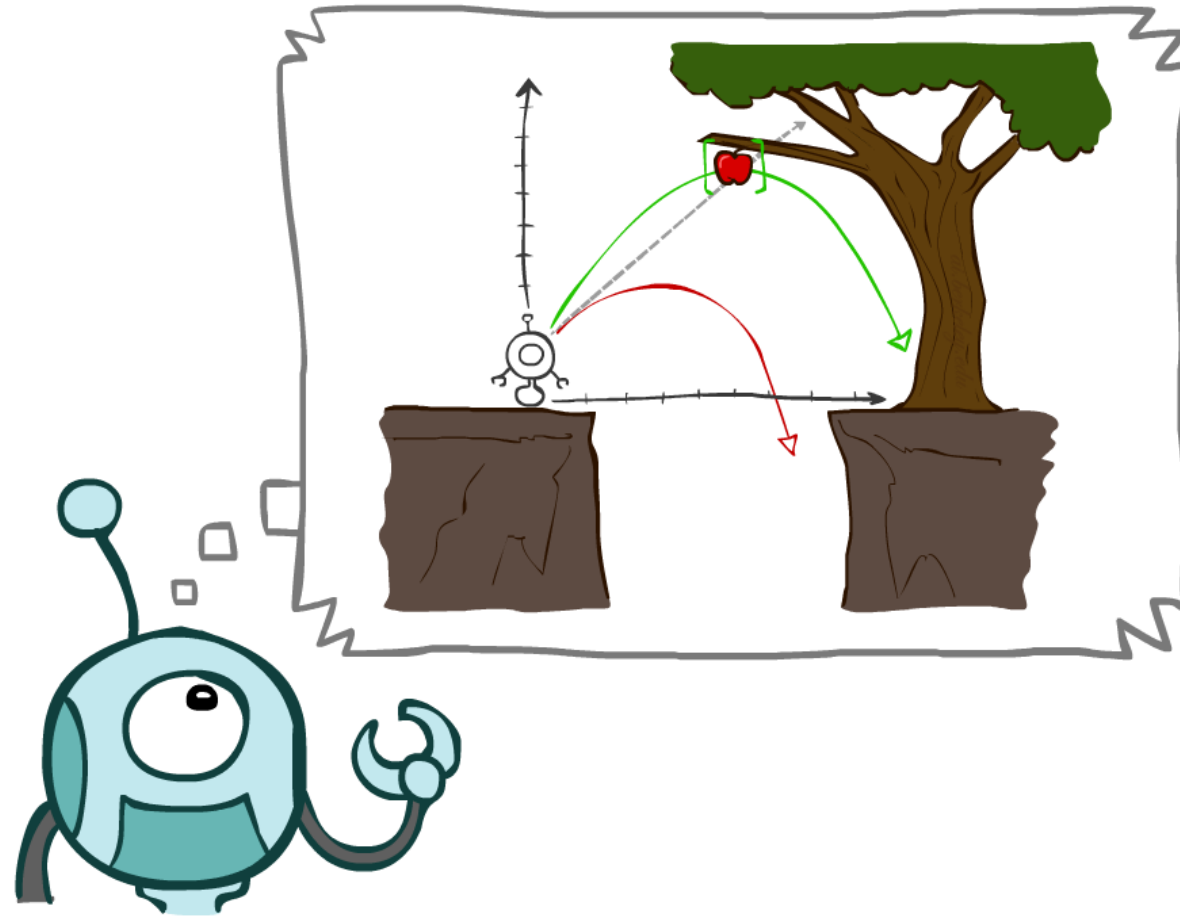
[These slides are a minor adaptation of the slides that were created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley ([ai.berkeley.edu](http://ai.berkeley.edu)).]

# Today

- Agents that Plan Ahead
- Search Problems
- Uninformed Search Methods
- Informed Search Methods

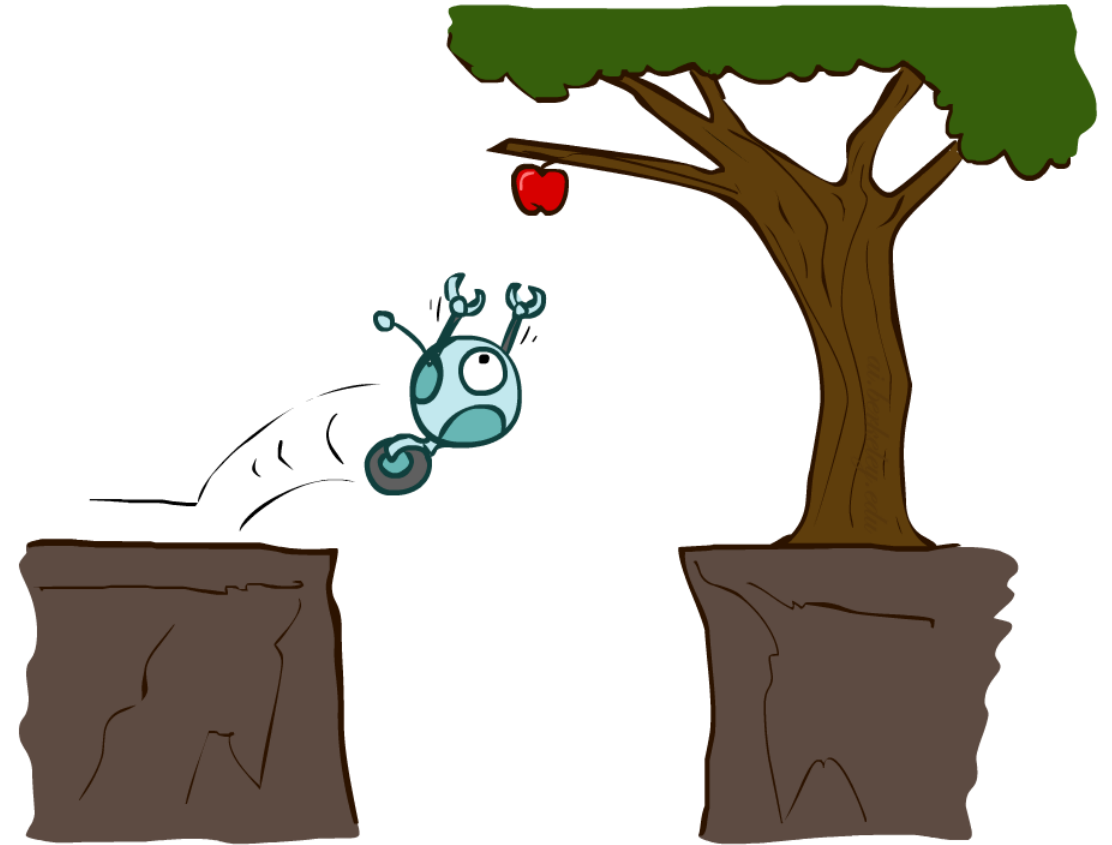


# Agents that Plan

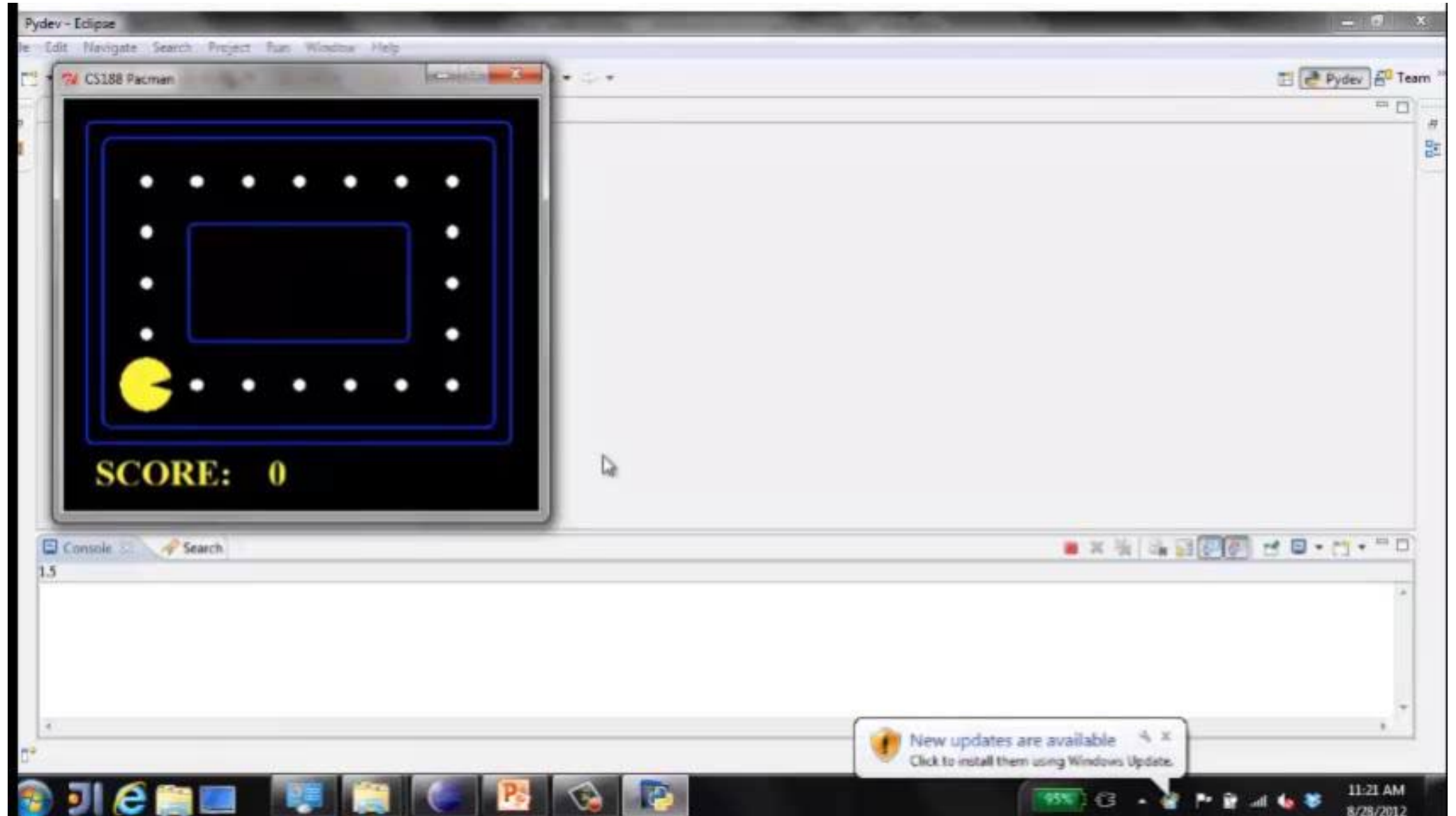


# Reflex Agents

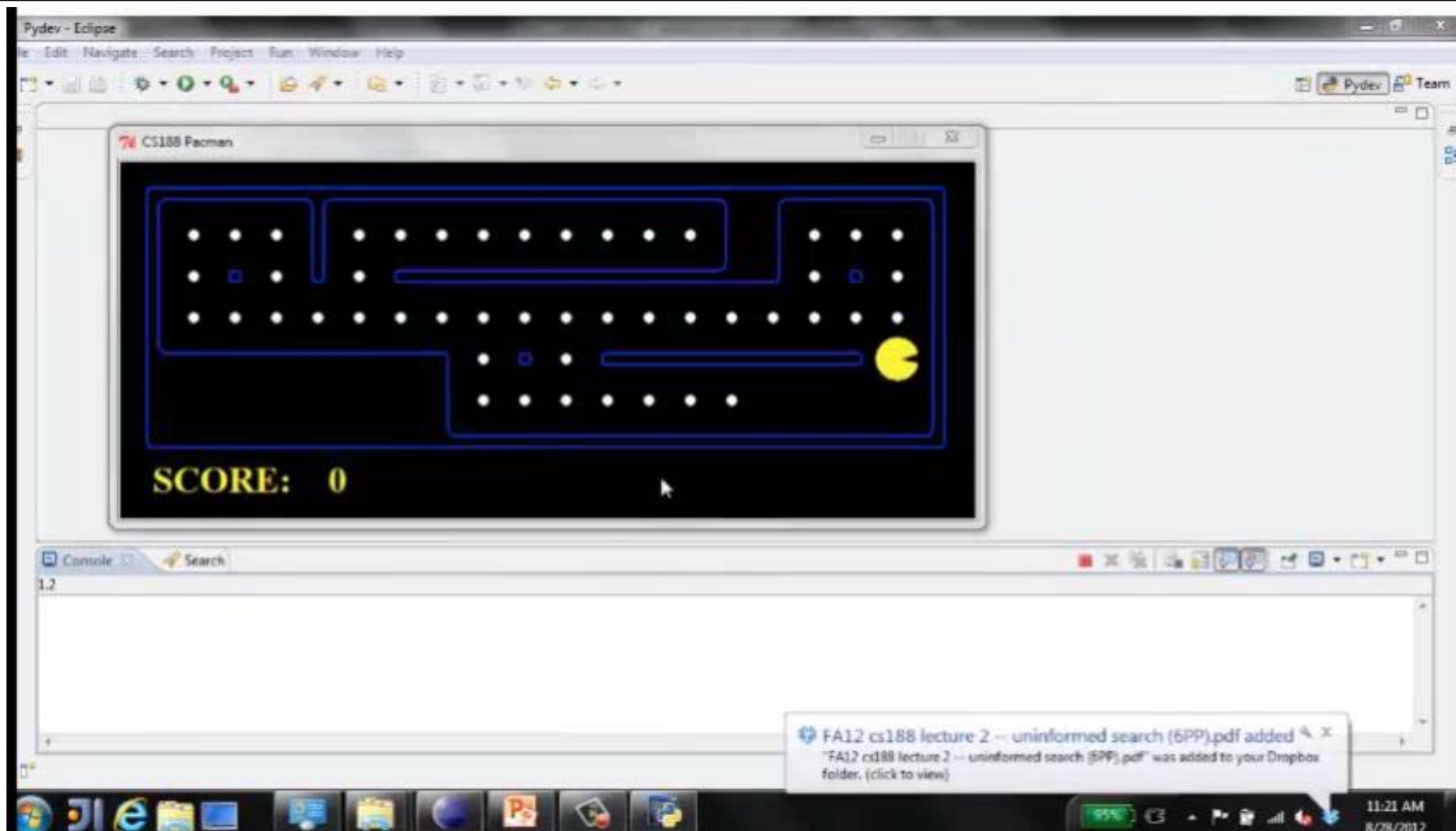
- Reflex agents:
  - Choose action based on current percept (and maybe memory)
  - May have memory or a model of the world's current state
  - Do not consider the future consequences of their actions
  - Consider how the world IS
- Can a reflex agent be rational?



# Video of Demo Reflex Optimal

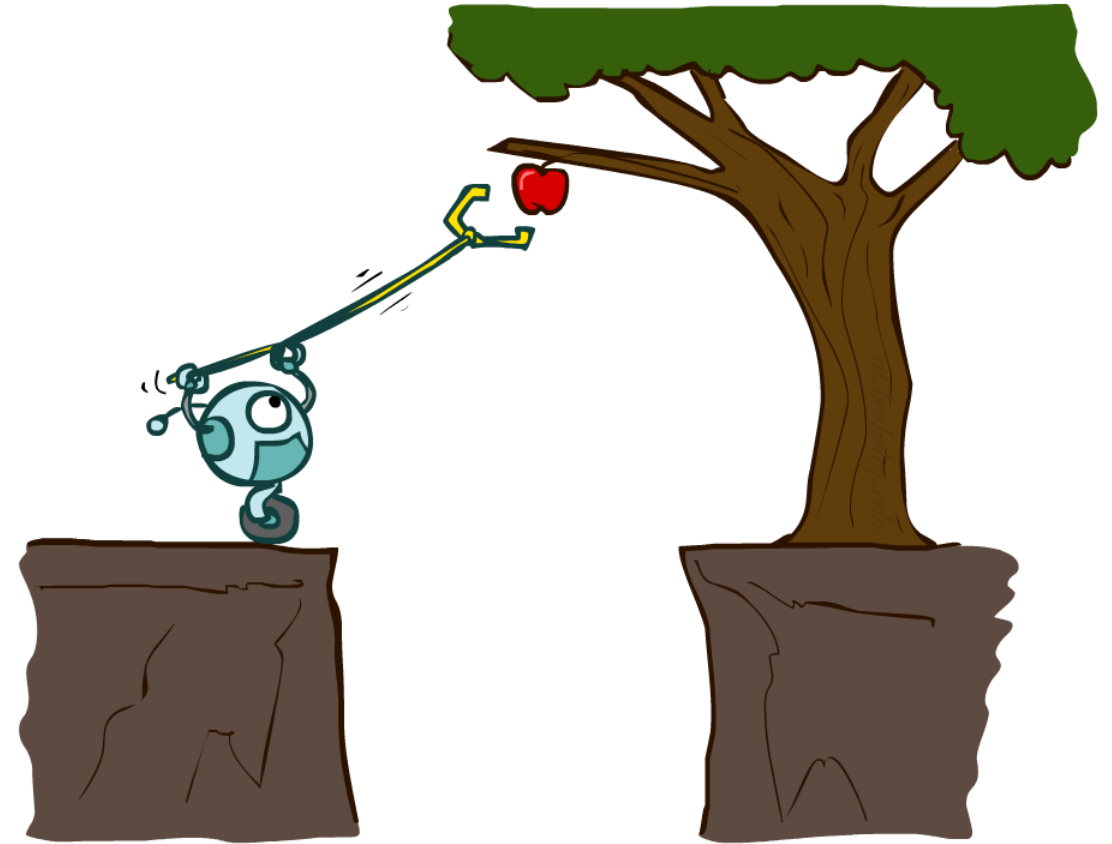


# Video of Demo Reflex Odd

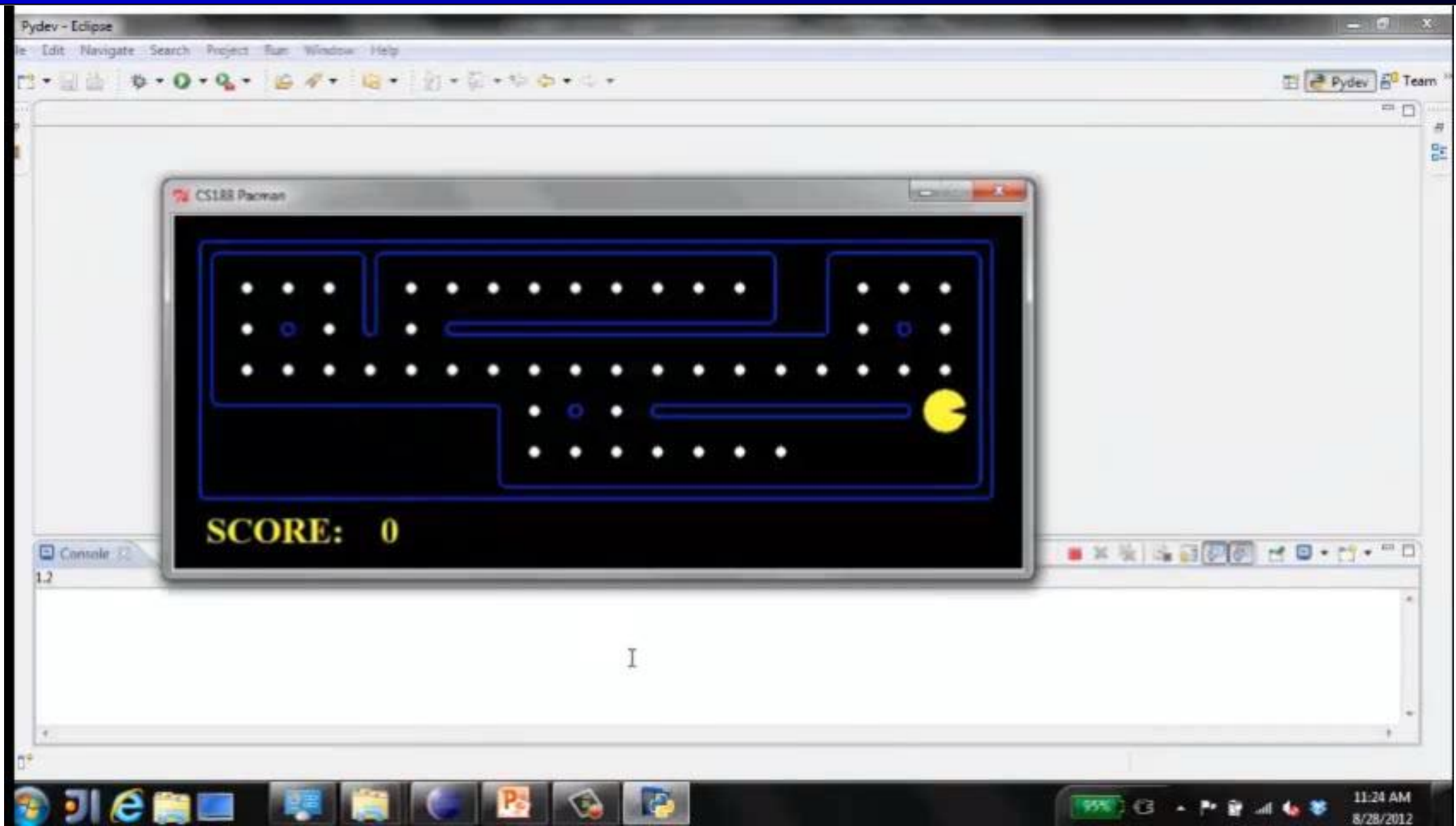


# Planning Agents

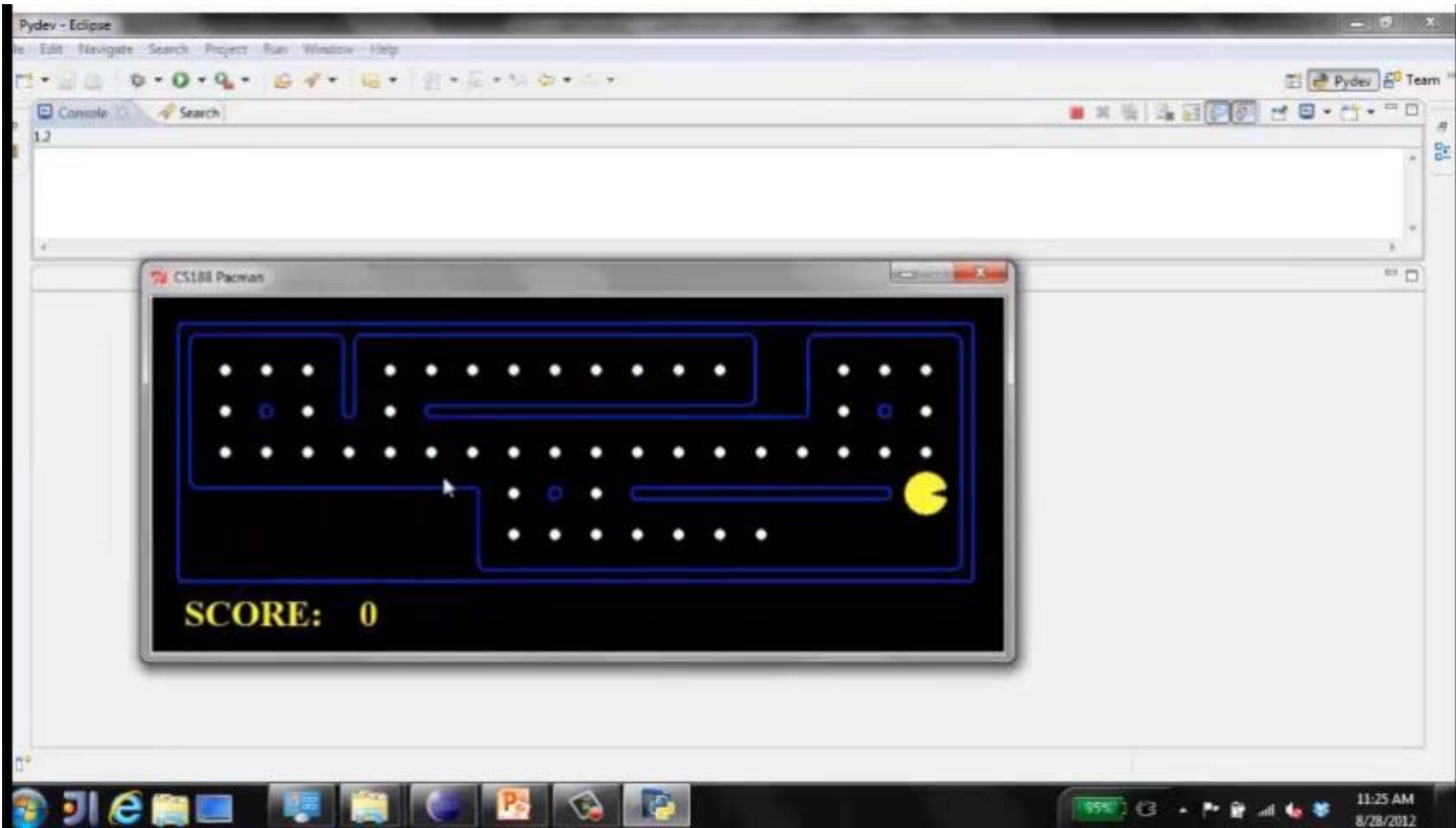
- Planning agents:
  - Ask “what if”
  - Decisions based on (hypothesized) consequences of actions
  - Must have a model of how the world evolves in response to actions
  - Must formulate a goal (test)
  - Consider how the world **WOULD BE**
- Optimal vs. complete planning
- Planning vs. replanning



# Video of Demo Replanning



# Video of Demo Mastermind



# Search Problems

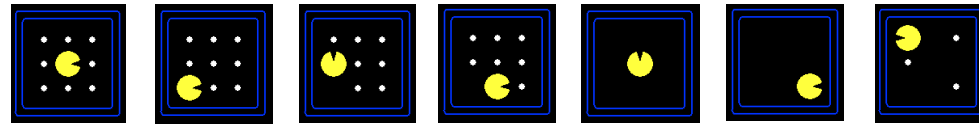
---



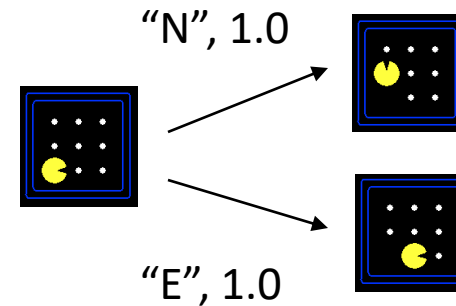
# Search Problems

- A **search problem** consists of:

- A state space



- A successor function  
(with actions, costs)



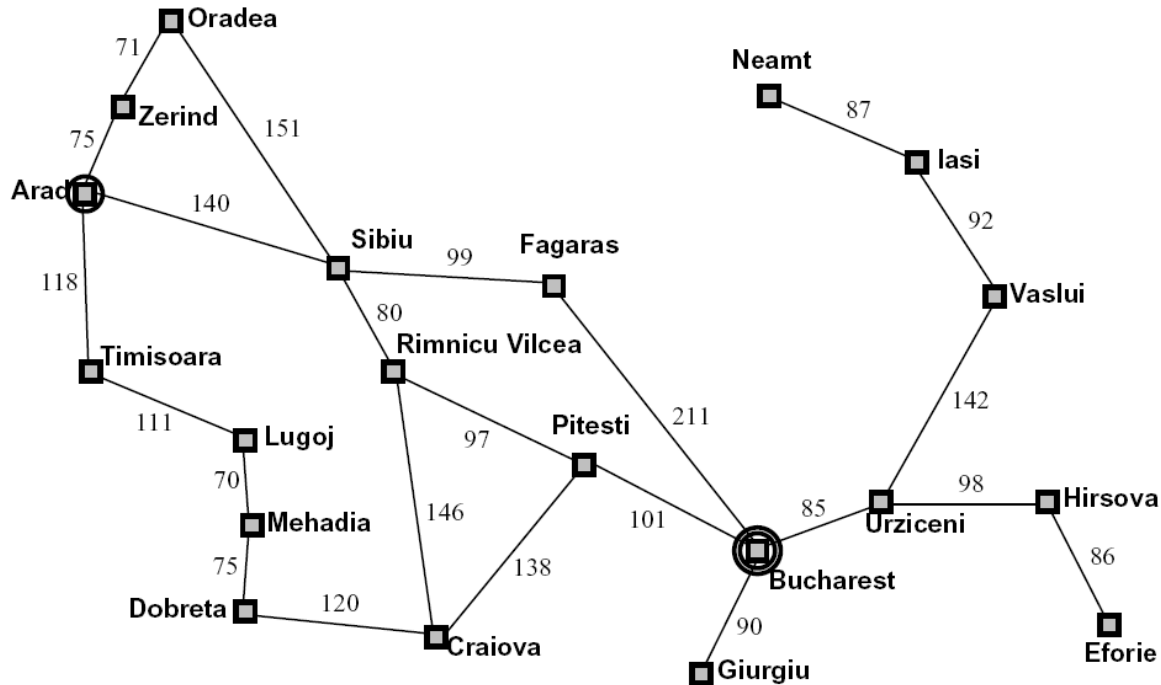
- A start state and a goal test
- A **solution** is a sequence of actions (a plan) which transforms the start state to a goal state

# Search Problems Are Models

---



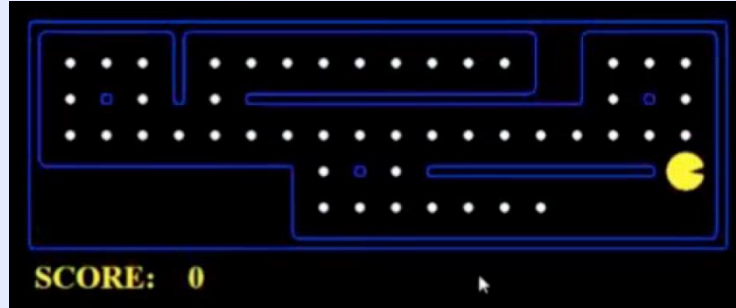
# Example: Traveling in Romania



- State space:
  - Cities
- Successor function:
  - Roads: Go to adjacent city with cost = distance
- Start state:
  - Arad
- Goal test:
  - Is state == Bucharest?
- Solution?

# What's in a State Space?

The **world state** includes every last detail of the environment



A **search state** keeps only the details needed for planning (abstraction)

## ■ Problem: Pathing

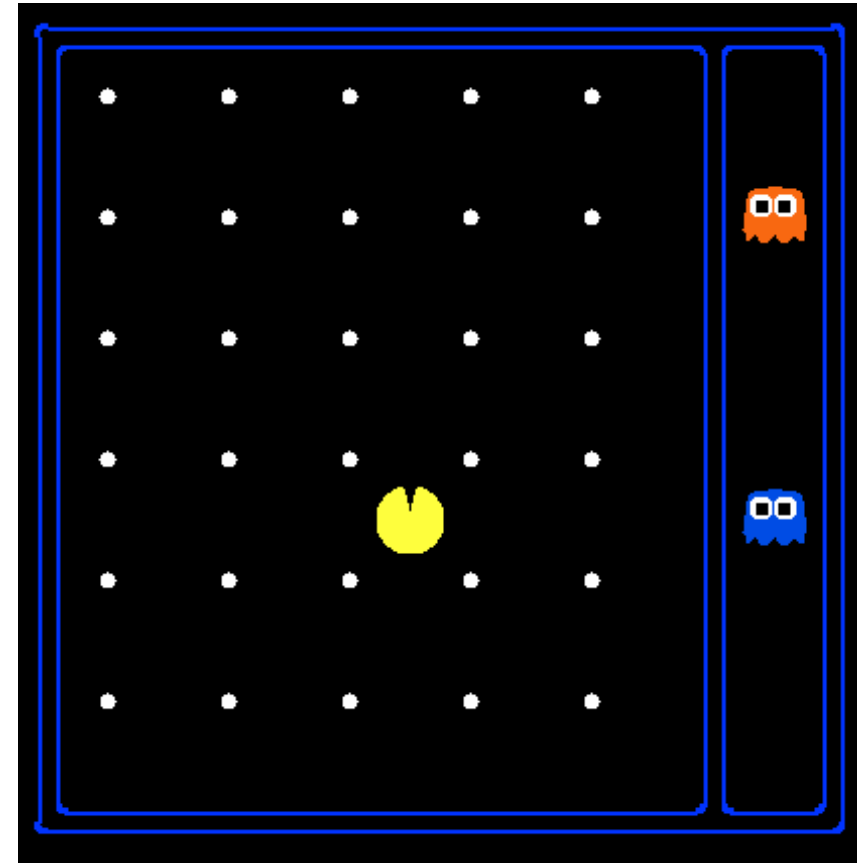
- States:  $(x,y)$  location
- Actions: NSEW
- Successor: update location only
- Goal test: is  $(x,y)=\text{END}$

## ■ Problem: Eat-All-Dots

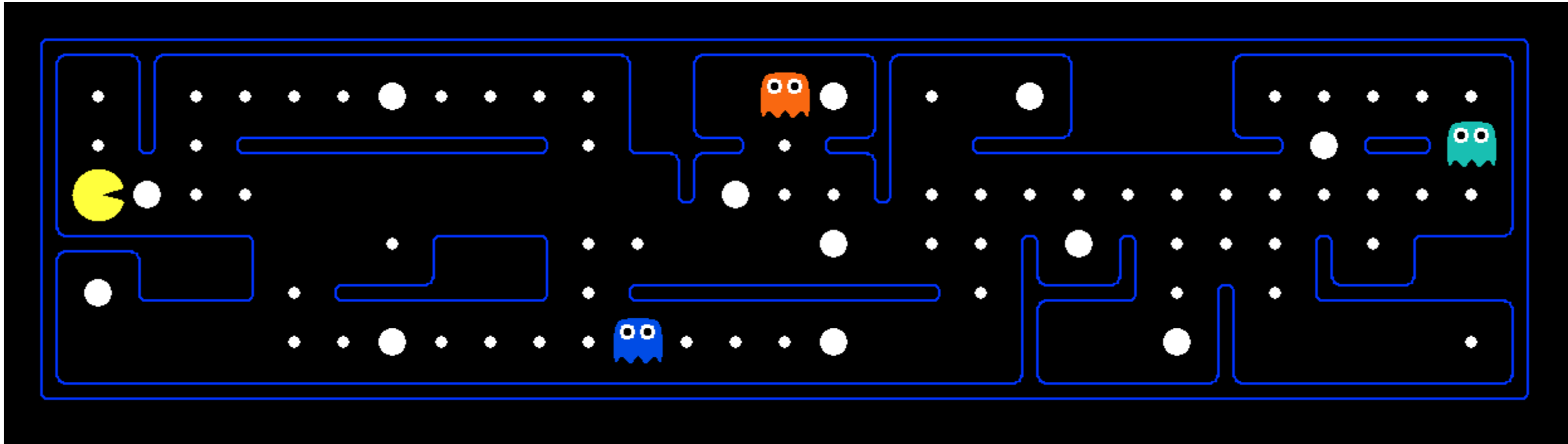
- States:  $\{(x,y), \text{dot booleans}\}$
- Actions: NSEW
- Successor: update location and possibly a dot boolean
- Goal test: dots all false

# State Space Sizes?

- World state:
  - Agent positions: 120
  - Food count: 30
  - Ghost positions: 12
  - Agent facing: NSEW
- How many
  - World states?  
 $120 \times (2^{30}) \times (12^2) \times 4$
  - States for pathing?  
120
  - States for eat-all-dots?  
 $120 \times (2^{30})$



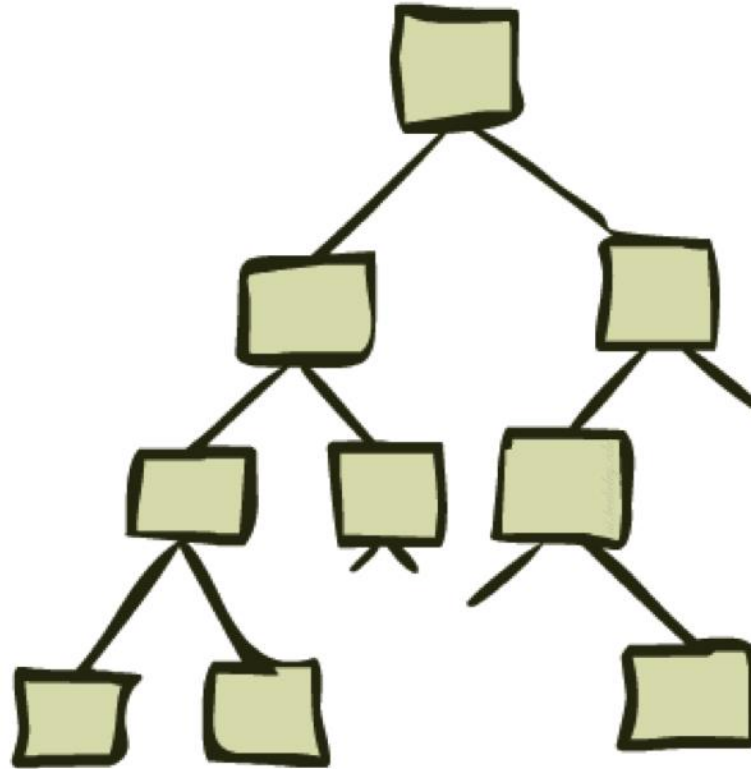
# Quiz: Safe Passage



- Problem: eat all dots while keeping the ghosts perma-scared
- What does the state space have to specify?
  - (agent position, dot booleans, power pellet booleans, remaining scared time)

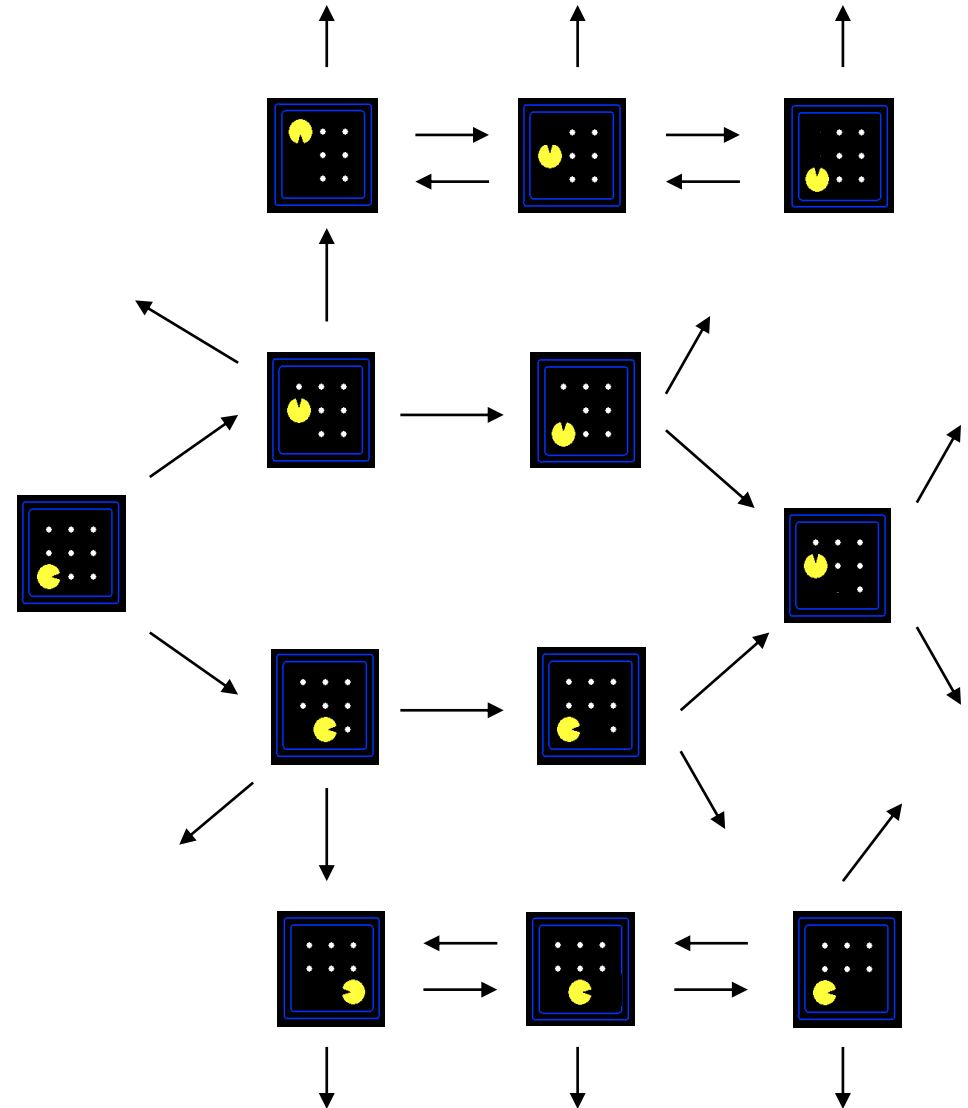
# State Space Graphs and Search Trees

---



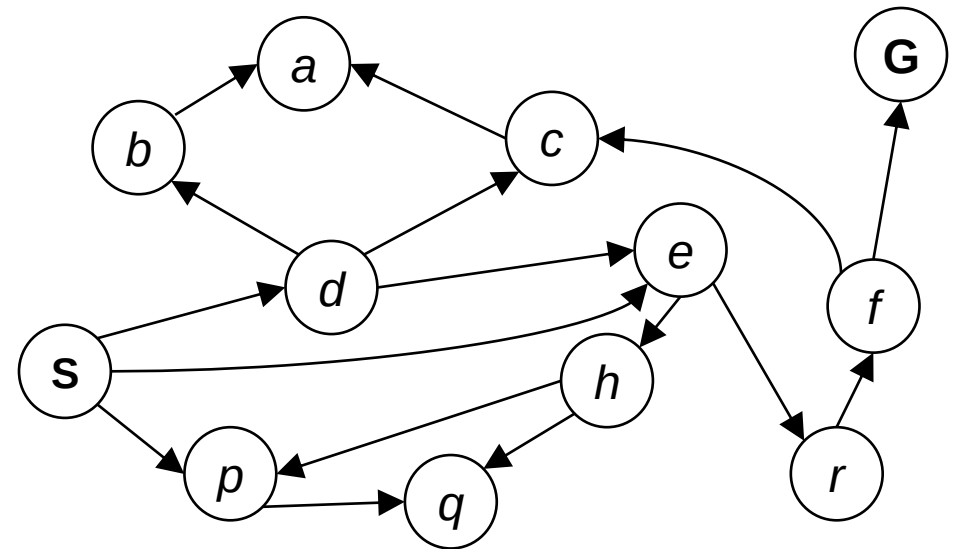
# State Space Graphs

- State space graph: A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Arcs represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



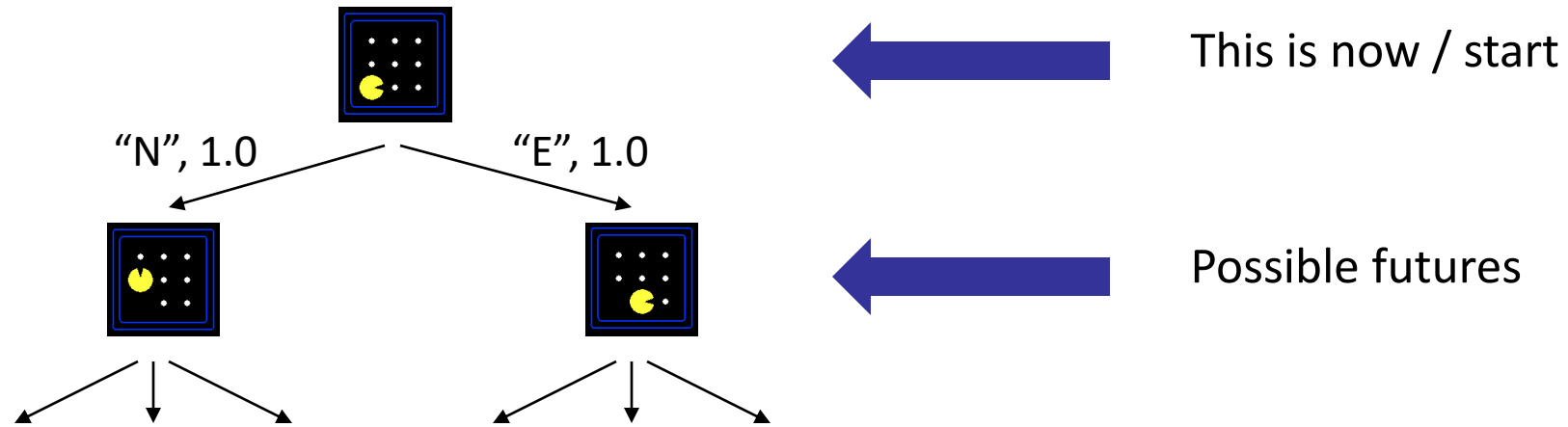
# State Space Graphs

- State space graph: A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Arcs represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



*Tiny state space graph for a tiny search problem*

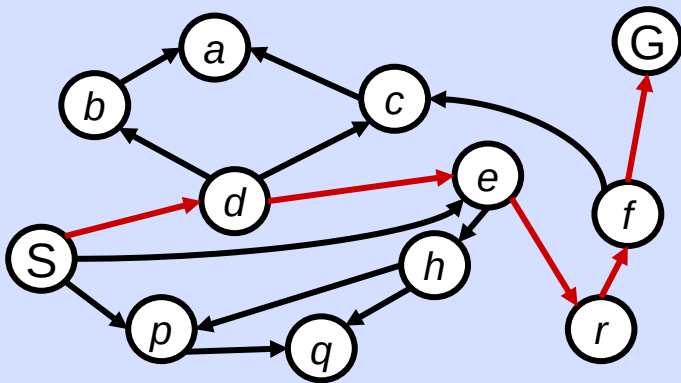
# Search Trees



- A search tree:
  - A “what if” tree of plans and their outcomes
  - The start state is the root node
  - Children correspond to successors
  - Nodes show states, but correspond to PLANS that achieve those states
  - For most problems, we can never actually build the whole tree

# State Space Graphs vs. Search Trees

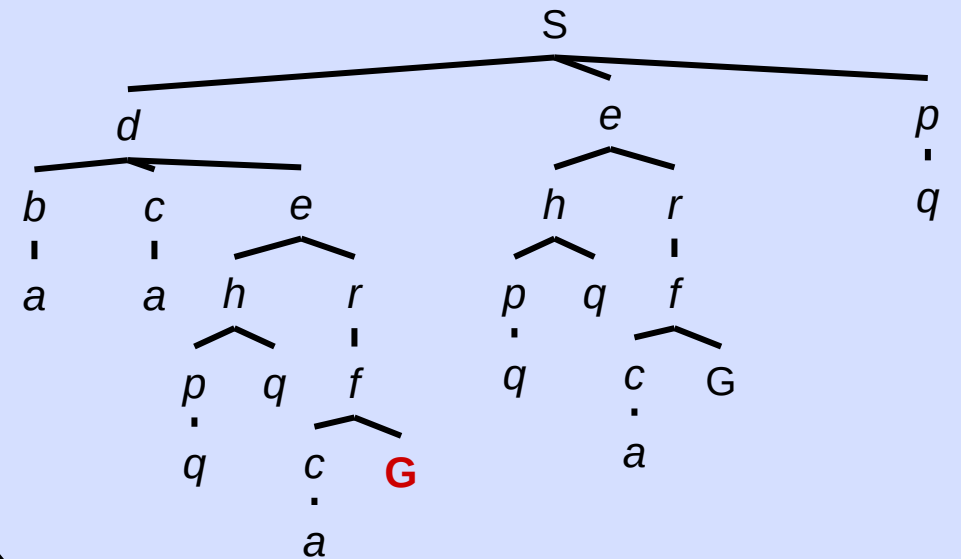
## State Space Graph



*Each NODE in the search tree is an entire PATH in the state space graph.*

*We construct both on demand – and we construct as little as possible.*

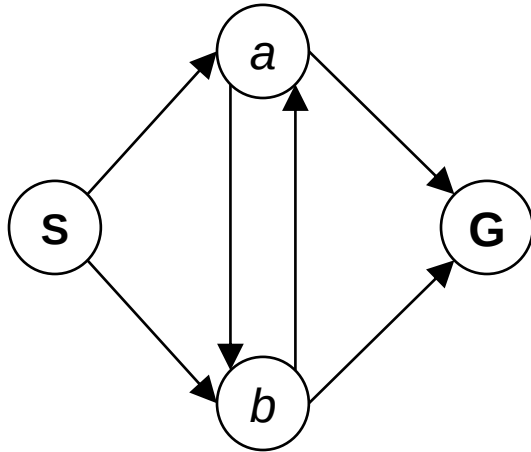
## Search Tree



# Quiz: State Space Graphs vs. Search Trees

---

Consider this 4-state graph:

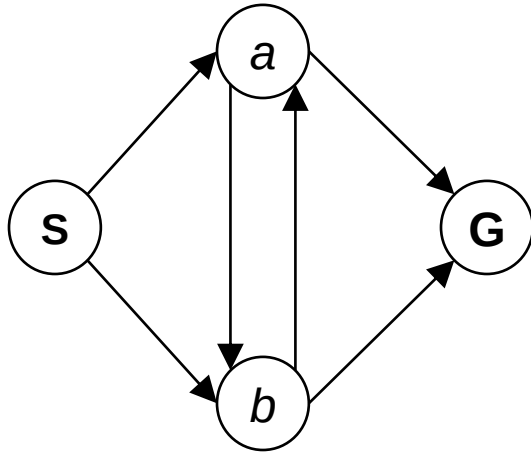


How big is its search tree (from S)?

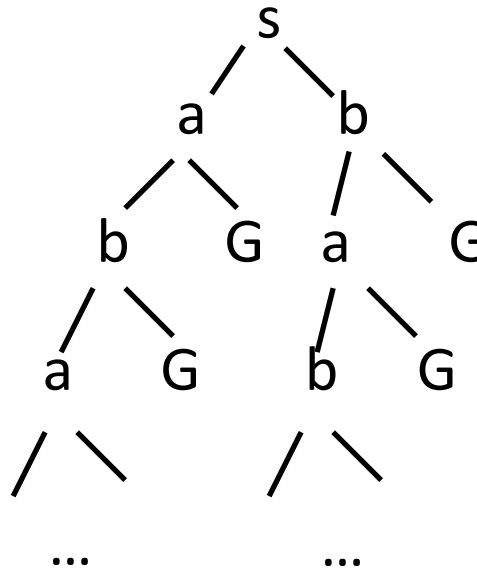


# Quiz: State Space Graphs vs. Search Trees

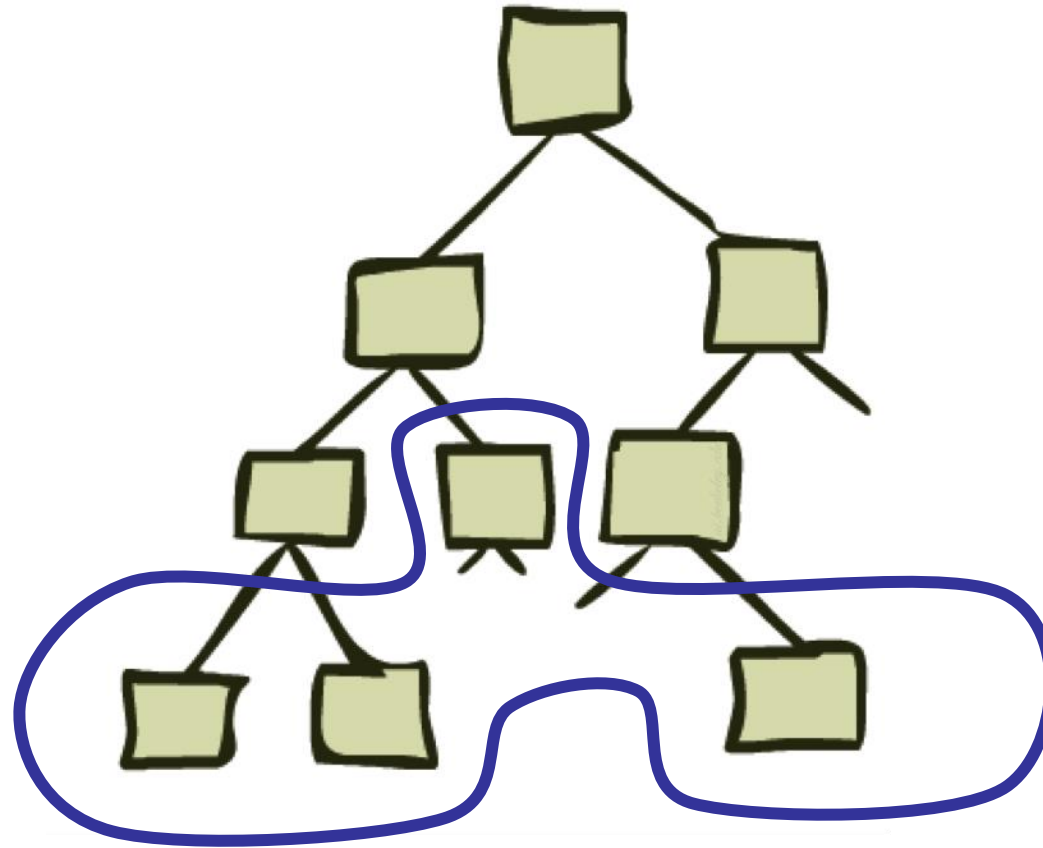
Consider this 4-state graph:



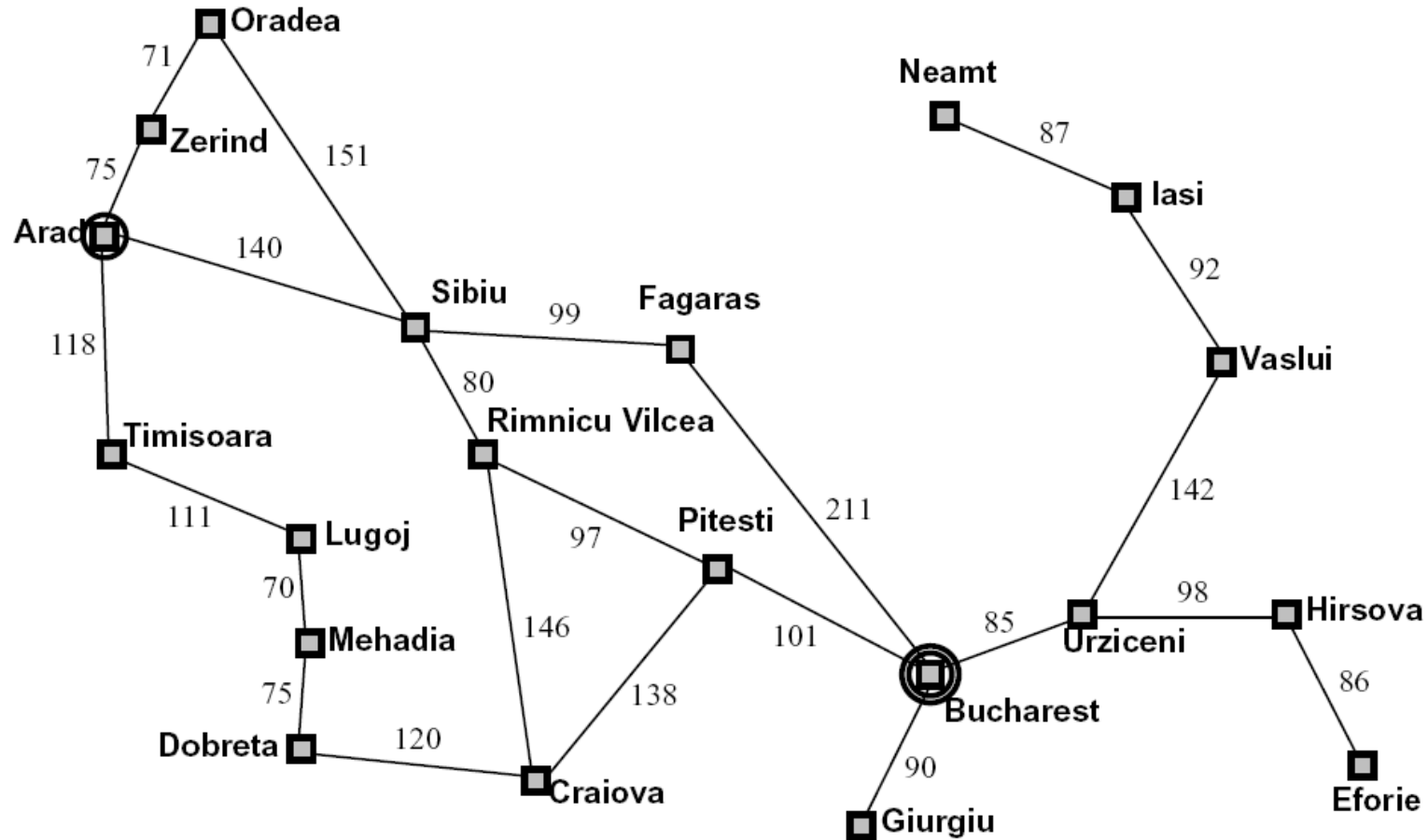
How big is its search tree (from S)?



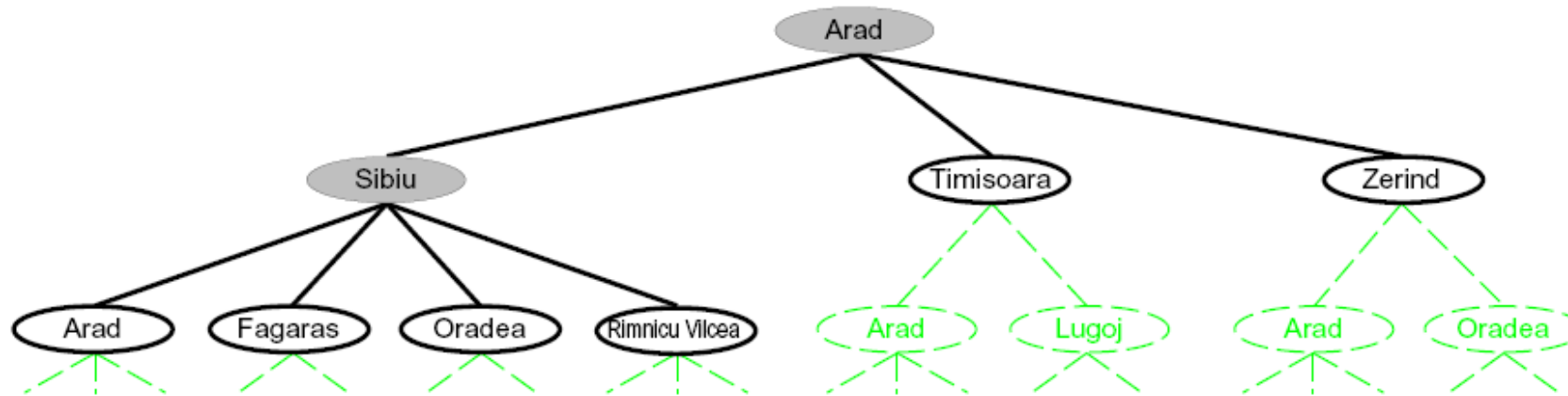
Important: Lots of repeated structure in the search tree!



# Search Example: Romania



# Searching with a Search Tree



## ■ Search:

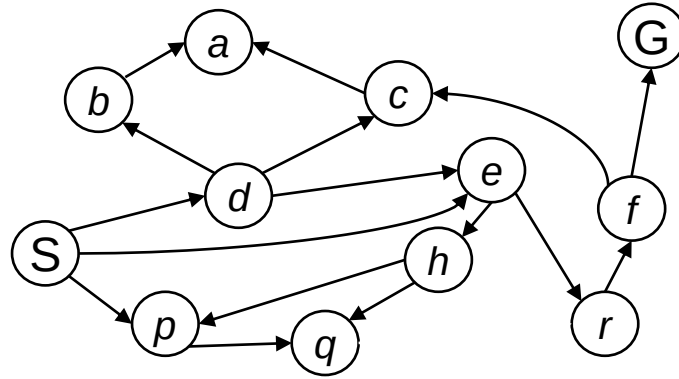
- Expand out potential plans (tree nodes)
- Maintain a **fringe** of partial plans under consideration
- Try to expand as few tree nodes as possible

# General Tree Search

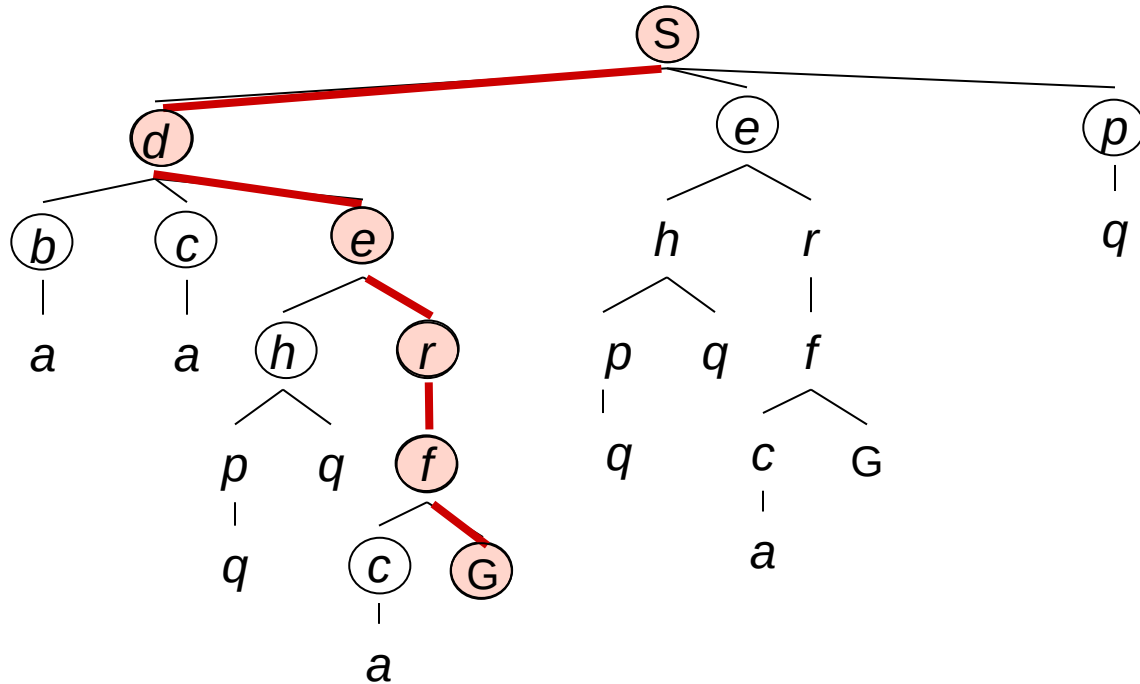
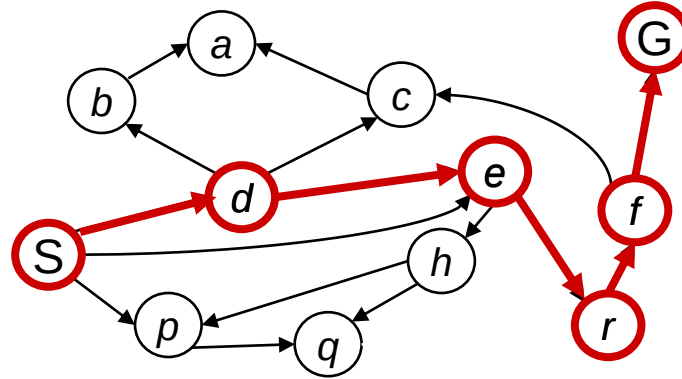
```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

- Important ideas:
  - Fringe
  - Expansion
  - Exploration strategy
- Main question: which fringe nodes to explore?

# Example: Tree Search



# Example: Tree Search



~~s~~  
~~s → d~~  
s → e  
s → p  
s → d → b  
s → d → c  
~~s → d → e~~  
s → d → e → h  
~~s → d → e → r~~  
~~s → d → e → r → f~~  
s → d → e → r → f → c  
~~s → d → e → r → f → G~~

# Depth-First Search

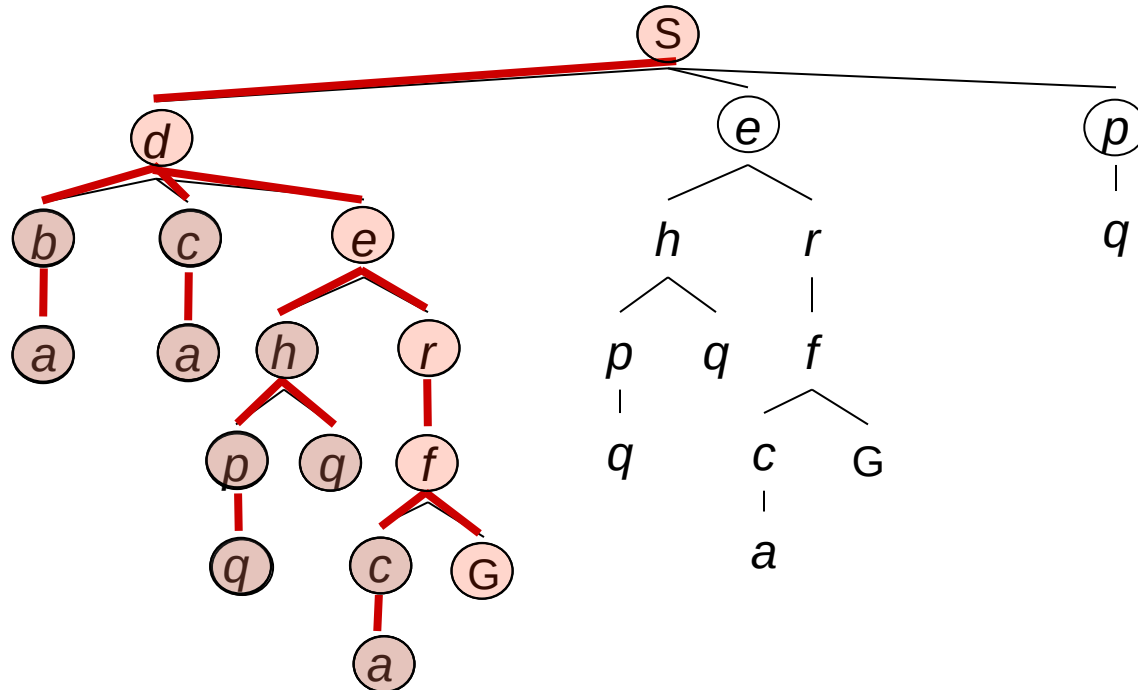
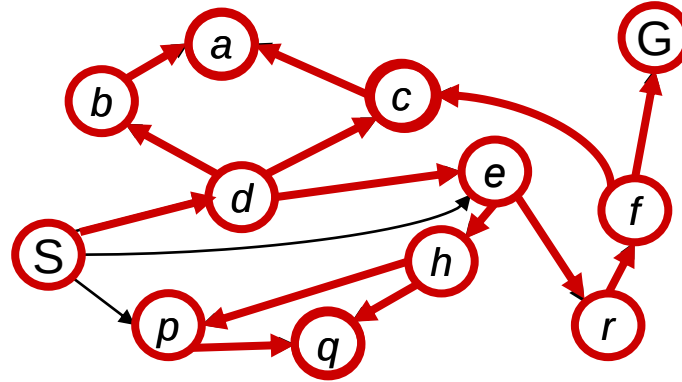
---



# Depth-First Search

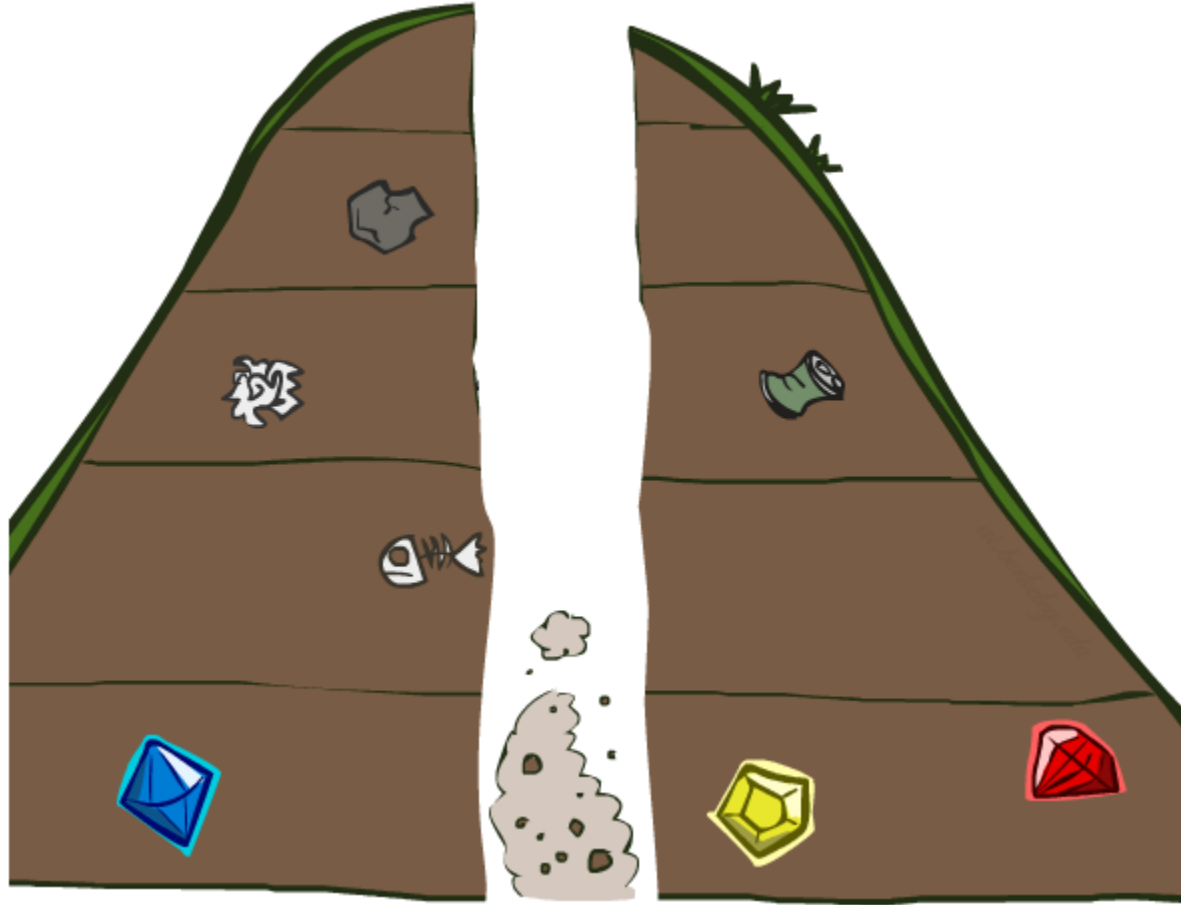
*Strategy: expand a  
deepest node first*

*Implementation:  
Fringe is a LIFO stack*



# Search Algorithm Properties

---



# Search Algorithm Properties

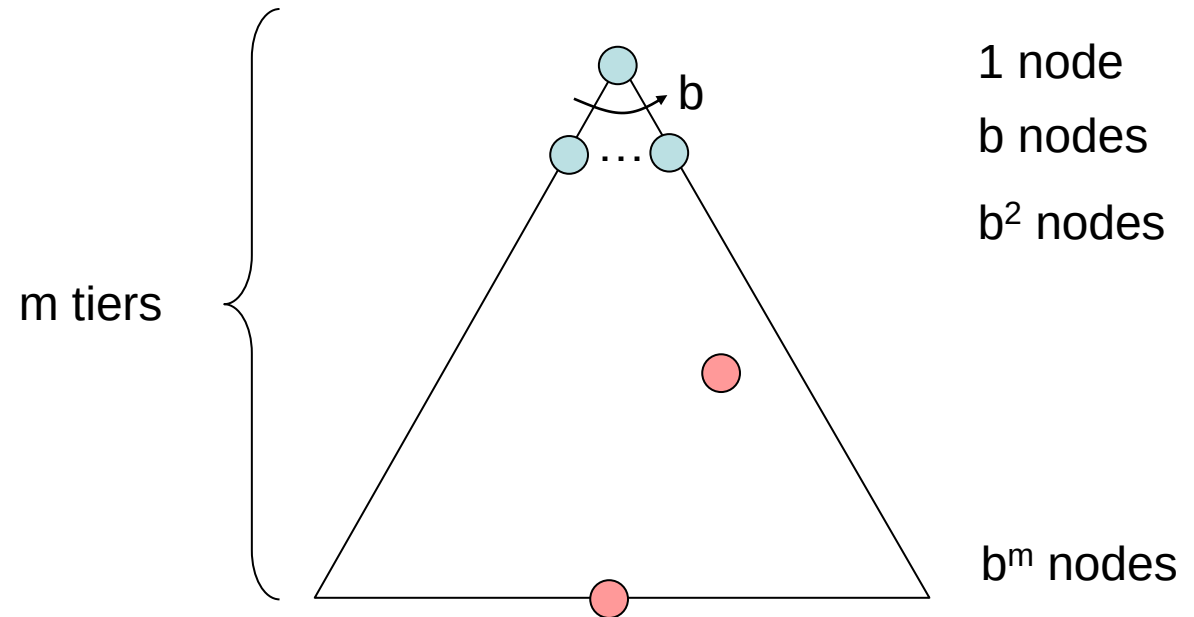
- Complete: Guaranteed to find a solution if one exists?
- Optimal: Guaranteed to find the least cost path?
- Time complexity?
- Space complexity?

- Cartoon of search tree:

- $b$  is the branching factor
- $m$  is the maximum depth
- solutions at various depths

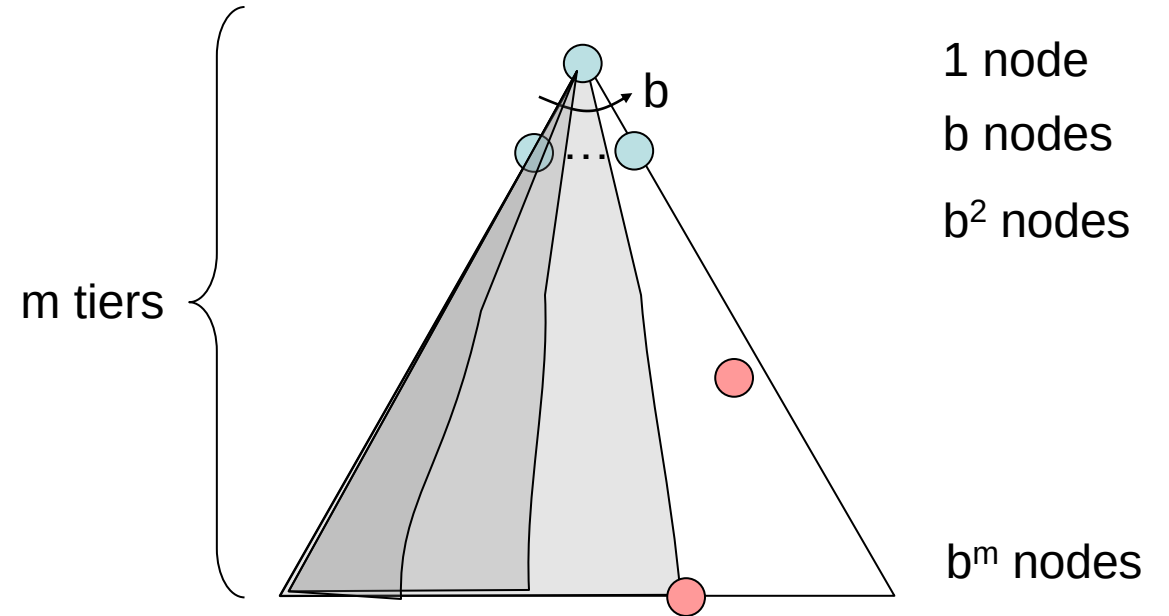
- Number of nodes in entire tree?

- $1 + b + b^2 + \dots + b^m = O(b^{m+1})$



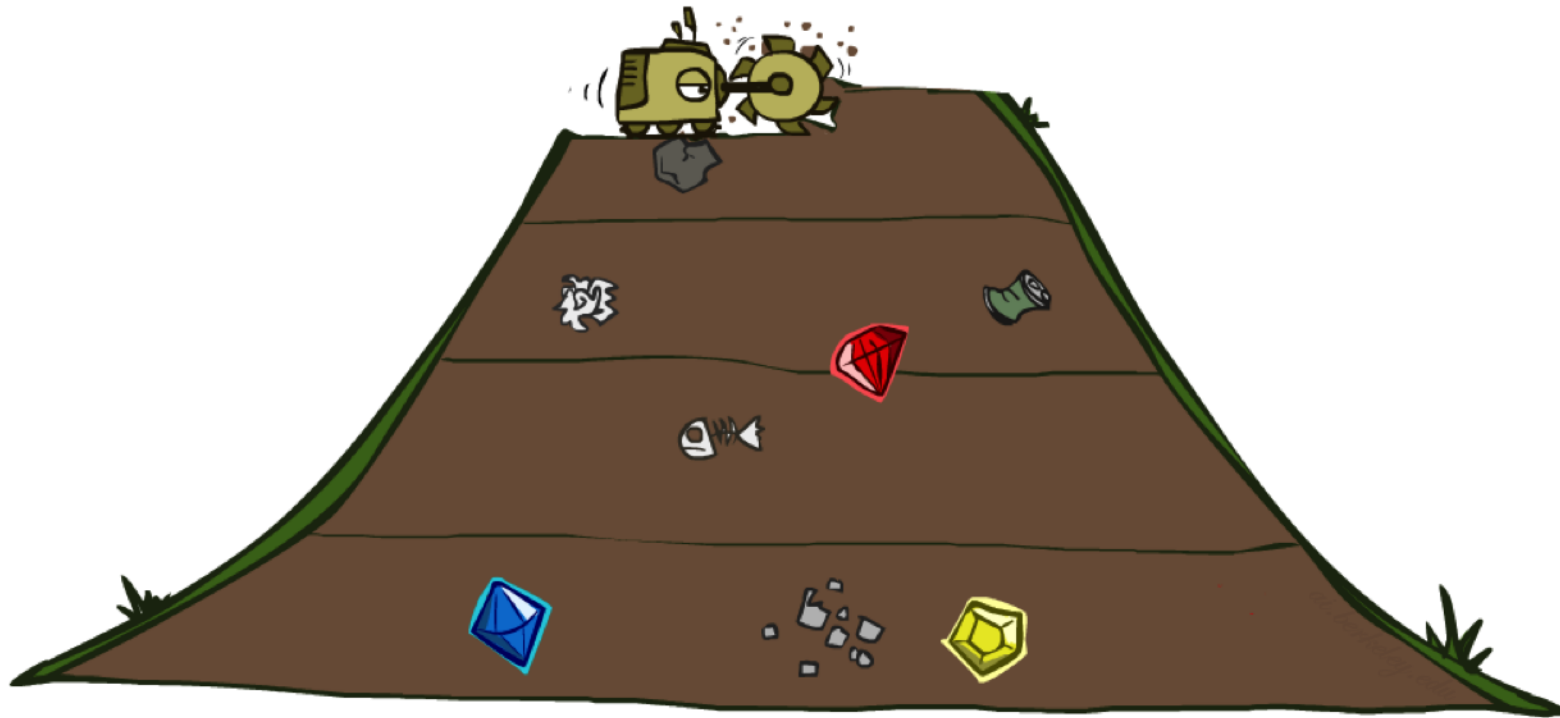
# Depth-First Search (DFS) Properties

- What nodes DFS expand?
  - Some left prefix of the tree.
  - Could process the whole tree!
  - If  $m$  is finite, takes time  $O(b^m)$
- How much space does the fringe take?
  - Only has siblings on path to root, so  $O(bm)$
- Is it complete?
  - $m$  could be infinite, so only if we prevent cycles (more later)
- Is it optimal?
  - No, it finds the “leftmost” solution, regardless of depth or cost



# Breadth-First Search

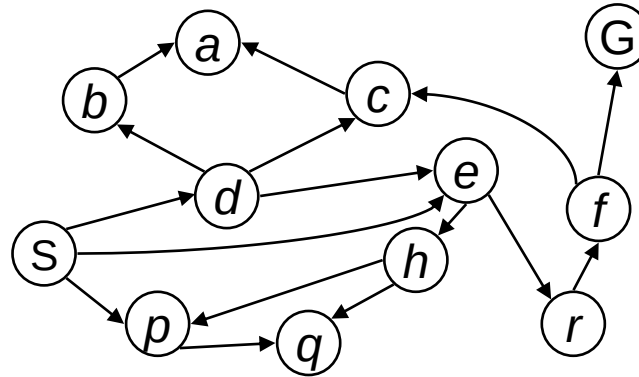
---



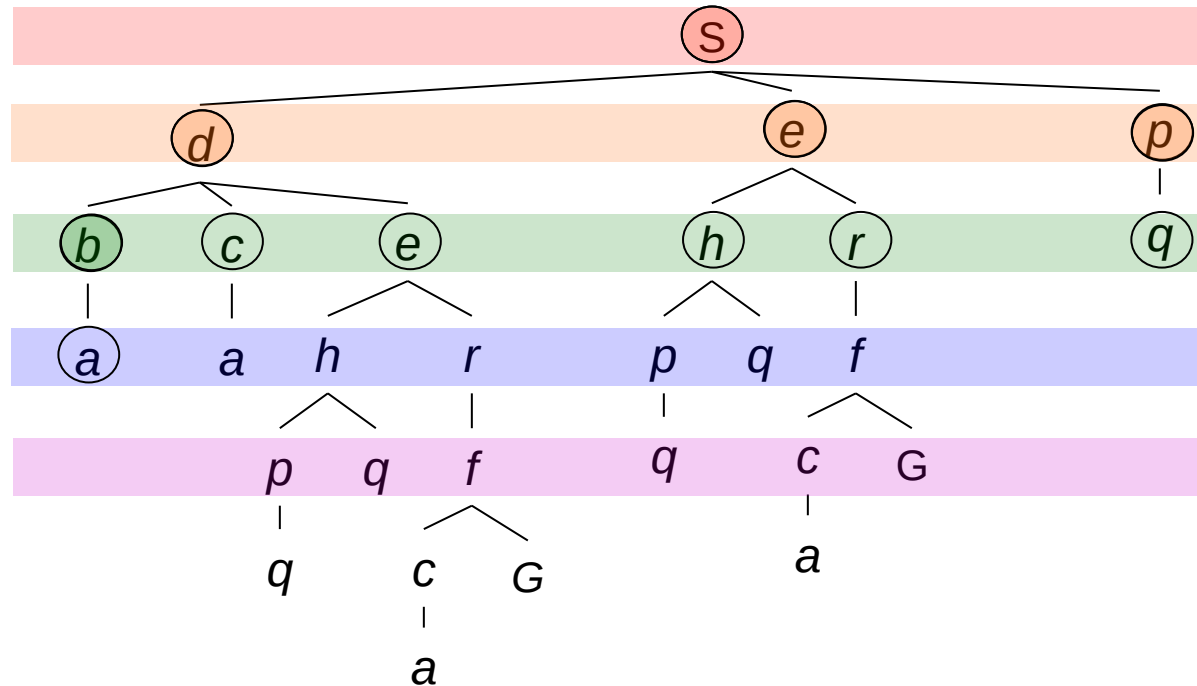
# Breadth-First Search

*Strategy: expand a shallowest node first*

*Implementation: Fringe is a FIFO queue*

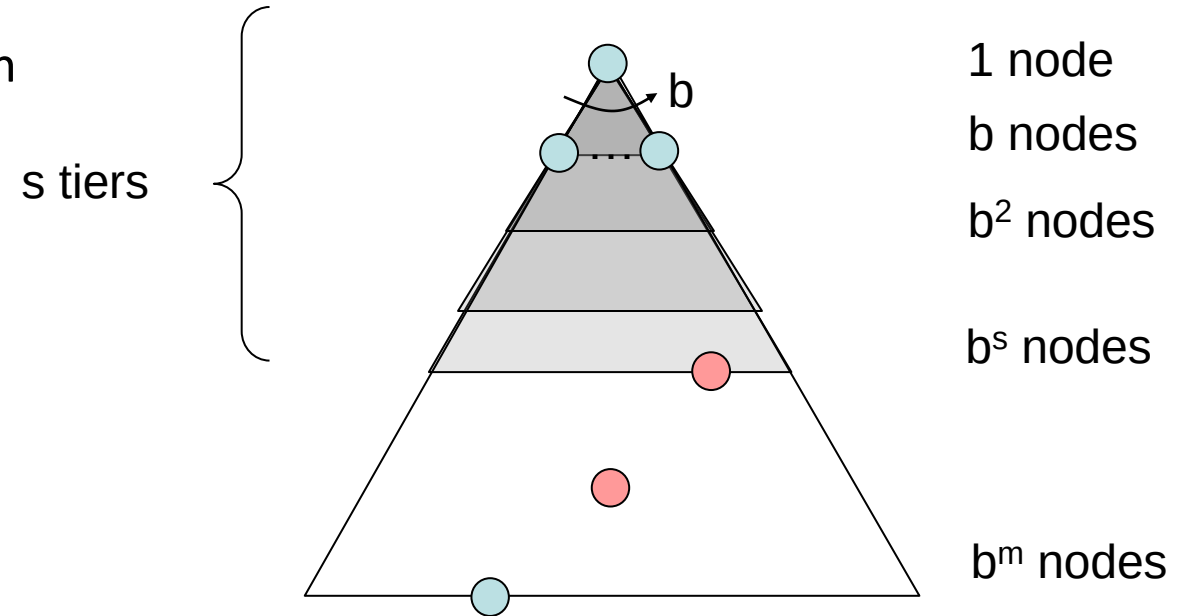


Search  
Tiers

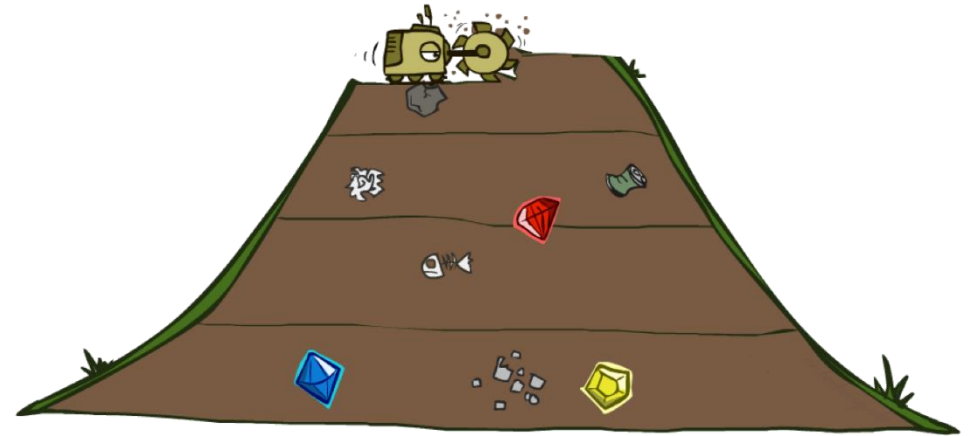


# Breadth-First Search (BFS) Properties

- What nodes does BFS expand?
  - Processes all nodes above shallowest solution
  - Let depth of shallowest solution be  $s$
  - Search takes time  $O(b^s)$
- How much space does the fringe take?
  - Has roughly the last tier, so  $O(b^s)$
- Is it complete?
  - $s$  must be finite if a solution exists, so yes!
- Is it optimal?
  - Only if costs are all 1 (more on costs later)



# Quiz: DFS vs BFS



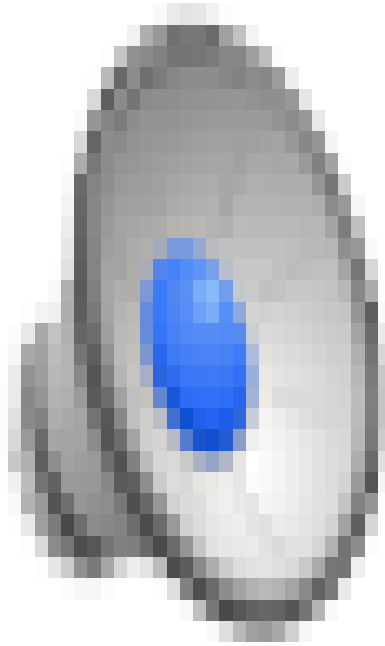
# Quiz: DFS vs BFS

---

- When will BFS outperform DFS?
- When will DFS outperform BFS?

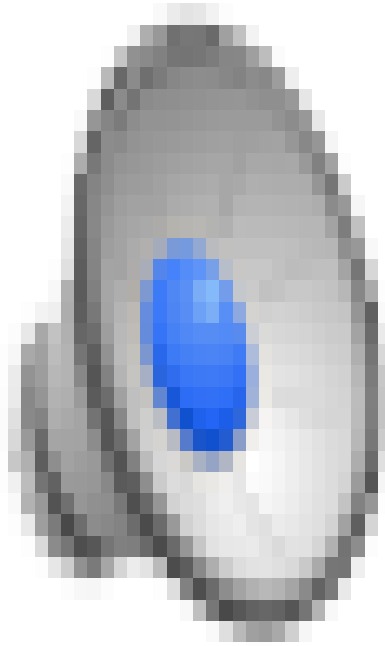
# Video of Demo Maze Water DFS/BFS (part 1)

---



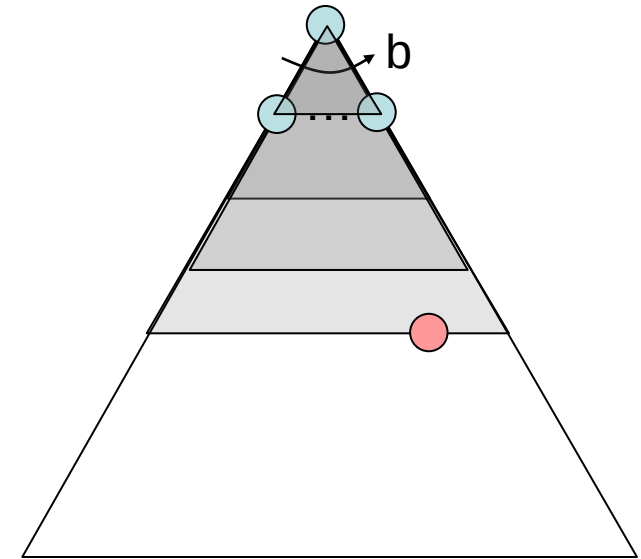
# Video of Demo Maze Water DFS/BFS (part 2)

---

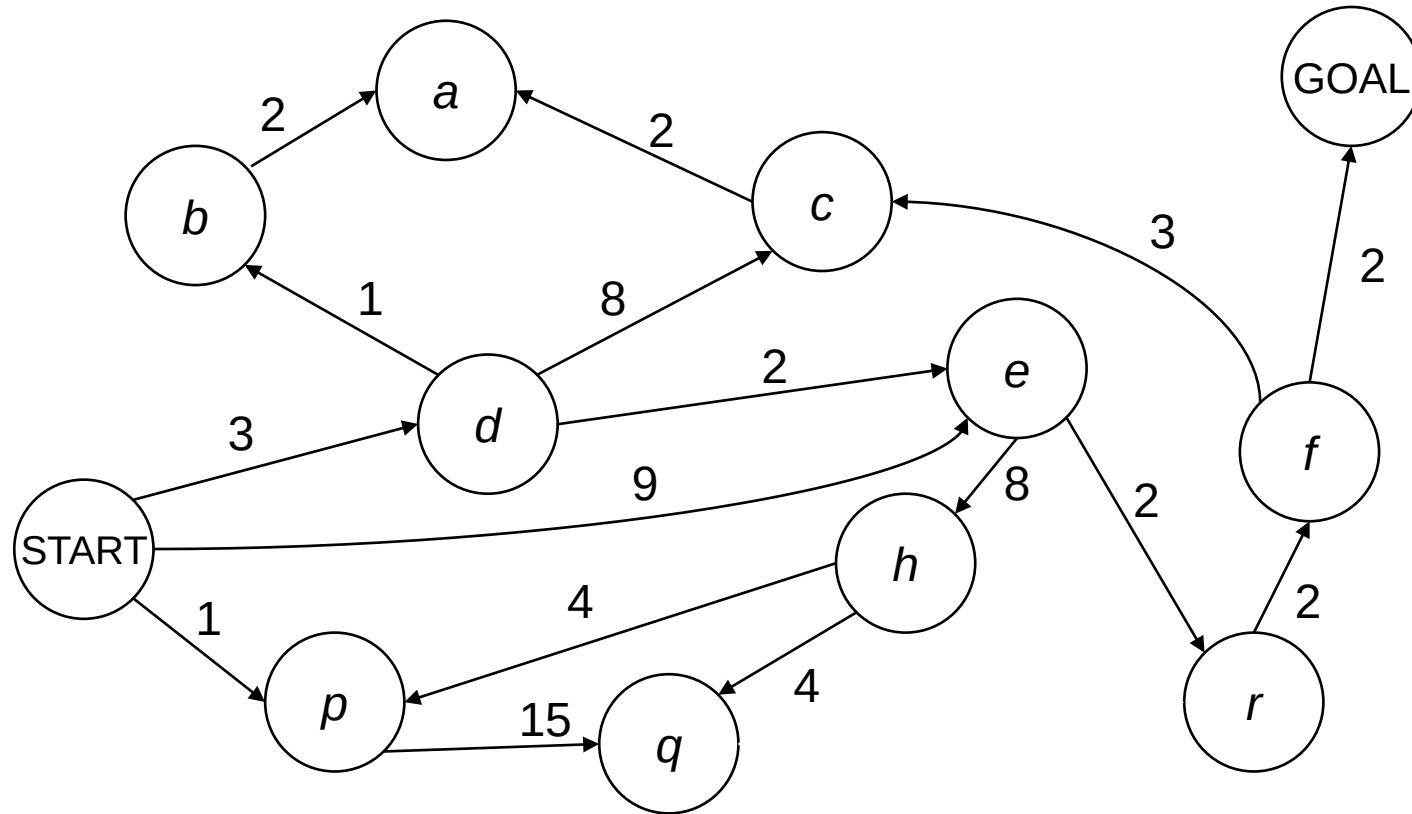


# Iterative Deepening

- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
  - Run a DFS with depth limit 1. If no solution...
  - Run a DFS with depth limit 2. If no solution...
  - Run a DFS with depth limit 3. ....
- Isn't that wastefully redundant?
  - Generally most work happens in the lowest level searched, so not so bad!



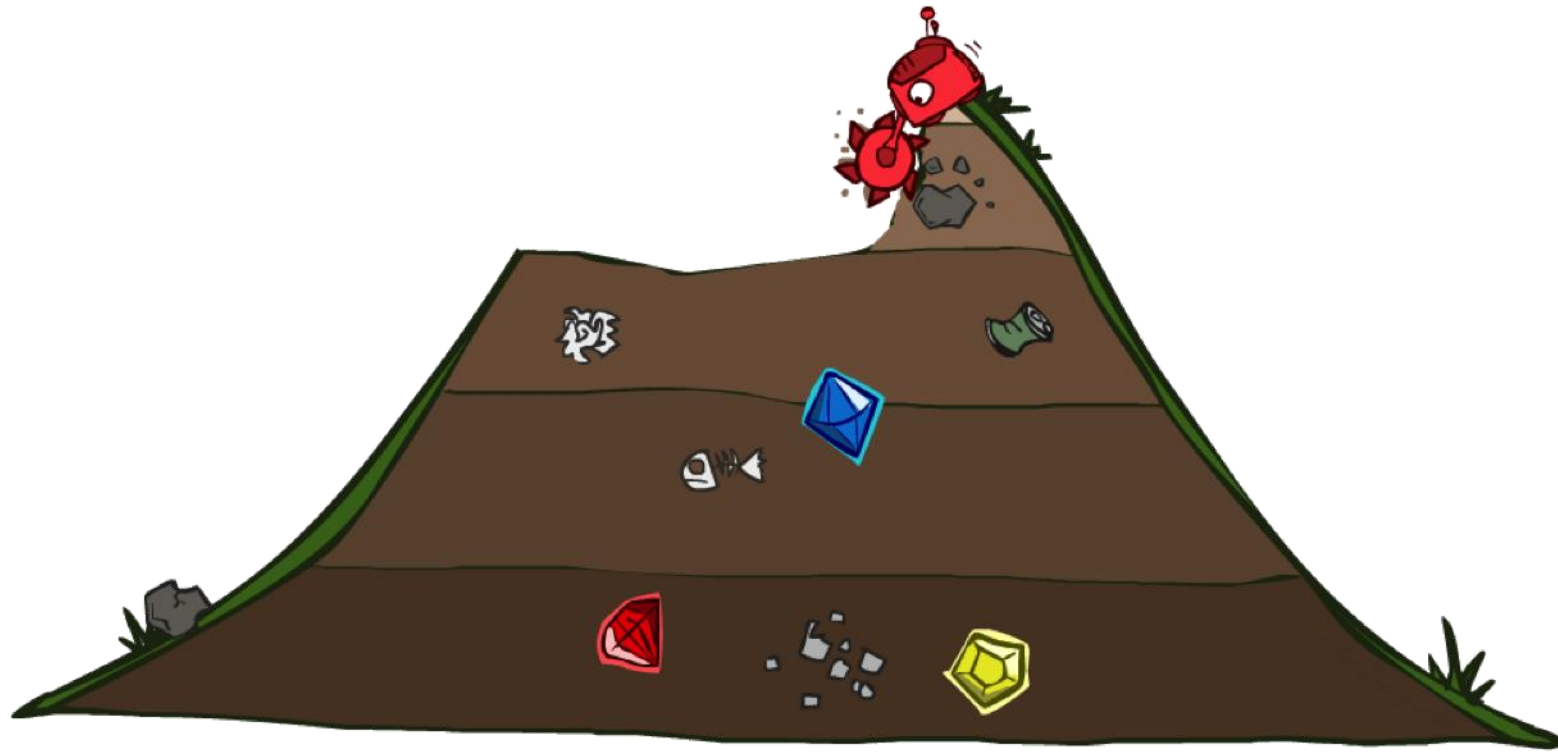
# Cost-Sensitive Search



BFS finds the shortest path in terms of number of actions.  
It does not find the least-cost path. We will now cover  
a similar algorithm which does find the least-cost path.

# Uniform Cost Search

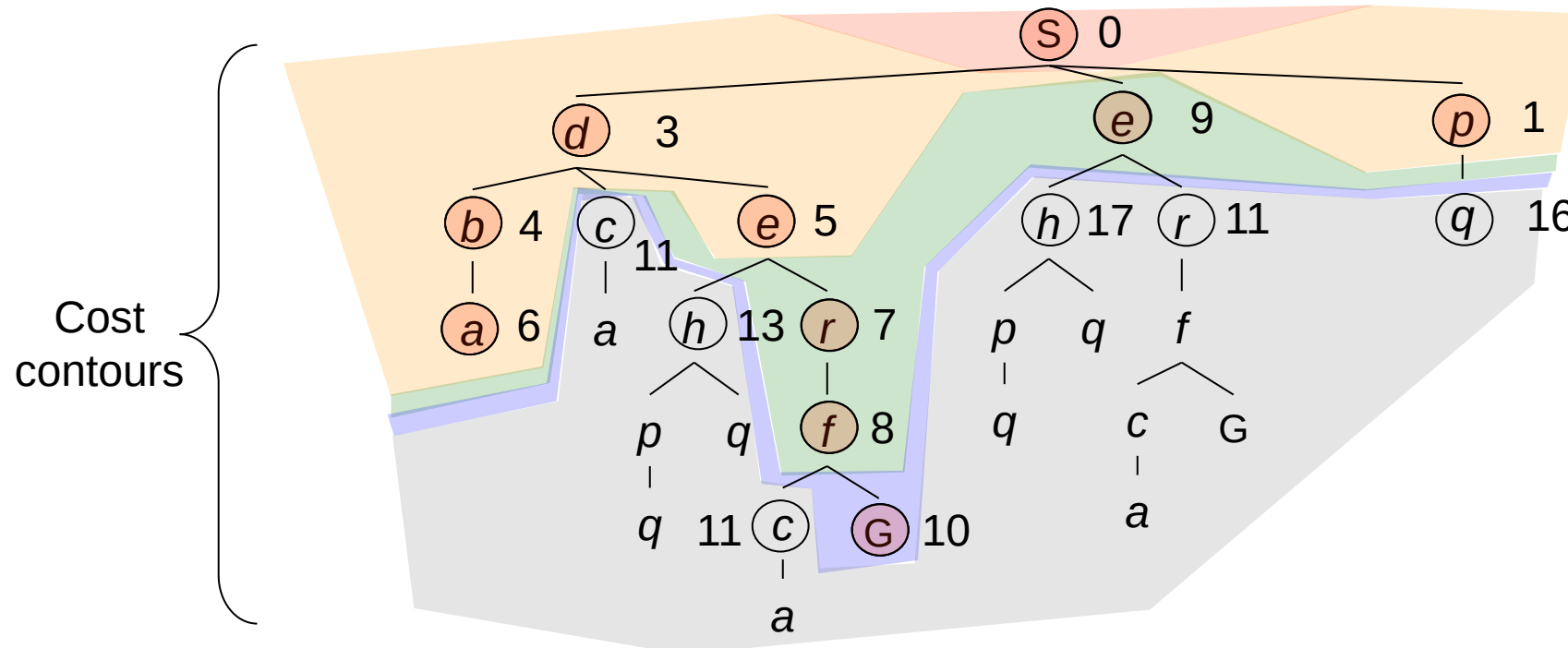
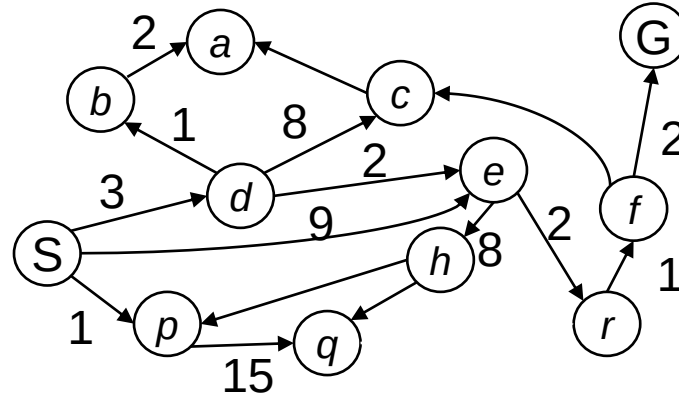
---



# Uniform Cost Search

Strategy: expand a  
cheapest node first:

Fringe is a priority queue  
(priority: cumulative cost)



# Uniform Cost Search (UCS) Properties

- What nodes does UCS expand?

- Processes all nodes with cost less than cheapest solution!
- If that solution costs  $C^*$  and arcs cost at least  $\varepsilon$ , then the “effective depth” is roughly  $C^*/\varepsilon$
- Takes time  $O(b^{C^*/\varepsilon})$  (exponential in effective depth)

- How much space does the fringe take?

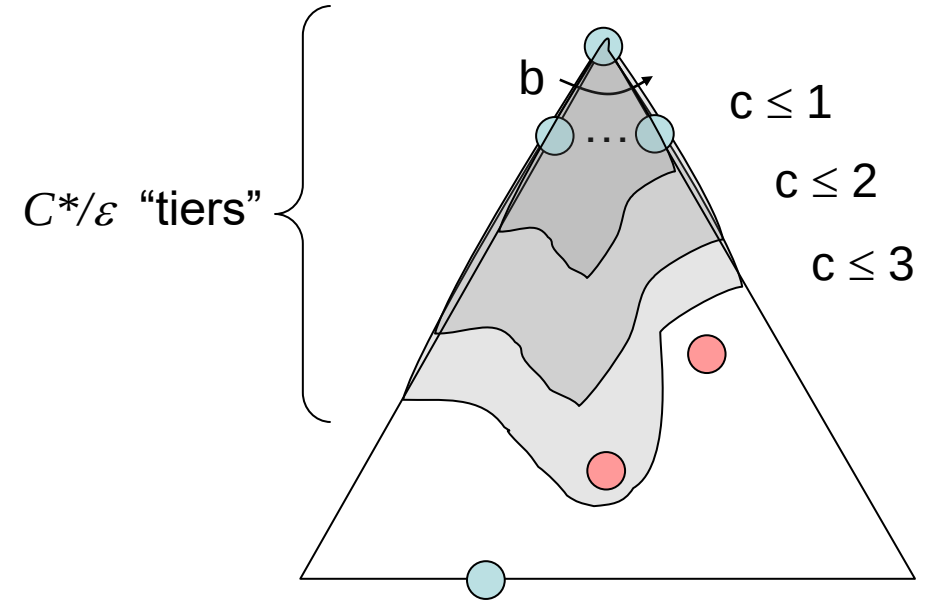
- Has roughly the last tier, so  $O(b^{C^*/\varepsilon})$

- Is it complete?

- Assuming best solution has a finite cost and minimum arc cost is positive, yes!

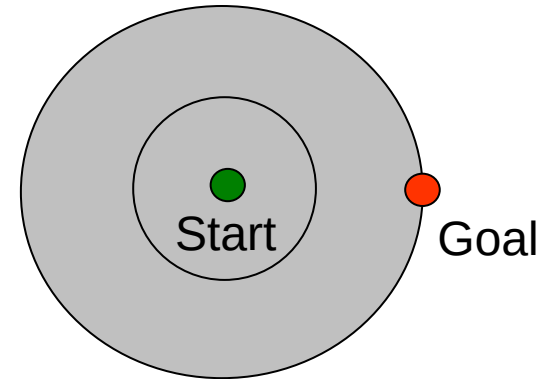
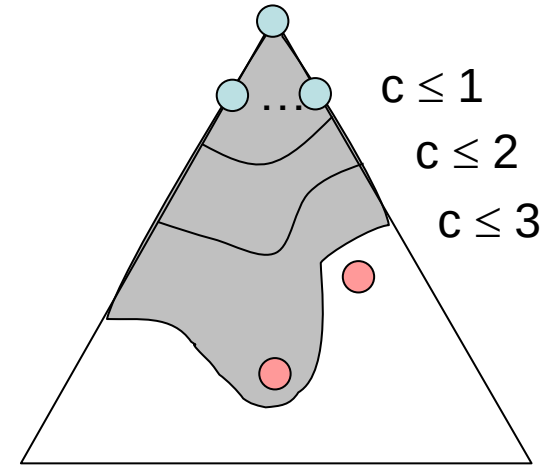
- Is it optimal?

- Yes! (Proof follows later via  $A^*$ )



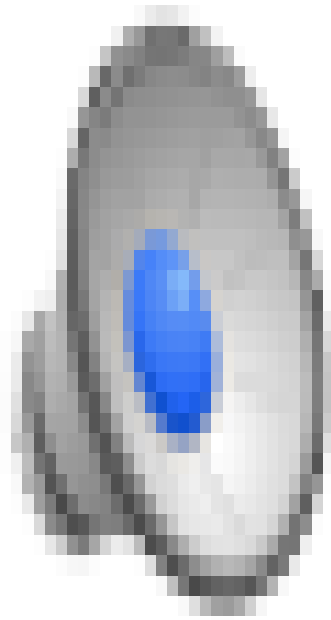
# Uniform Cost Issues

- Remember: UCS explores increasing cost contours
- The good: UCS is complete and optimal!
- The bad:
  - Explores options in every “direction”
  - No information about goal location
- We'll fix that soon!



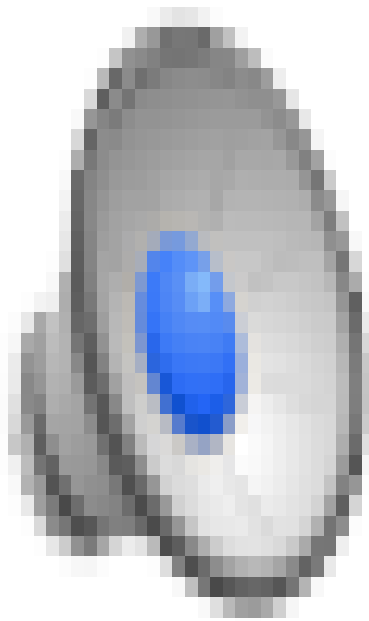
# Video of Demo Empty UCS

---



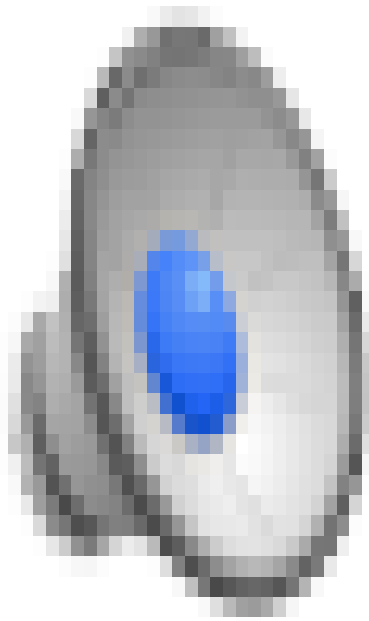
# Video of Demo Maze with Deep/Shallow Water --- DFS, BFS, or UCS? (part 1)

---



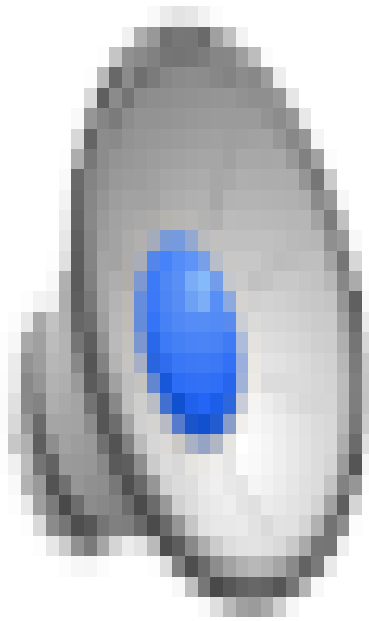
## Video of Demo Maze with Deep/Shallow Water --- DFS, BFS, or UCS? (part 2)

---



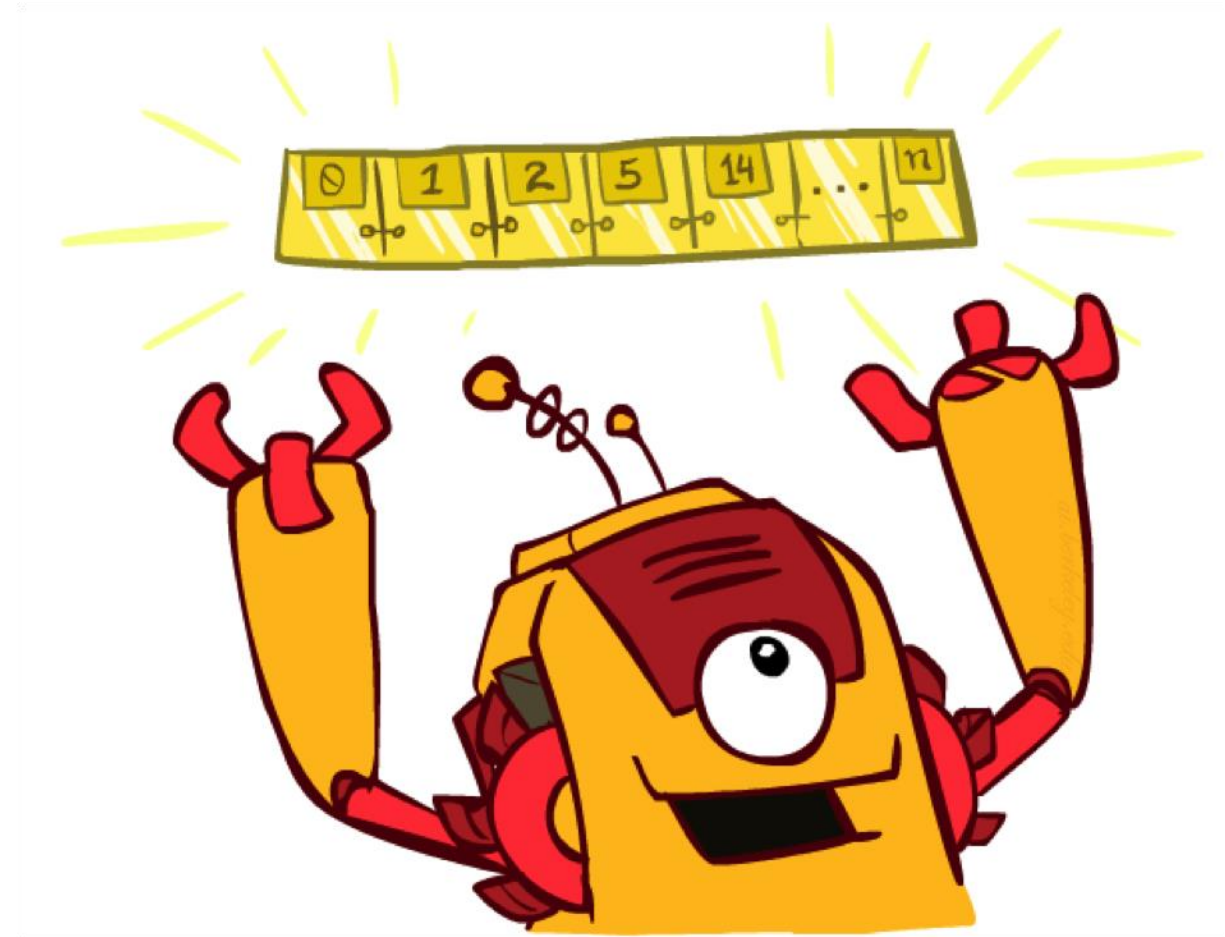
## Video of Demo Maze with Deep/Shallow Water --- DFS, BFS, or UCS? (part 3)

---



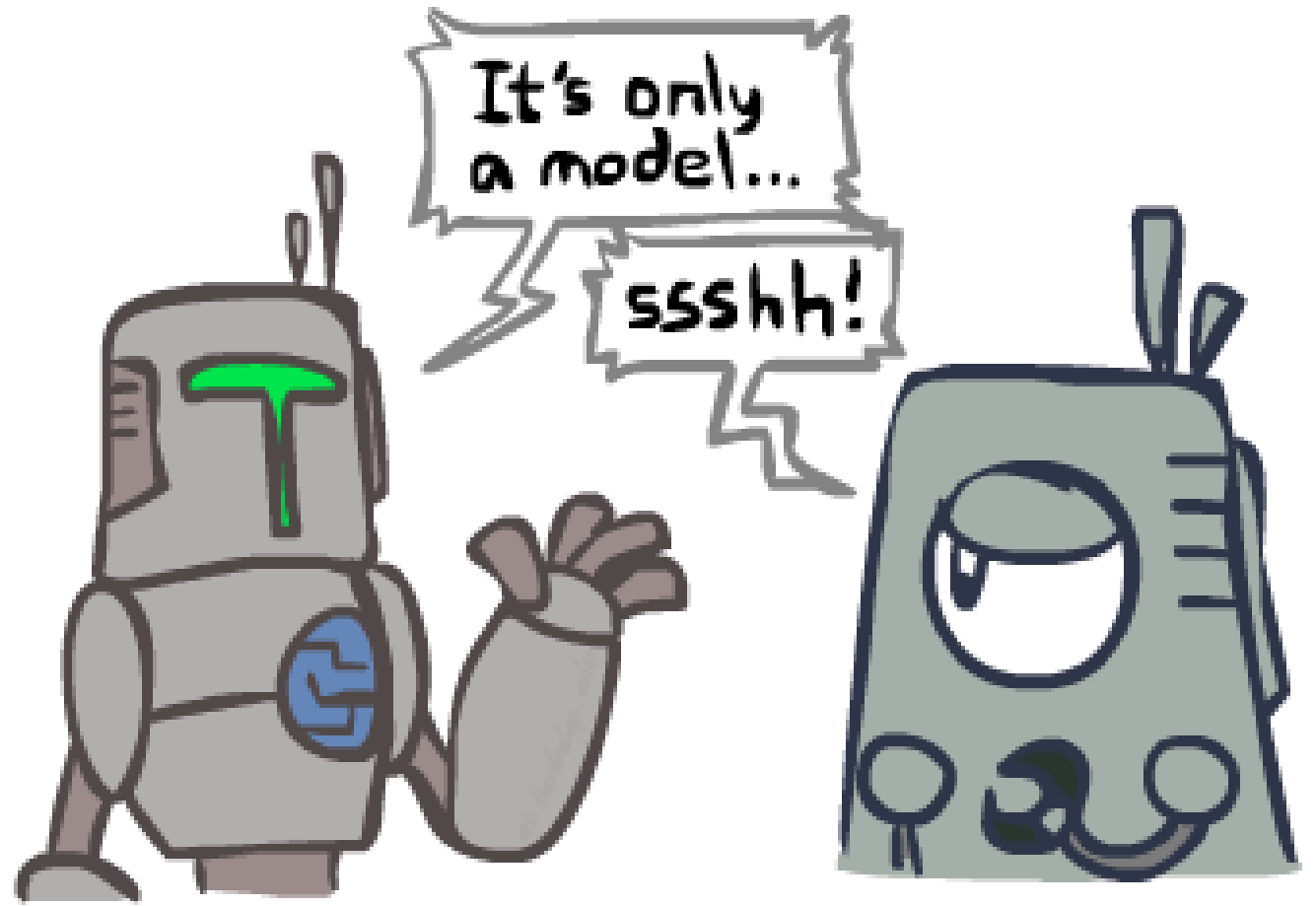
# The One Queue

- All these search algorithms are the same except for fringe strategies
  - Conceptually, all fringes are priority queues (i.e. collections of nodes with attached priorities)
  - Practically, for DFS and BFS, you can avoid the  $\log(n)$  overhead from an actual priority queue, by using stacks and queues
  - Can even code one implementation that takes a variable queuing object

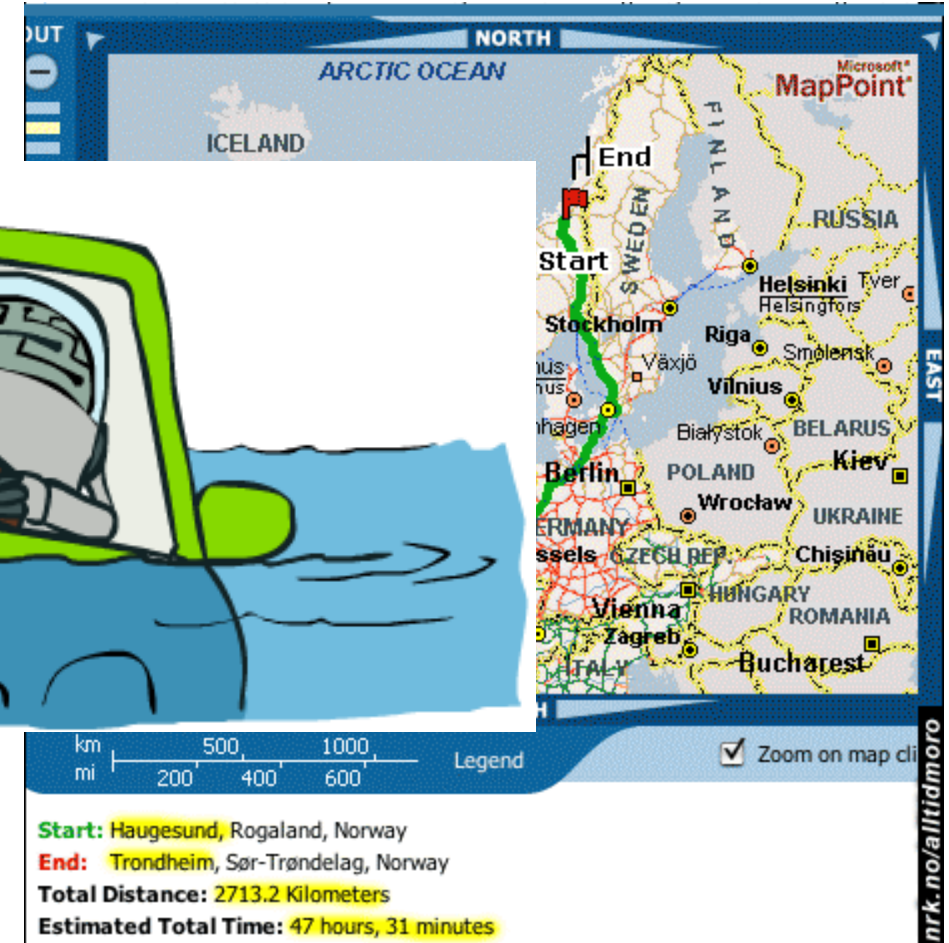
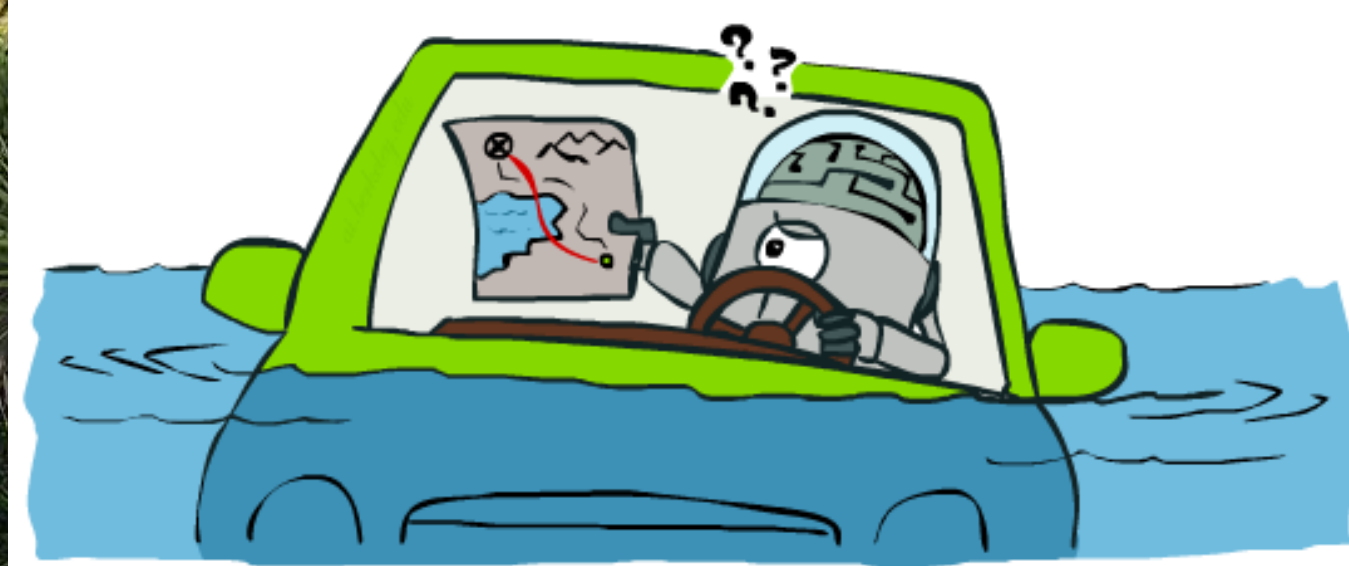
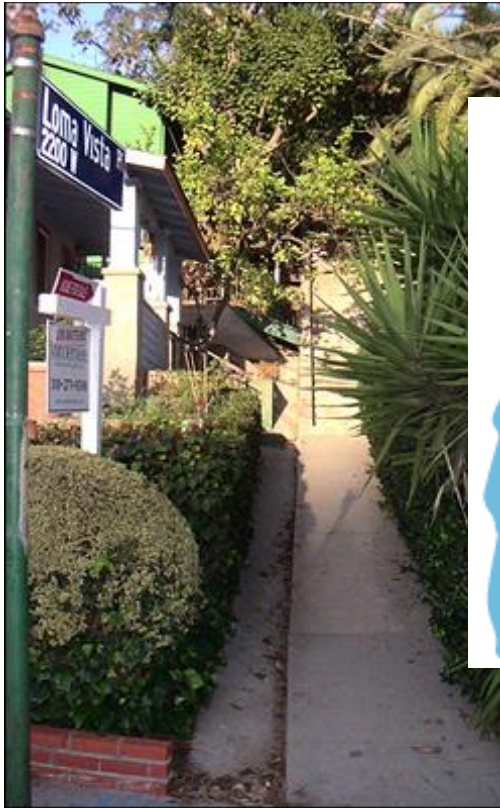


# Search and Models

- Search operates over models of the world
  - The agent doesn't actually try all the plans out in the real world!
  - Planning is all “in simulation”
  - Your search is only as good as your models...

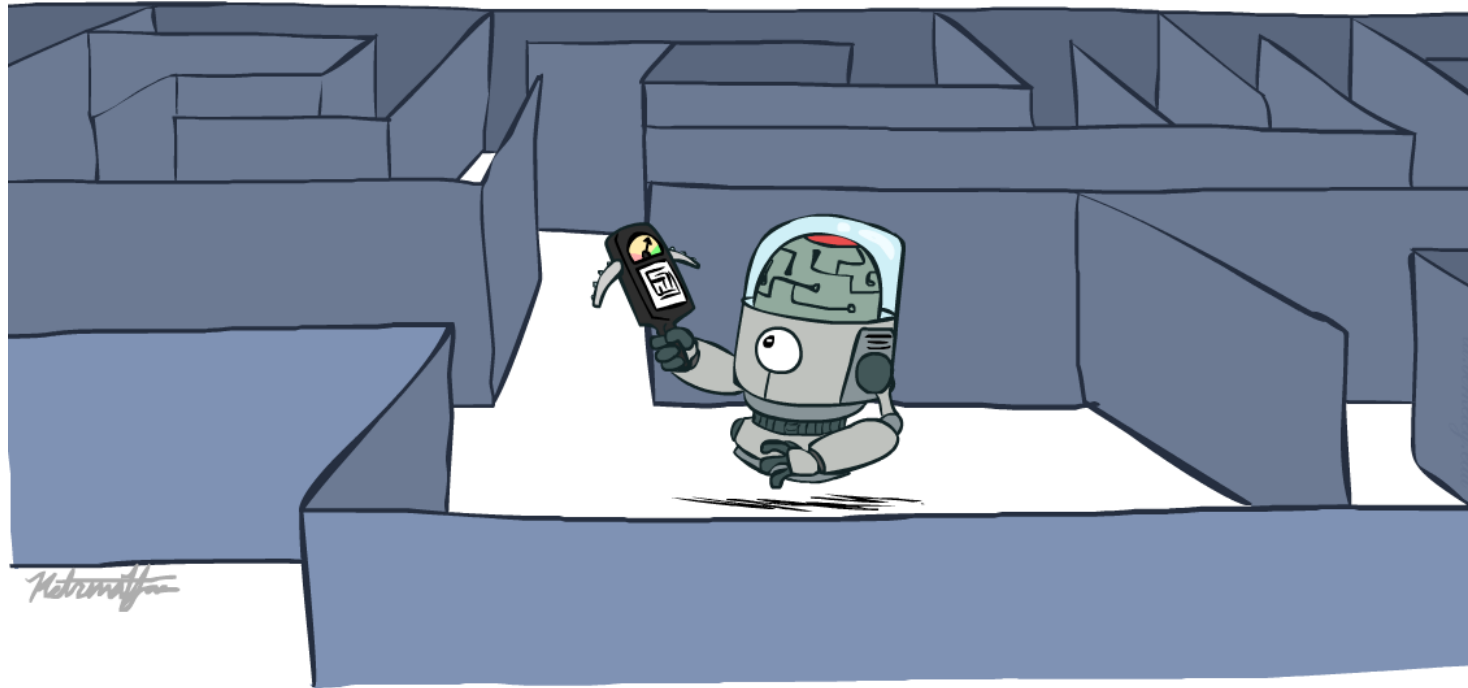


# Search Gone Wrong?



# Artificial Intelligence

## Part 2 - Informed Search



[These slides are a minor adaptation of the slides that were created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley ([ai.berkeley.edu](http://ai.berkeley.edu)).]

# Today – Part 2

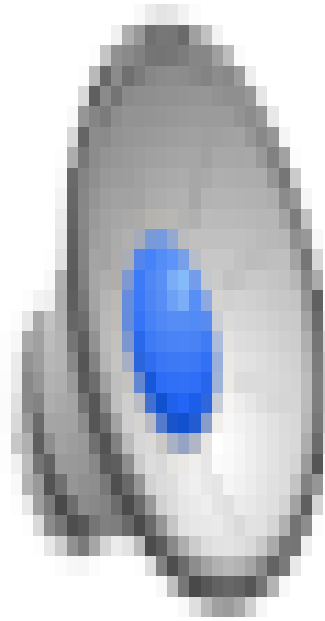
---

- Informed Search
  - Heuristics
  - Greedy Search
  - A\* Search
- Graph Search



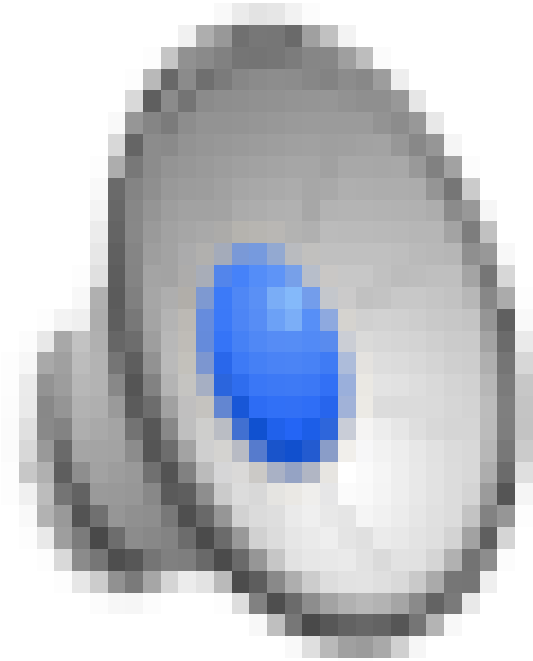
# Uninformed Search : Packman

---



# Uninformed Search : Packman

---



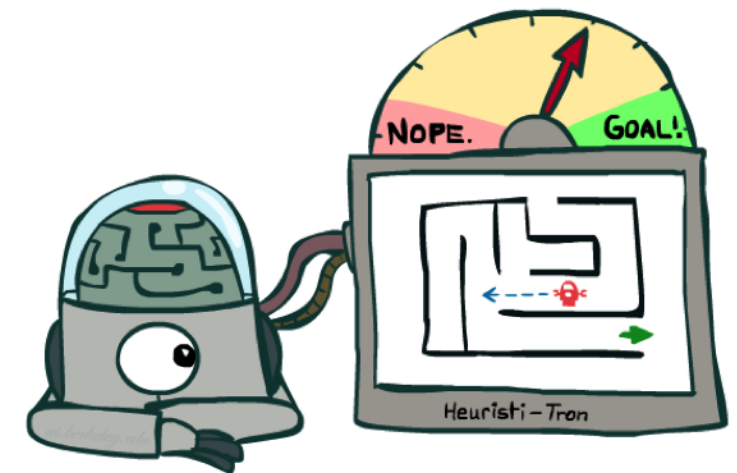
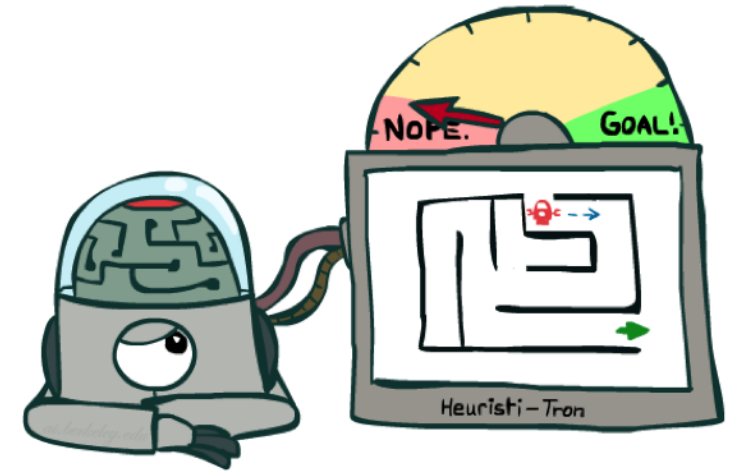
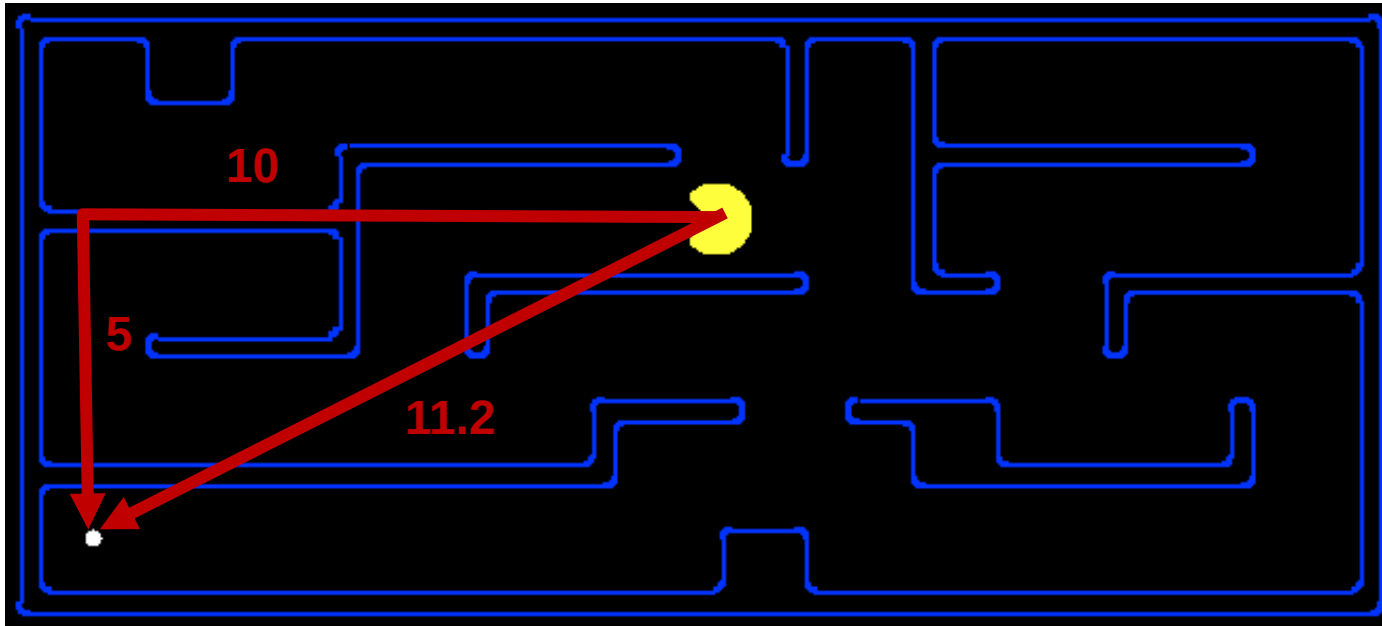
# Informed Search

---

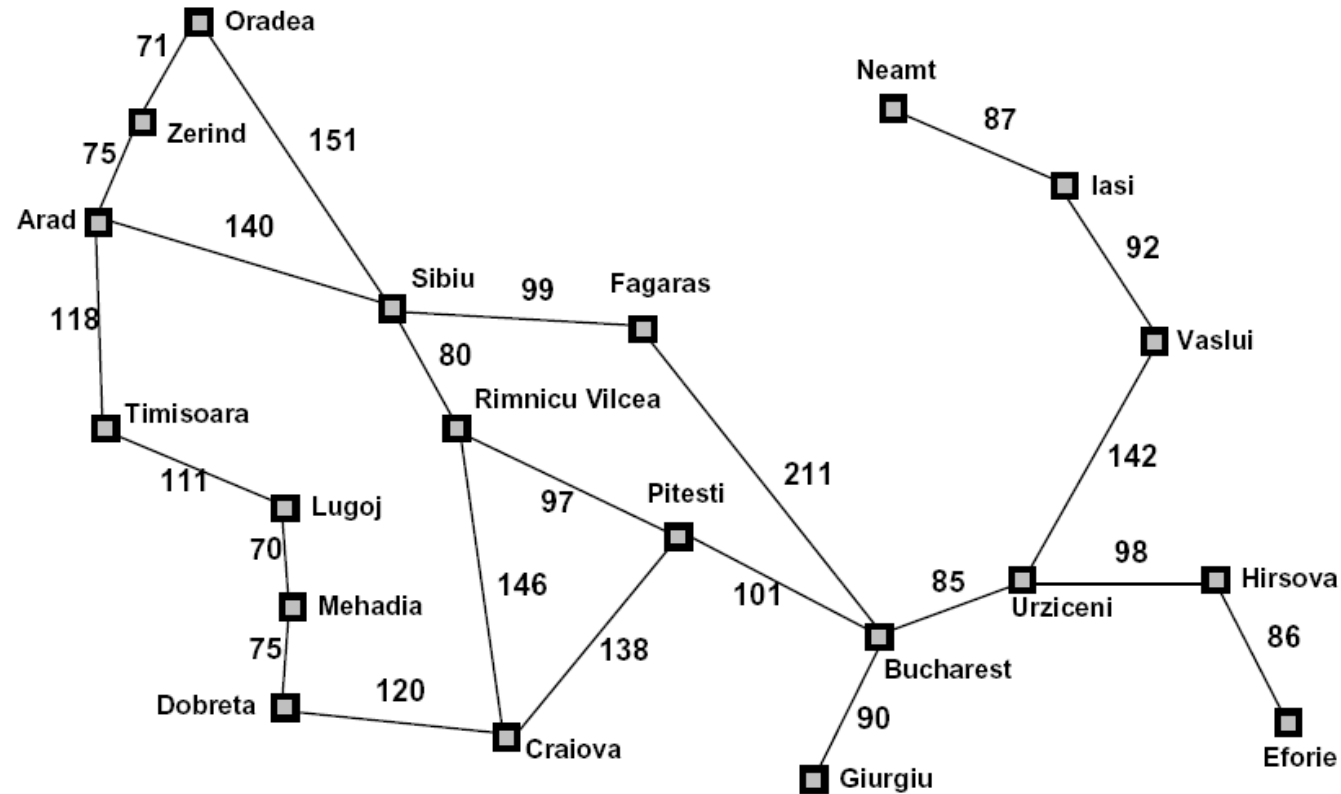


# Search Heuristics

- A heuristic is:
  - A function that *estimates* how close a state is to a goal
  - Designed for a particular search problem
  - Examples: Manhattan distance, Euclidean distance for pathing



# Example: Heuristic Function



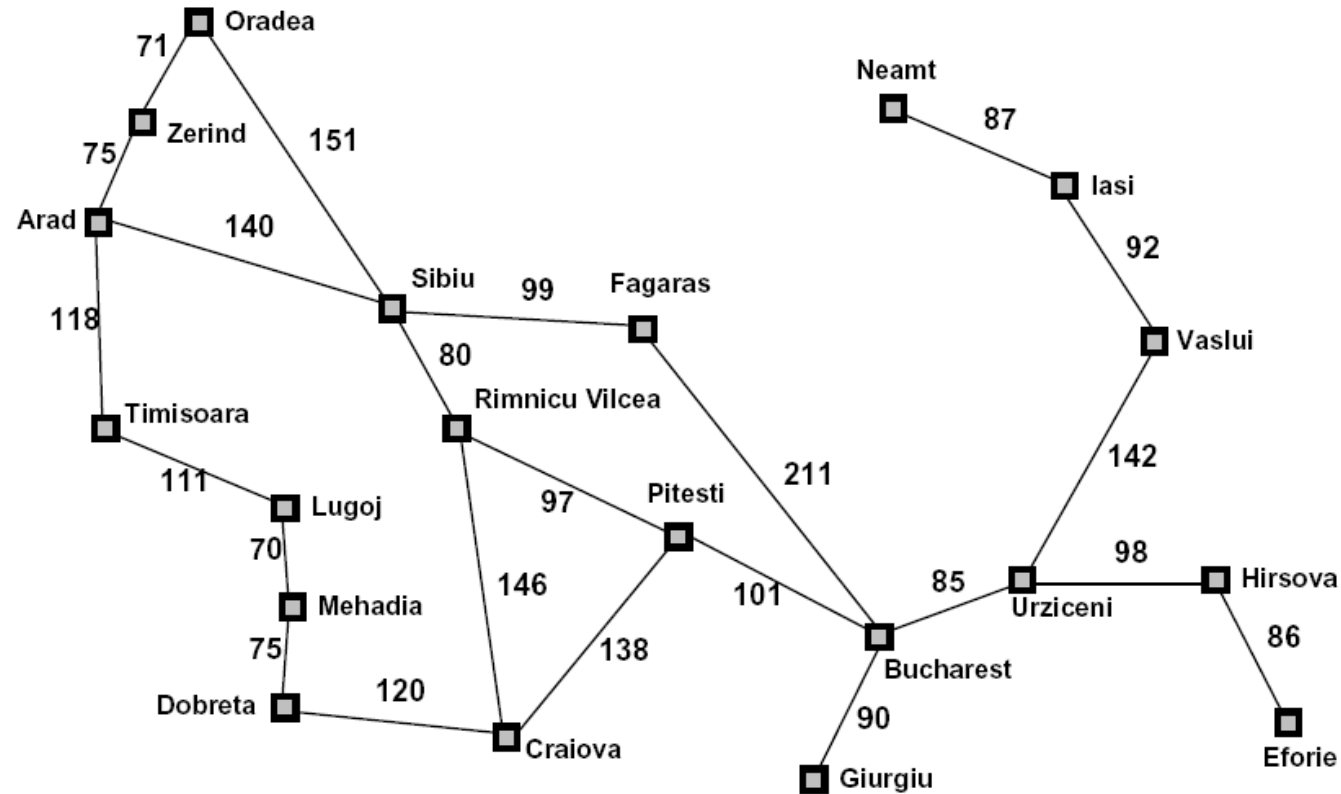
| Straight-line distance<br>to Bucharest |     |
|----------------------------------------|-----|
| Arad                                   | 366 |
| Bucharest                              | 0   |
| Craiova                                | 160 |
| Dobreta                                | 242 |
| Eforie                                 | 161 |
| Fagaras                                | 178 |
| Giurgiu                                | 77  |
| Hirsova                                | 151 |
| Iasi                                   | 226 |
| Lugoj                                  | 244 |
| Mehadia                                | 241 |
| Neamt                                  | 234 |
| Oradea                                 | 380 |
| Pitesti                                | 98  |
| Rimnicu Vilcea                         | 193 |
| Sibiu                                  | 253 |
| Timisoara                              | 329 |
| Urziceni                               | 80  |
| Vaslui                                 | 199 |
| Zerind                                 | 374 |

$h(x)$

# Greedy Search



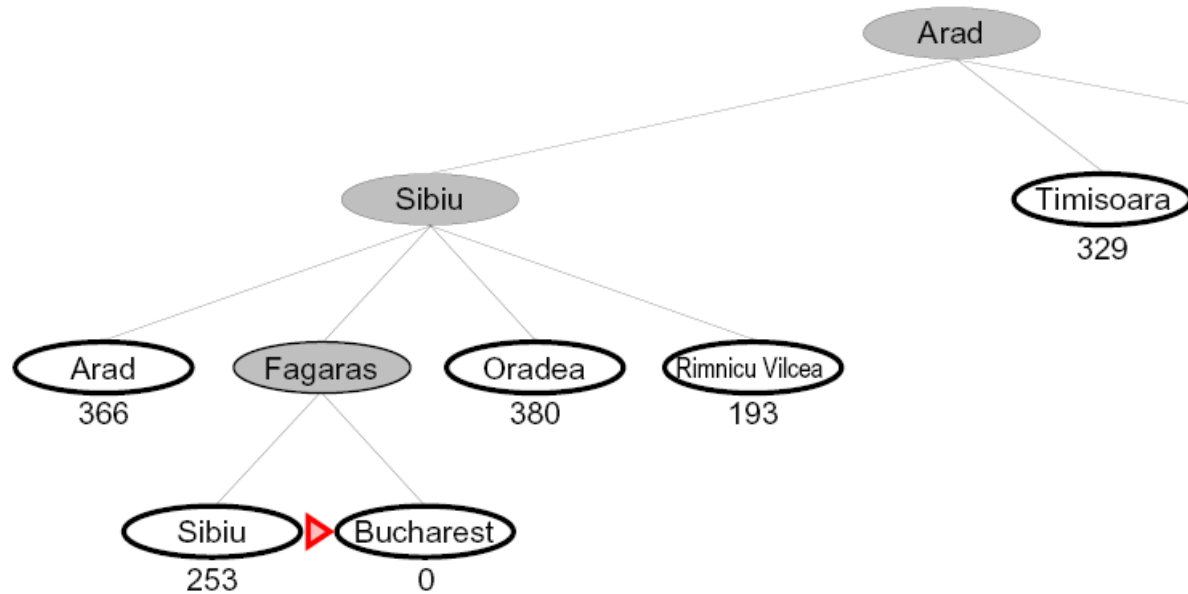
# Example: Heuristic Function



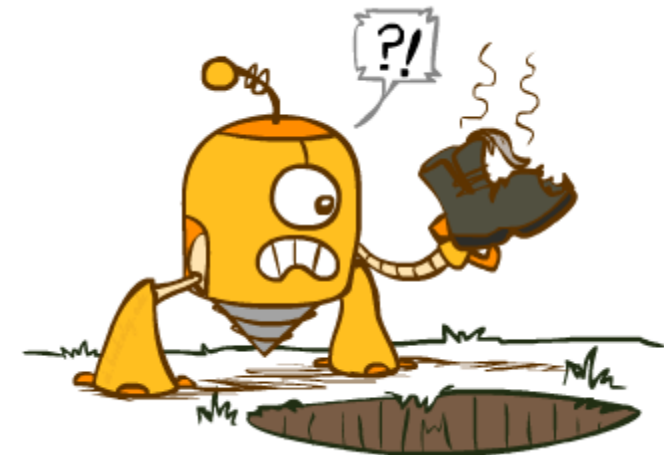
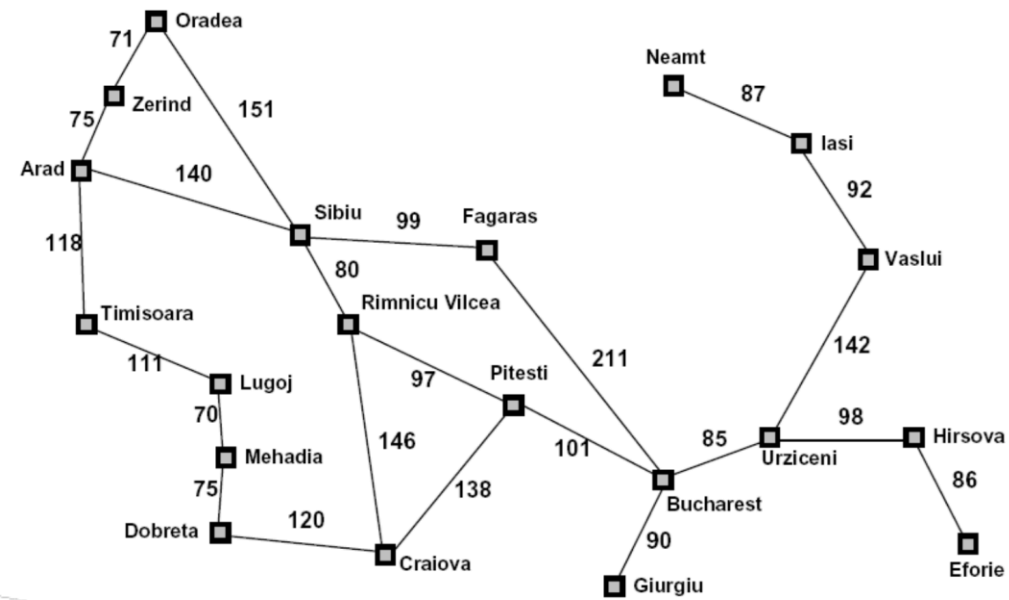
| Straight-line distance<br>to Bucharest |     |
|----------------------------------------|-----|
| Arad                                   | 366 |
| Bucharest                              | 0   |
| Craiova                                | 160 |
| Dobreta                                | 242 |
| Eforie                                 | 161 |
| Fagaras                                | 178 |
| Giurgiu                                | 77  |
| Hirsova                                | 151 |
| Iasi                                   | 226 |
| Lugoj                                  | 244 |
| Mehadia                                | 241 |
| Neamt                                  | 234 |
| Oradea                                 | 380 |
| Pitesti                                | 98  |
| Rimnicu Vilcea                         | 193 |
| Sibiu                                  | 253 |
| Timisoara                              | 329 |
| Urziceni                               | 80  |
| Vaslui                                 | 199 |
| Zerind                                 | 374 |

$h(x)$

- Expand the node that seems closest...

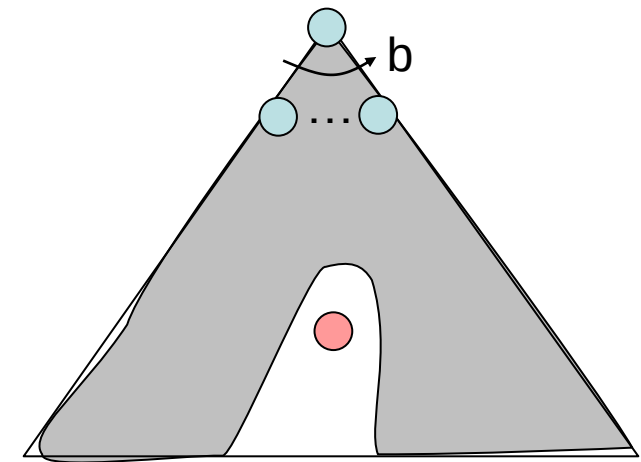
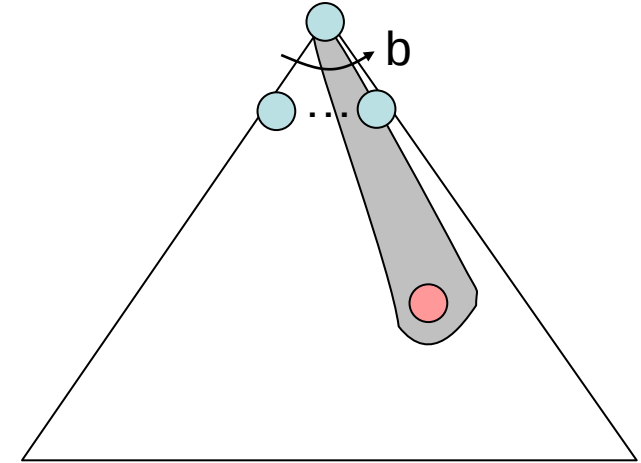


- What can go wrong?



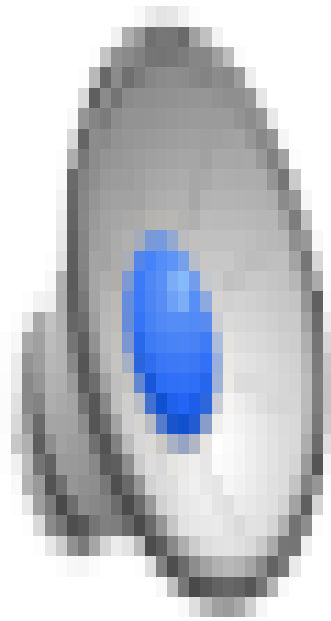
# Greedy Search

- Strategy: expand a node that you think is closest to a goal state
  - Heuristic: estimate of distance to nearest goal for each state
- A common case:
  - Best-first takes you straight to the (wrong) goal
- Worst-case: like a badly-guided DFS



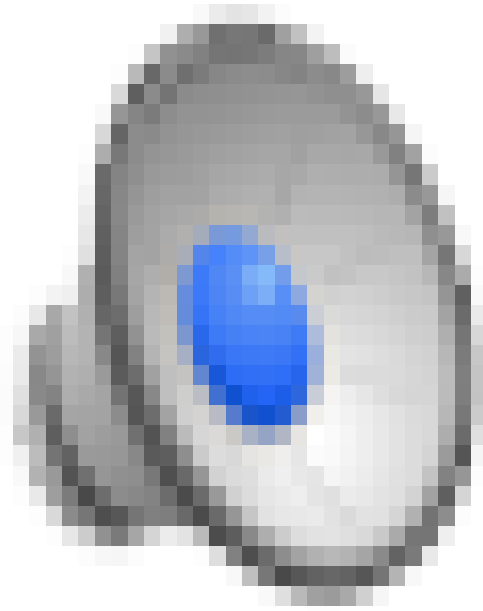
# Video of Demo Contours Greedy (Empty)

---



# Video of Demo Contours Greedy (Pacman Small Maze)

---



# A\* Search

---

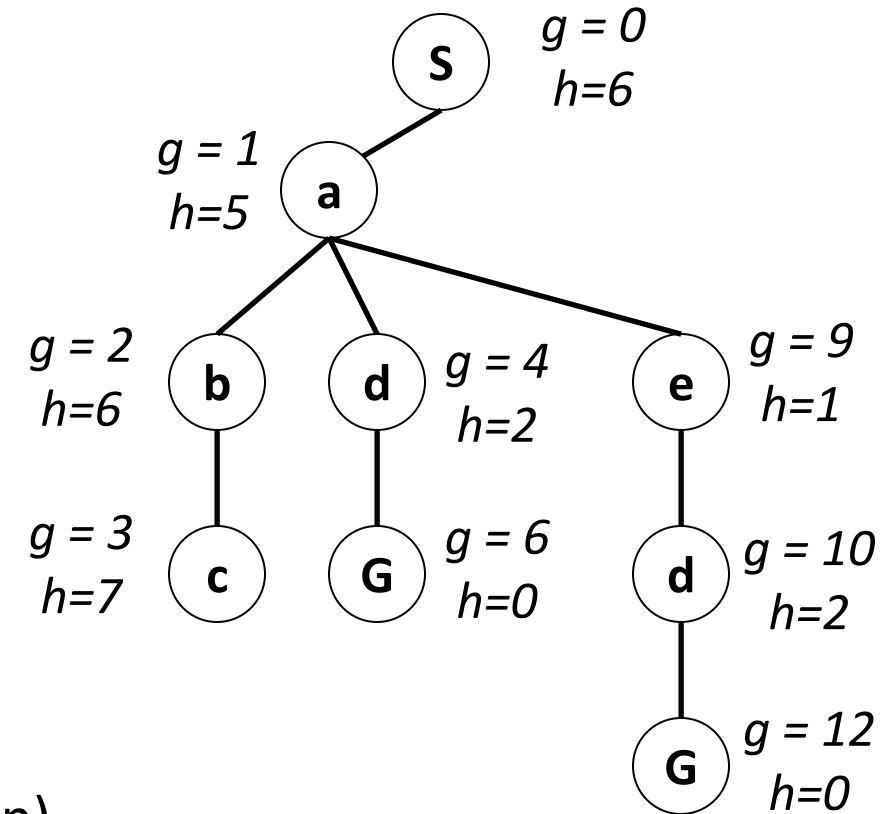
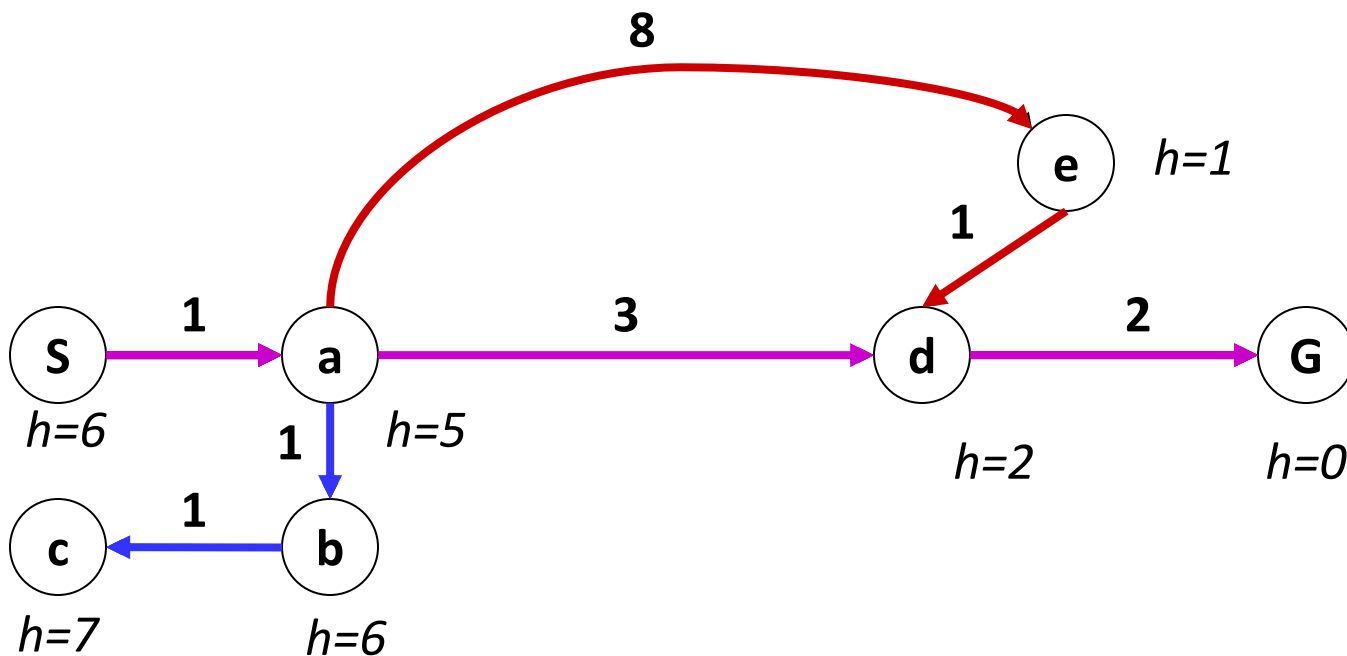


# A\* Search

---

# Combining UCS and Greedy

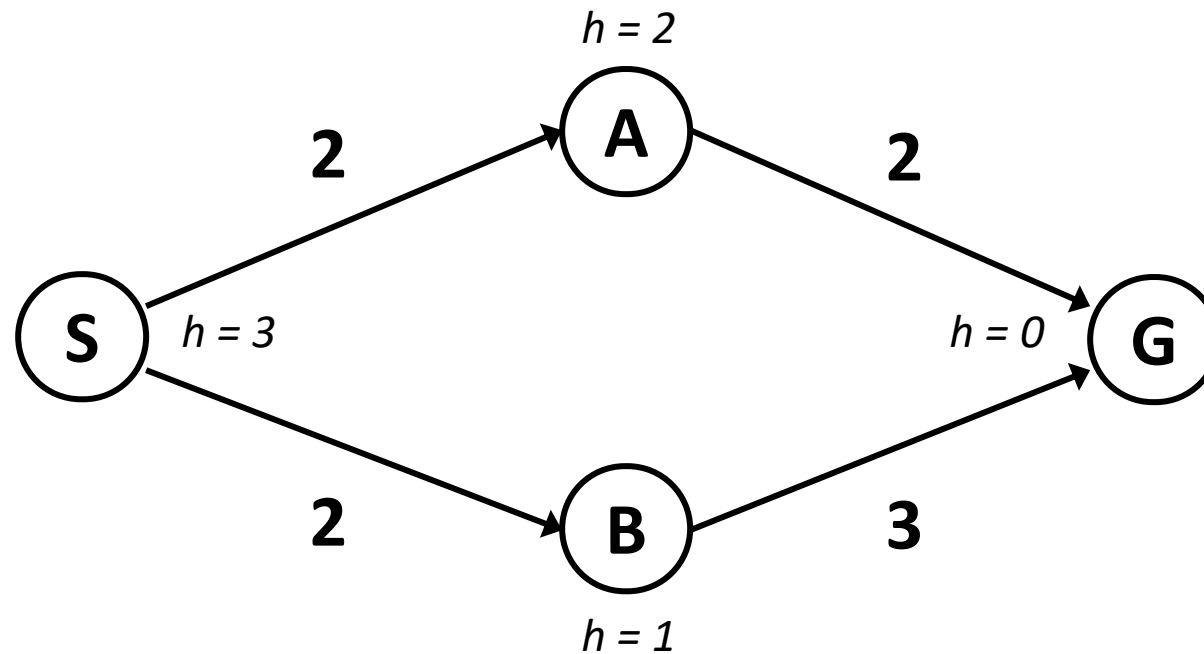
- **Uniform-cost** orders by path cost, or *backward cost*  $g(n)$
- **Greedy** orders by goal proximity, or *forward cost*  $h(n)$



- **A\* Search** orders by the sum:  $f(n) = g(n) + h(n)$

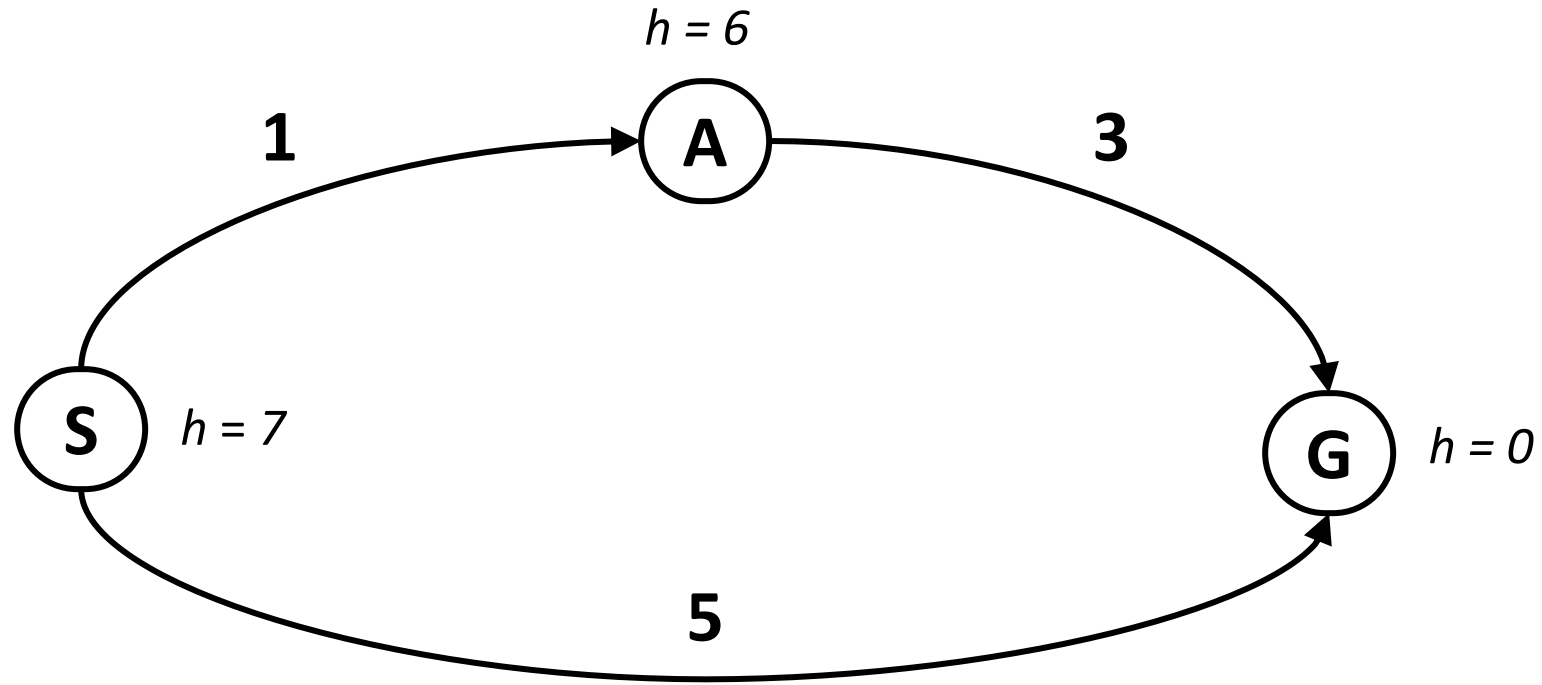
# When should A\* terminate?

- Should we stop when we enqueue a goal?



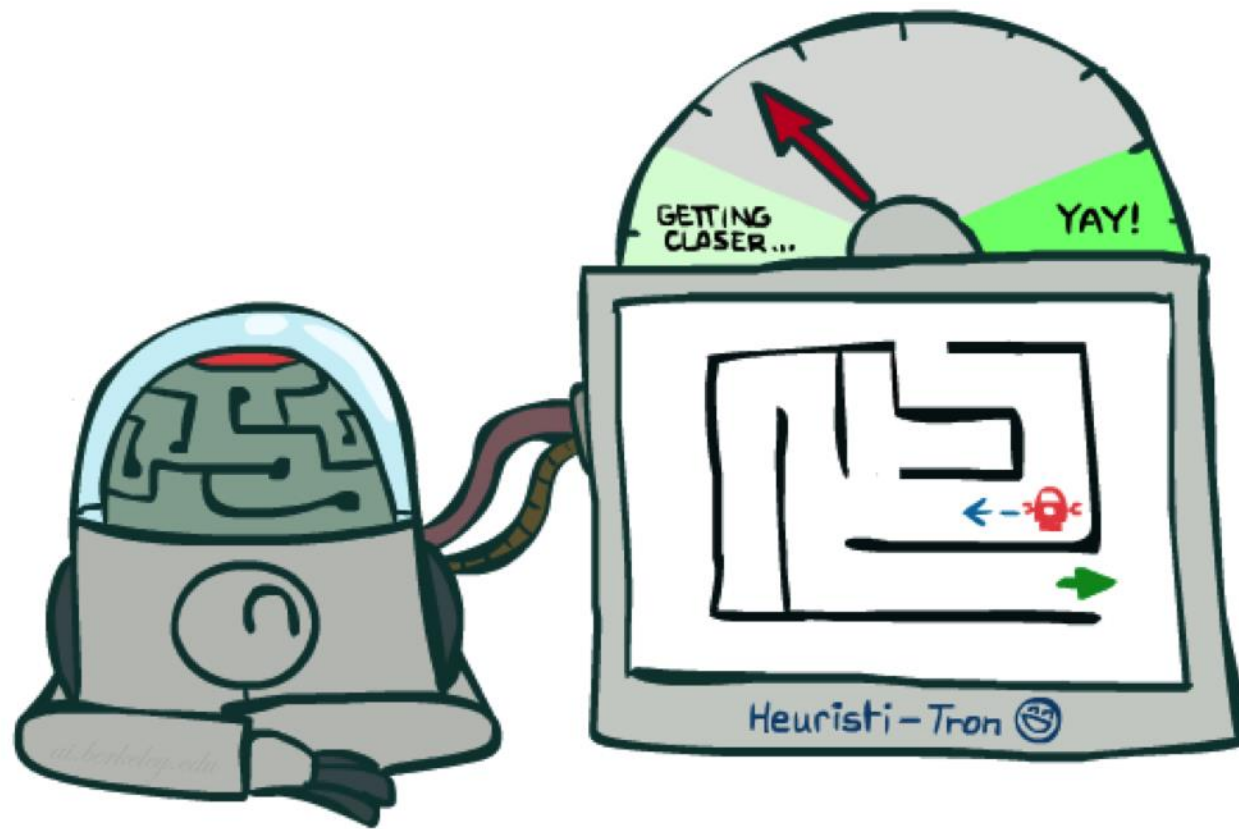
- No: only stop when we dequeue a goal

# Is A\* Optimal?

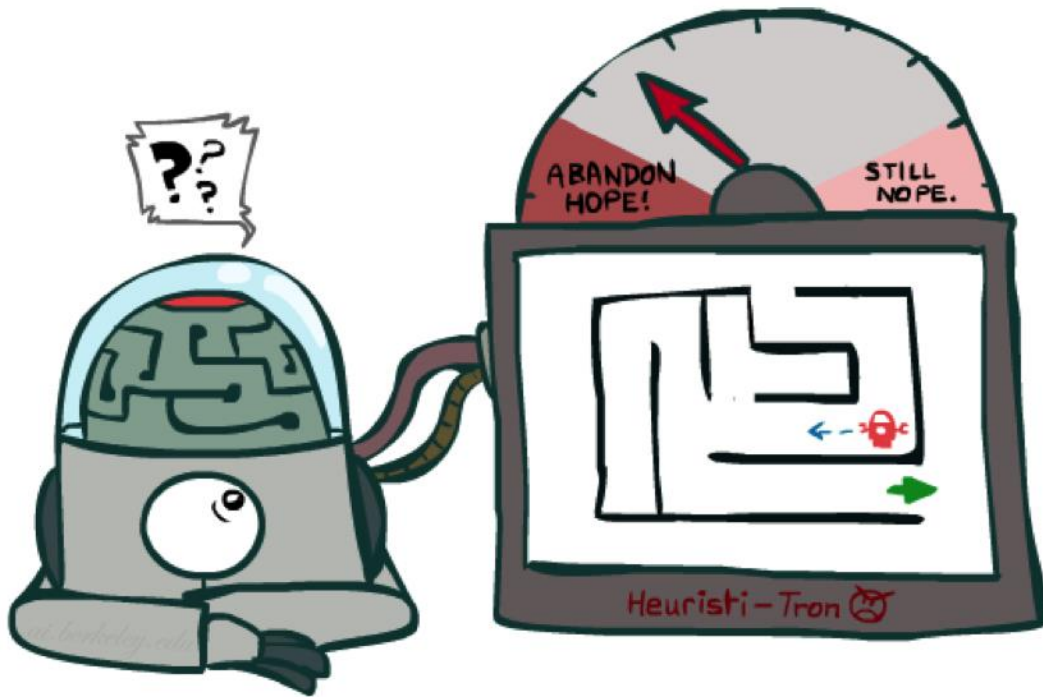


- What went wrong?
- Actual bad goal cost < estimated good goal cost
- We need estimates to be less than actual costs!

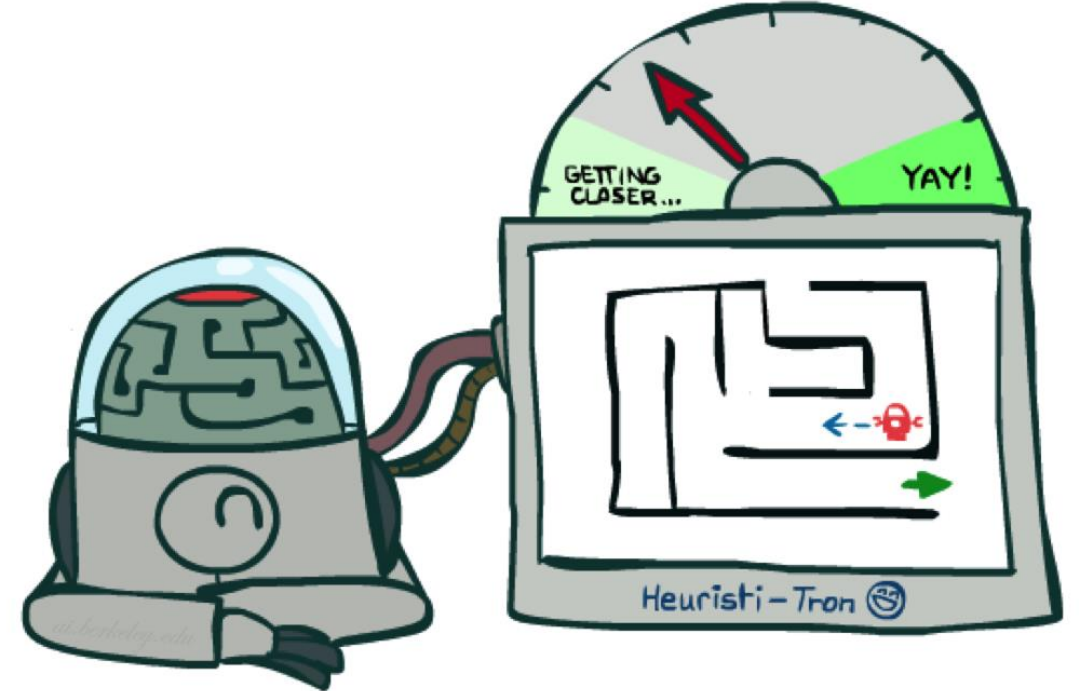
# Admissible Heuristics



# Idea: Admissibility



Inadmissible (pessimistic) heuristics break optimality by trapping good plans on the fringe



Admissible (optimistic) heuristics slow down bad plans but never outweigh true costs

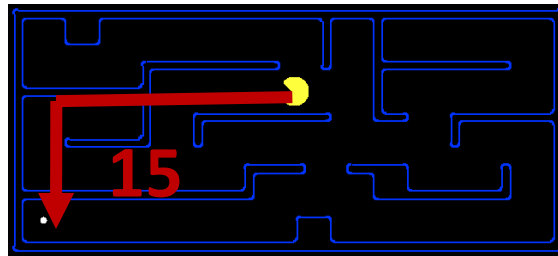
# Admissible Heuristics

- A heuristic  $h$  is *admissible* (optimistic) if:

$$0 \leq h(n) \leq h^*(n)$$

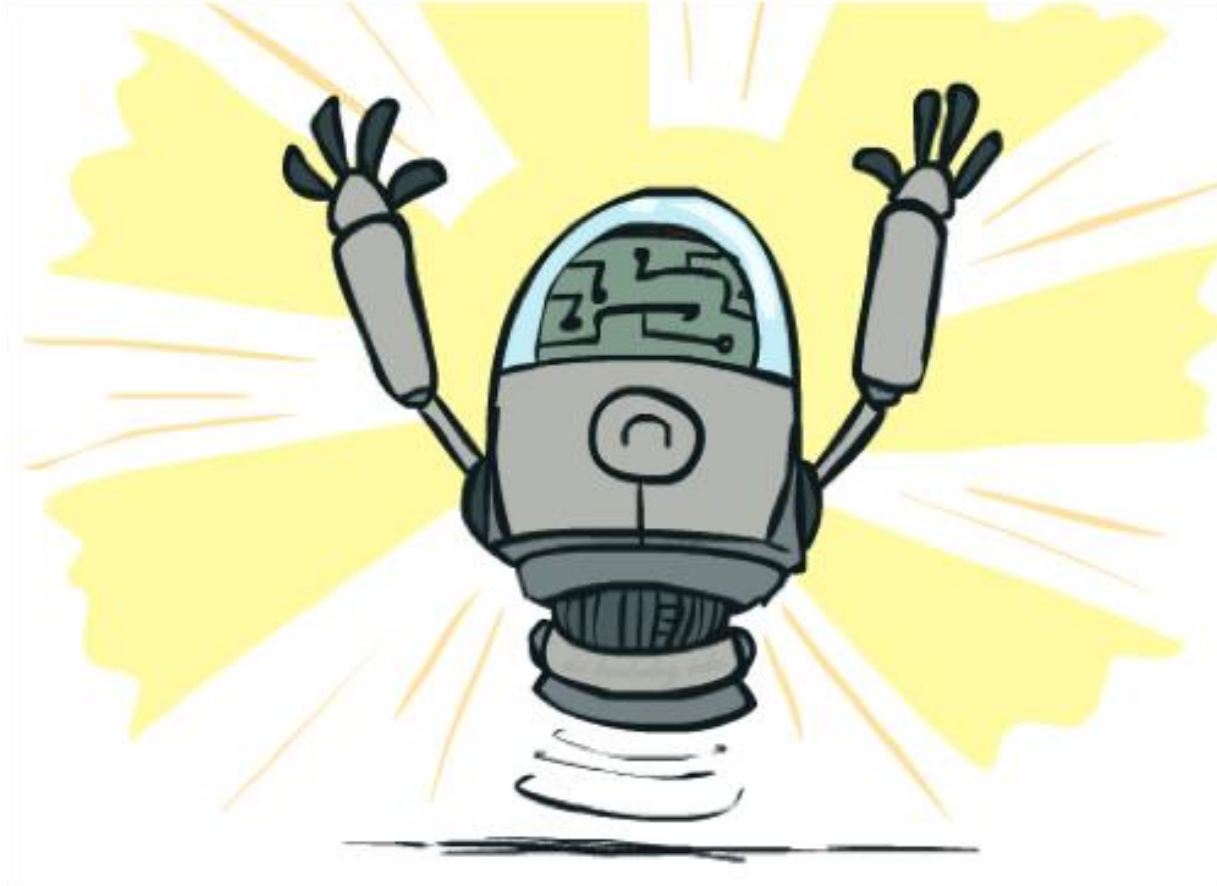
where  $h^*(n)$  is the true cost to a nearest goal

- Example:



- Coming up with admissible heuristics is most of what's involved in using A\* in practice.

# Optimality of A\* Tree Search



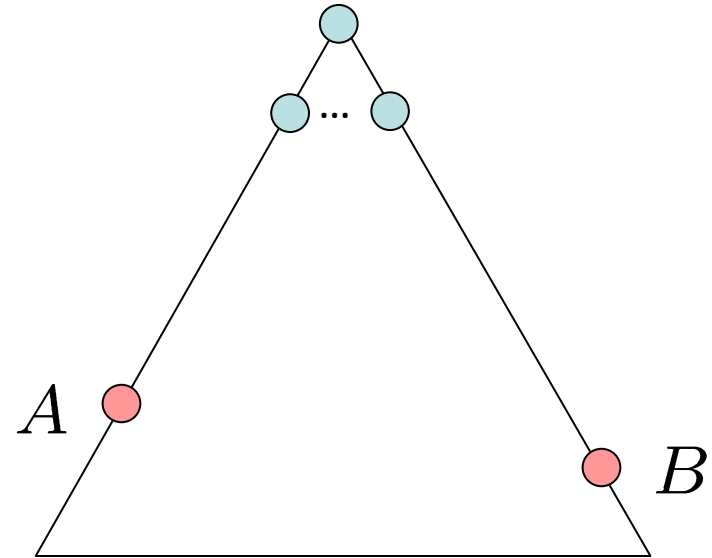
# Optimality of A\* Tree Search

Assume:

- A is an optimal goal node
- B is a suboptimal goal node
- $h$  is admissible

Claim:

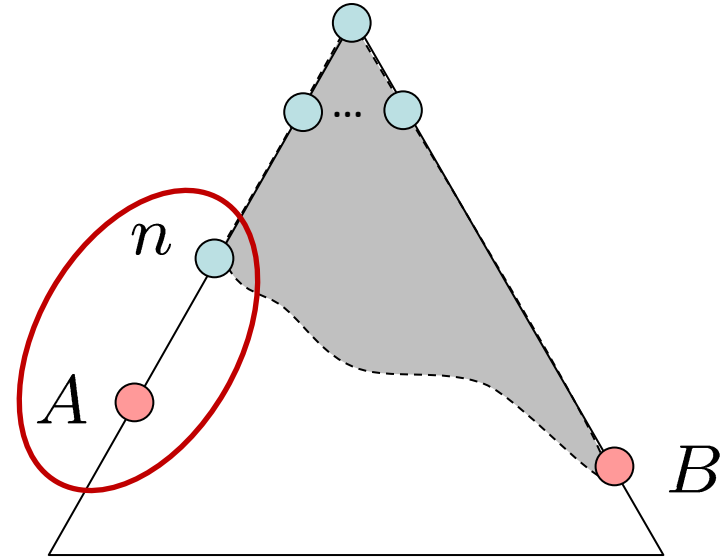
- A will exit the fringe before B



# Optimality of A\* Tree Search: Blocking

Proof:

- Imagine B is on the fringe
- Some ancestor  $n$  of A is on the fringe, too (maybe A!)
- Claim:  $n$  will be expanded before B
  1.  $f(n)$  is less or equal to  $f(A)$



$$f(n) = g(n) + h(n)$$

$$f(n) \leq g(A)$$

$$g(A) = f(A)$$

Definition of f-cost

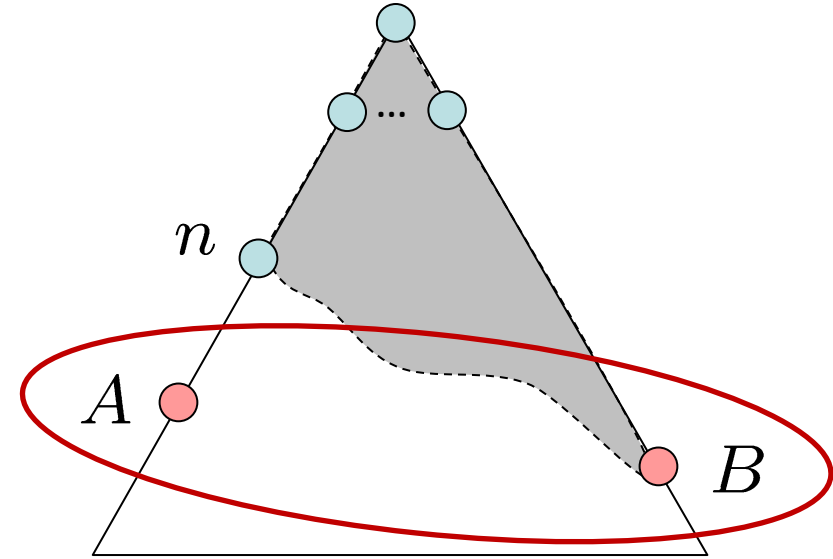
Admissibility of h

$h = 0$  at a goal

# Optimality of A\* Tree Search: Blocking

Proof:

- Imagine B is on the fringe
- Some ancestor  $n$  of A is on the fringe, too (maybe A!)
- Claim:  $n$  will be expanded before B
  1.  $f(n)$  is less or equal to  $f(A)$
  2.  $f(A)$  is less than  $f(B)$



$$g(A) < g(B)$$

$$f(A) < f(B)$$

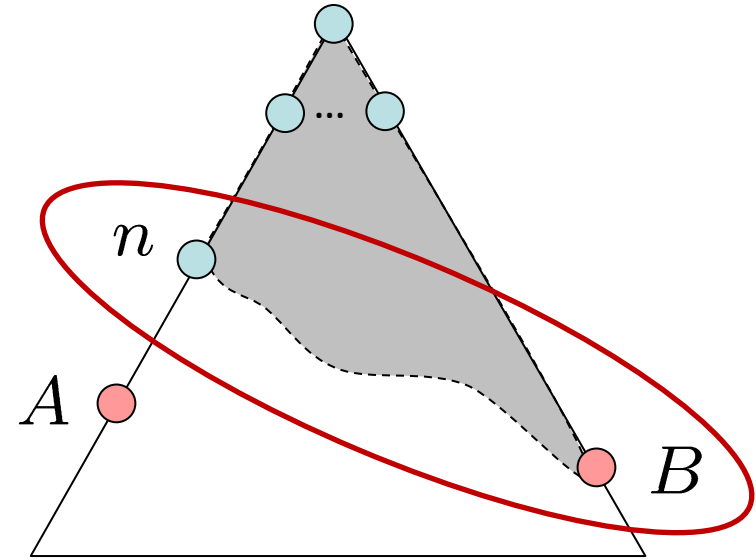
B is suboptimal

$h = 0$  at a goal

# Optimality of A\* Tree Search: Blocking

Proof:

- Imagine B is on the fringe
- Some ancestor  $n$  of A is on the fringe, too (maybe A!)
- Claim:  $n$  will be expanded before B
  1.  $f(n)$  is less or equal to  $f(A)$
  2.  $f(A)$  is less than  $f(B)$
  3.  $n$  expands before B
- All ancestors of A expand before B
- A expands before B
- A\* search is optimal

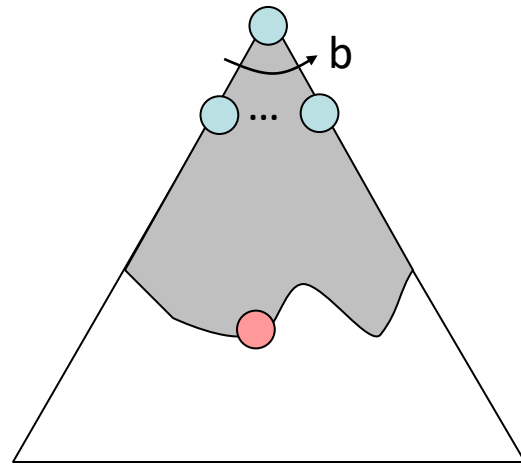


$$f(n) \leq f(A) < f(B)$$

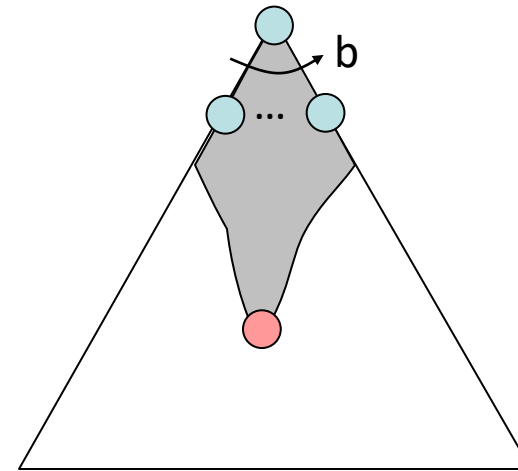
# Properties of $A^*$

# Properties of $A^*$

Uniform-Cost

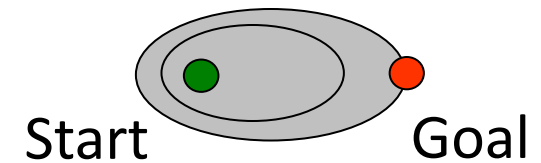
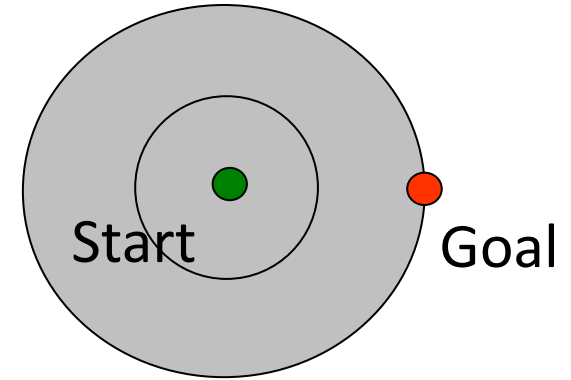


$A^*$



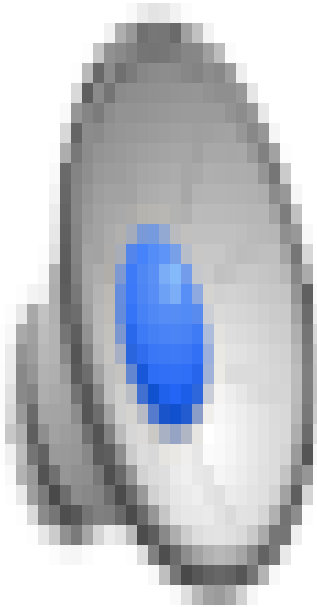
# UCS vs A\* Contours

- Uniform-cost expands equally in all “directions”
- A\* expands mainly toward the goal, but does hedge its bets to ensure optimality



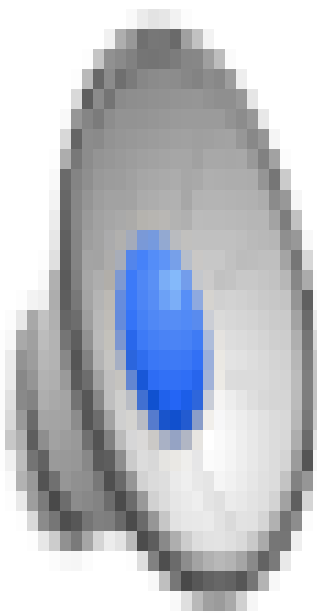
# Video of Demo Contours (Empty) -- UCS

---



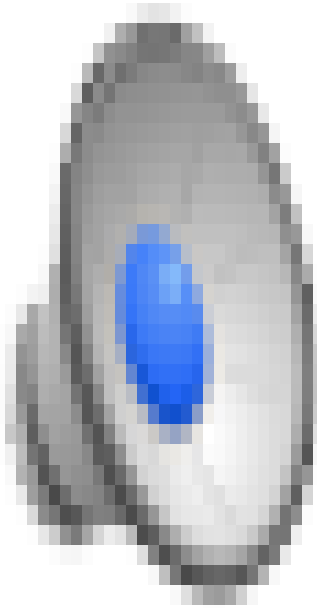
# Video of Demo Contours (Empty) -- Greedy

---



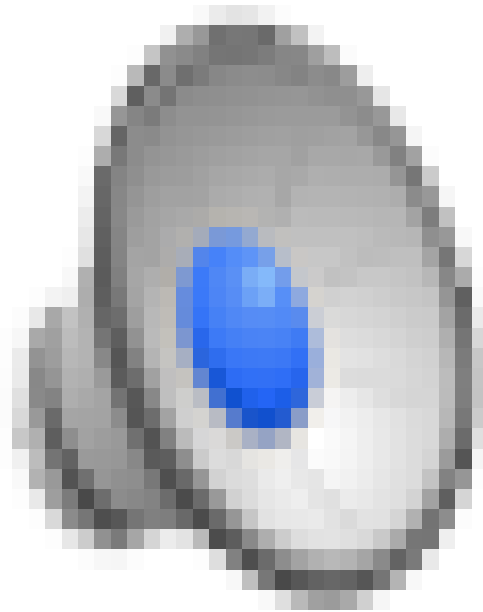
# Video of Demo Contours (Empty) – A\*

---



# Video of Demo Contours (Pacman Small Maze) – A\*

---



# Comparison



Greedy



Uniform Cost



A\*

# A\* Applications

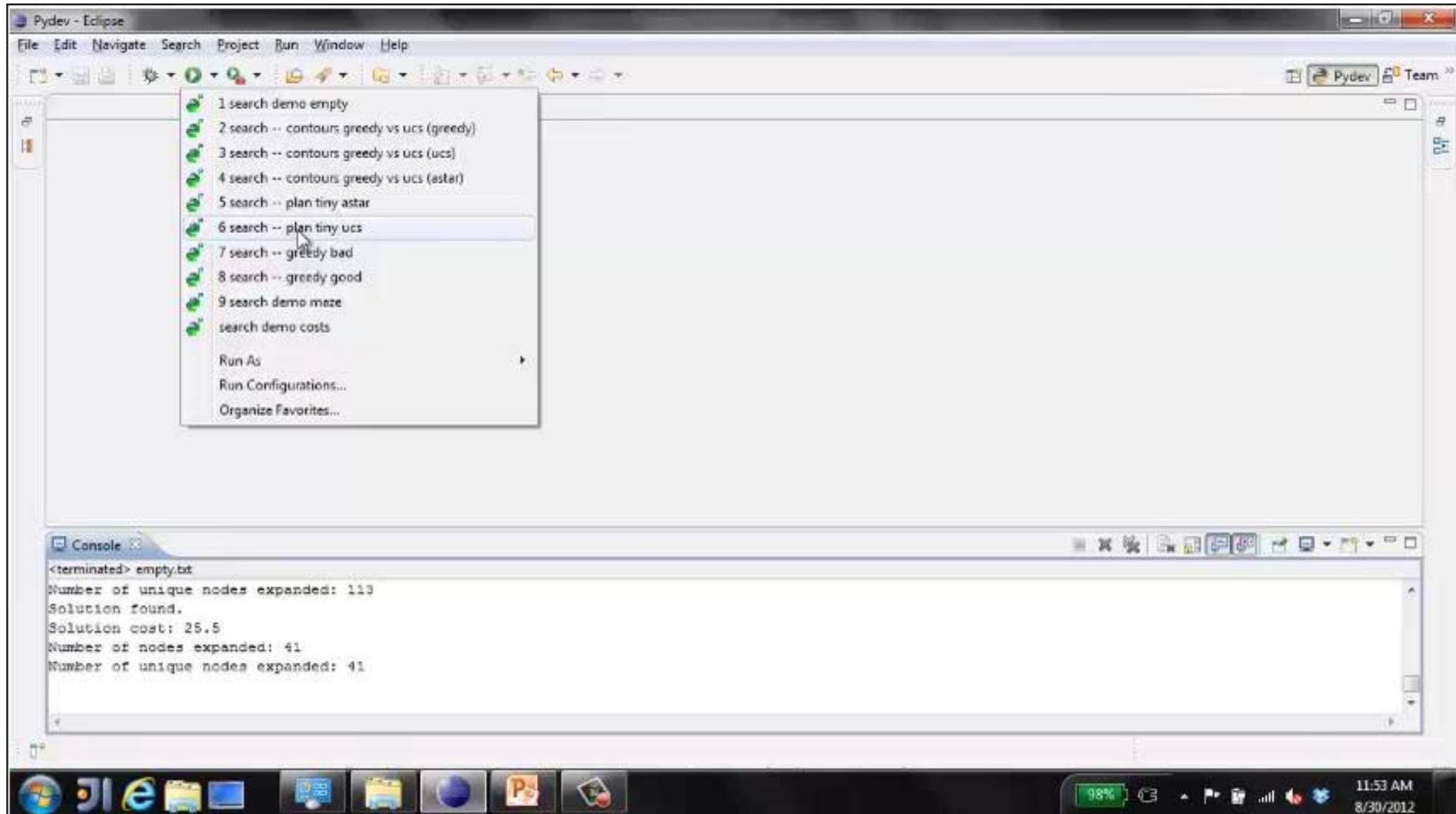


# A\* Applications

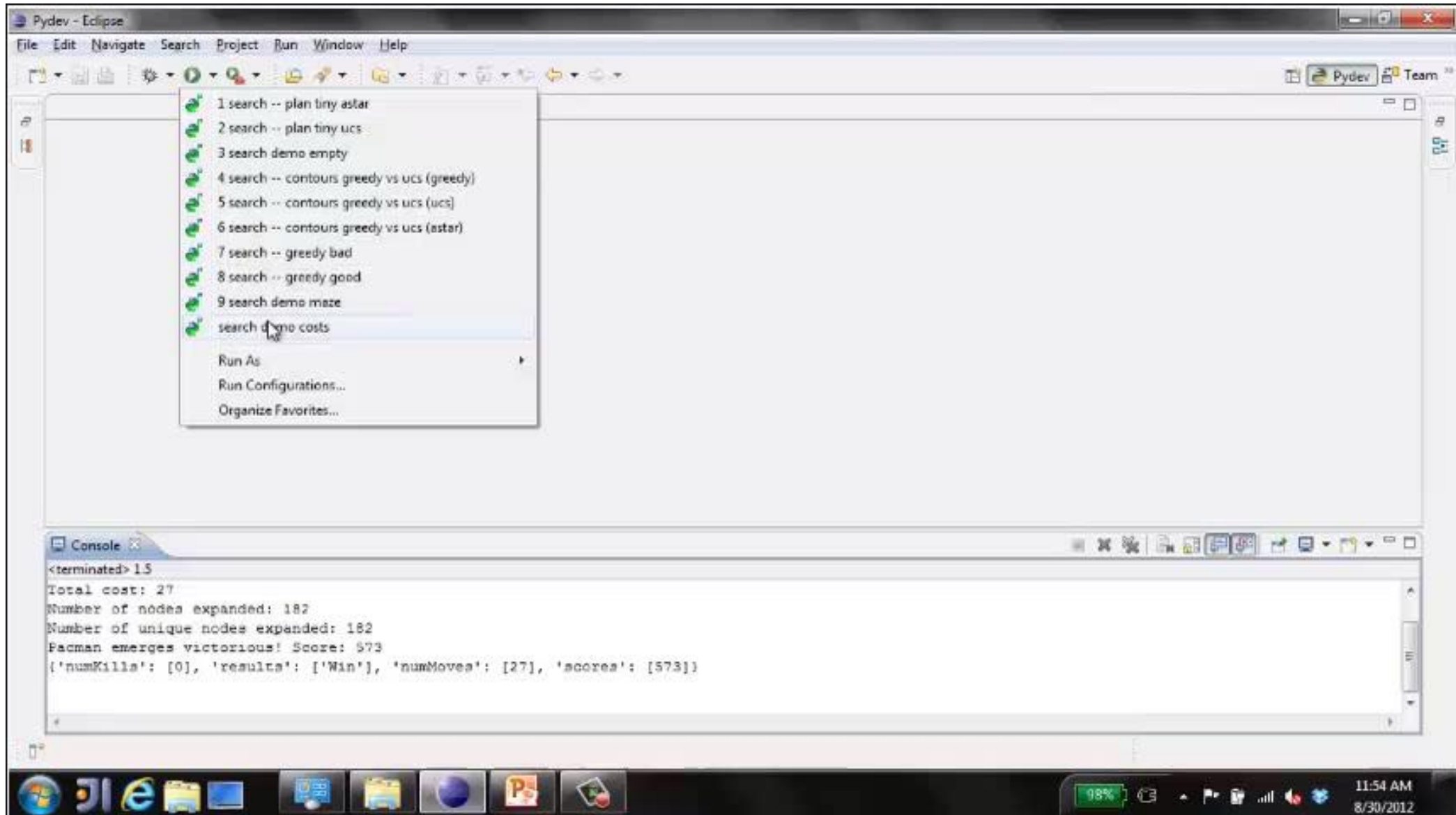
- Video games
- Pathing / routing problems
- Resource planning problems
- Robot motion planning
- Language analysis
- Machine translation
- Speech recognition
- ...



# Video of Demo Pacman (Tiny Maze) – UCS / A\*

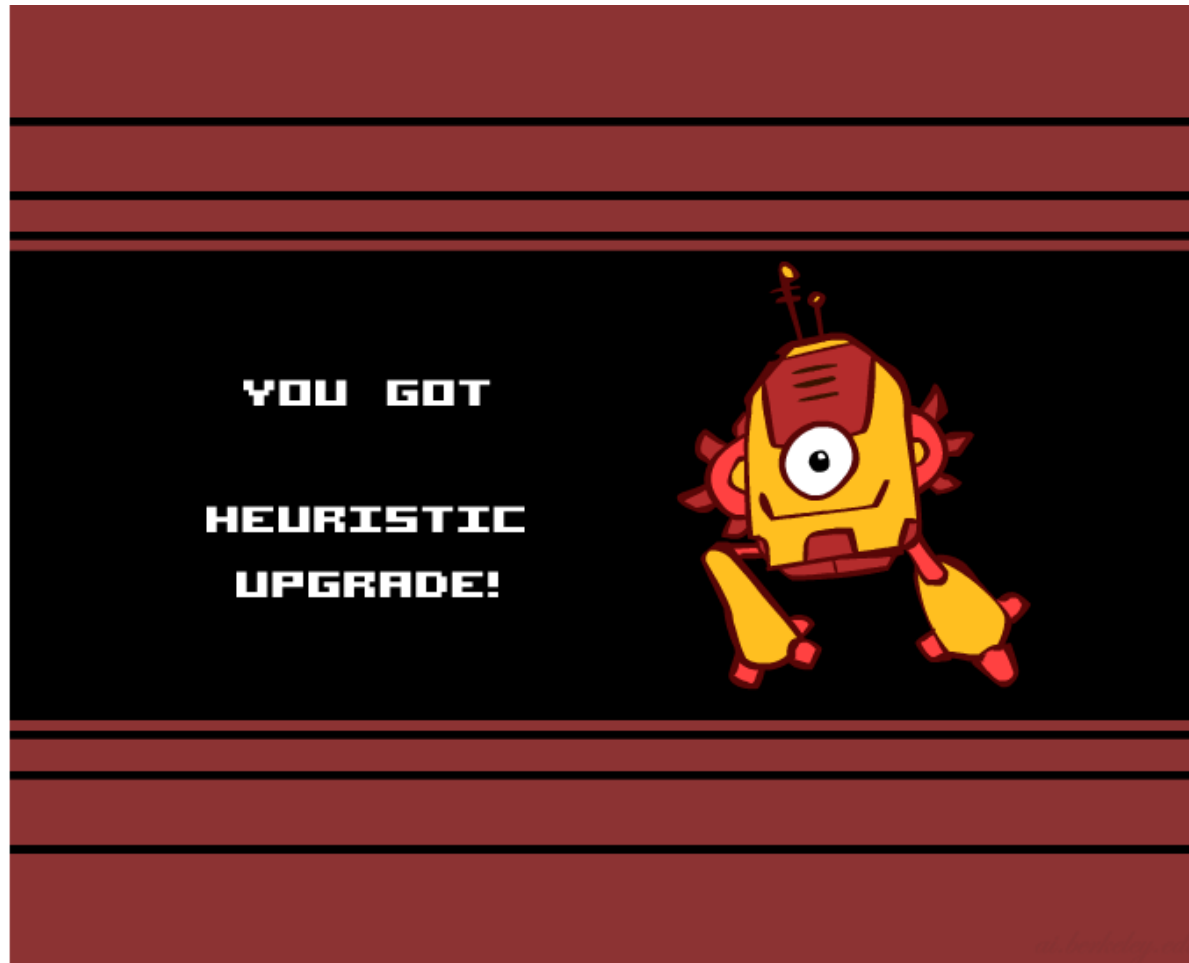


# Video of Demo Empty Water Shallow/Deep – Guess Algorithm



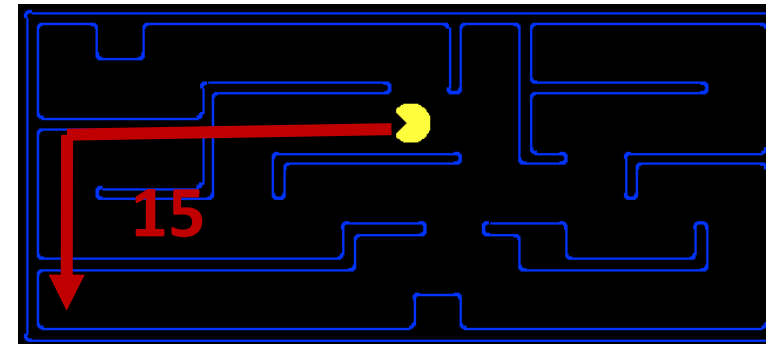
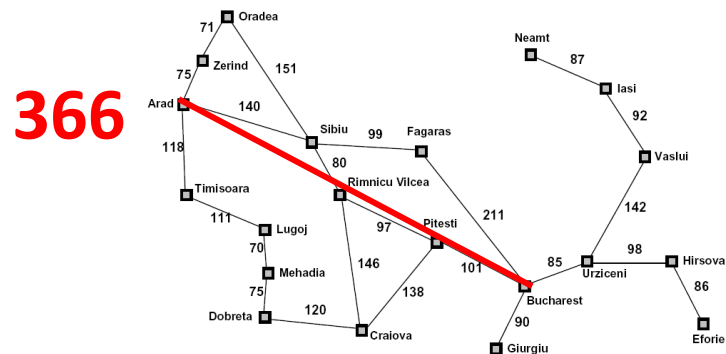
# Creating Heuristics

---



# Creating Admissible Heuristics

- Most of the work in solving hard search problems optimally is in coming up with admissible heuristics
- Often, admissible heuristics are solutions to *relaxed problems*, where new actions are available

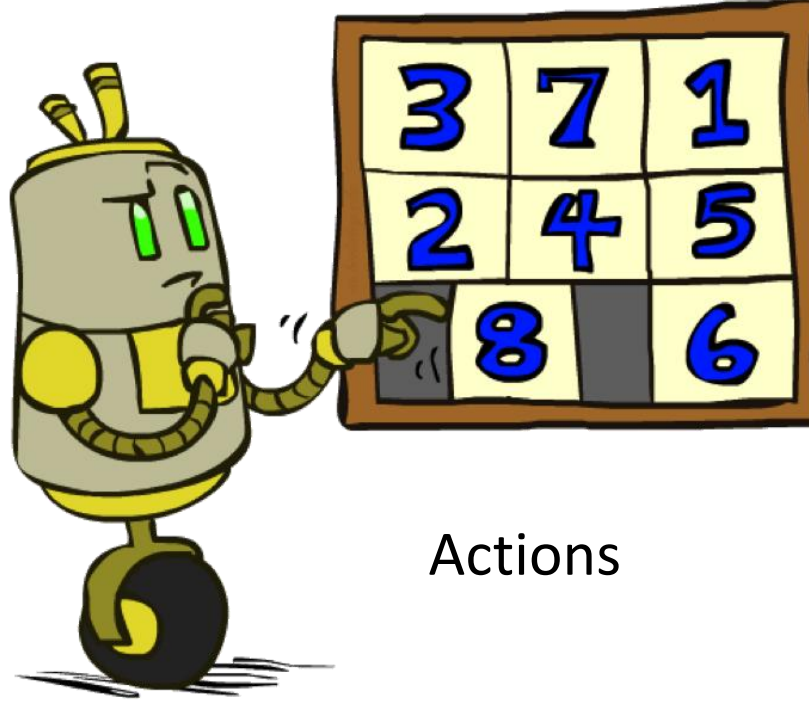


- Inadmissible heuristics are often useful too

# Example: 8 Puzzle

|   |   |   |
|---|---|---|
| 7 | 2 | 4 |
| 5 |   | 6 |
| 8 | 3 | 1 |

Start State



Actions

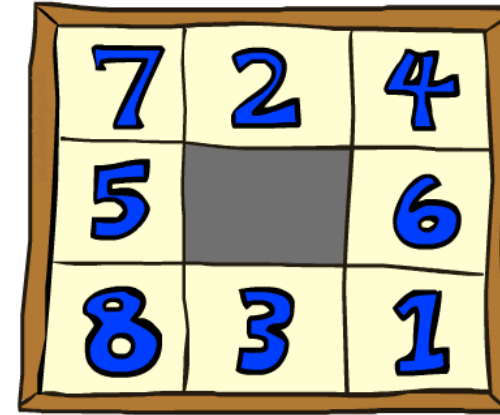
|   |   |   |
|---|---|---|
|   | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

Goal State

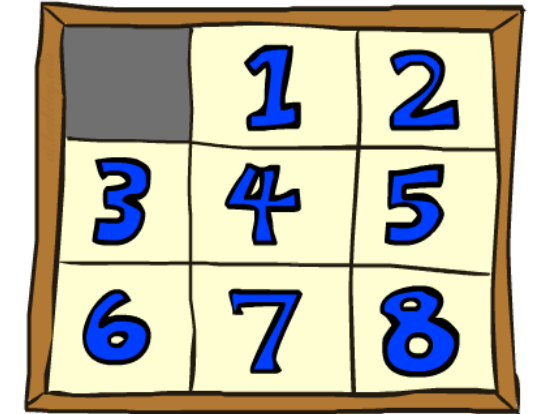
- What are the states?
- How many states?
- What are the actions?
- How many successors from the start state?
- What should the costs be?

# 8 Puzzle I

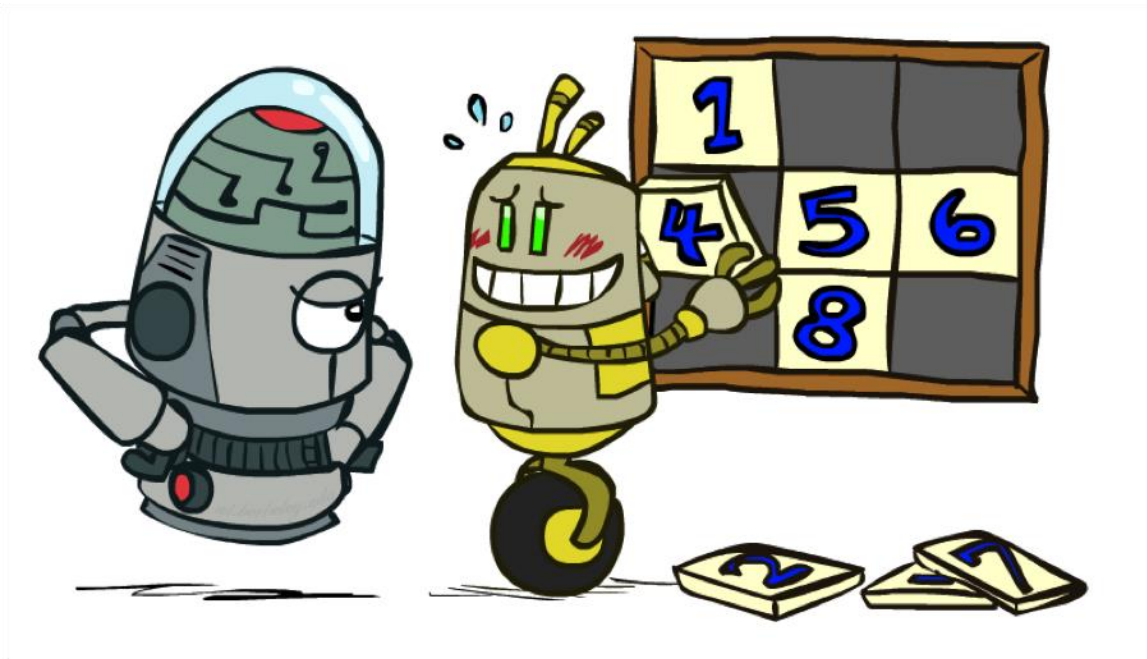
- Heuristic: Number of tiles misplaced
- Why is it admissible?
- $h(\text{start}) = 8$
- This is a *relaxed-problem* heuristic



Start State



Goal State

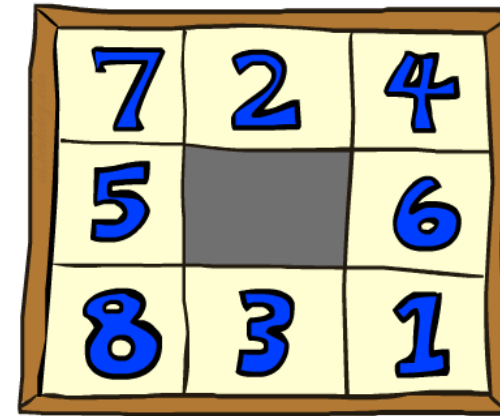


Average nodes expanded  
when the optimal path has...

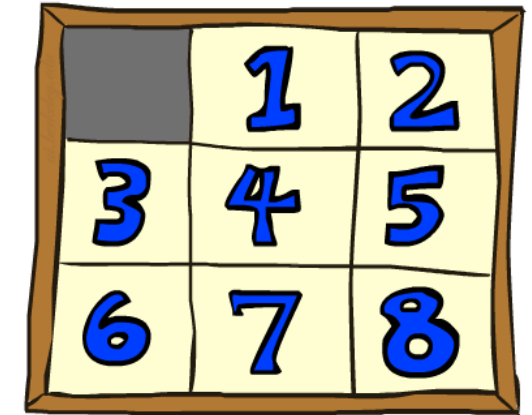
|       | ...4 steps | ...8 steps | ...12 steps       |
|-------|------------|------------|-------------------|
| UCS   | 112        | 6,300      | $3.6 \times 10^6$ |
| TILES | 13         | 39         | 227               |

# 8 Puzzle II

- What if we had an easier 8-puzzle where any tile could slide any direction at any time, ignoring other tiles?
- Total *Manhattan* distance
- Why is it admissible?
- $h(\text{start}) = 3 + 1 + 2 + \dots = 18$



Start State



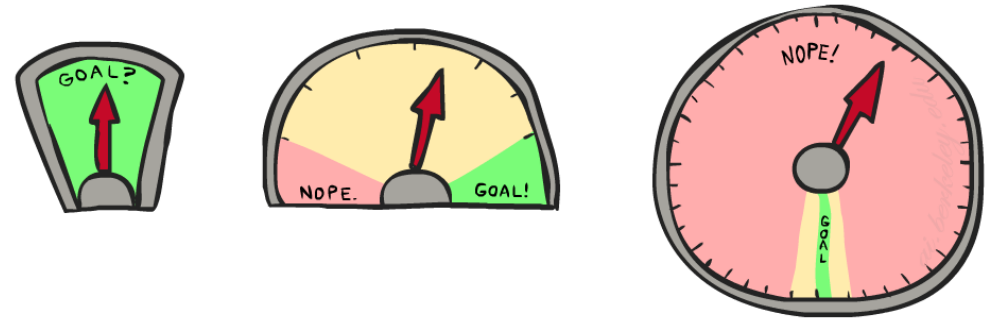
Goal State

| Average nodes expanded when the optimal path has... |            |            |             |
|-----------------------------------------------------|------------|------------|-------------|
|                                                     | ...4 steps | ...8 steps | ...12 steps |
| TILES                                               | 13         | 39         | 227         |
| MANHATTAN                                           | 12         | 25         | 73          |

# 8 Puzzle III

- How about using the *actual cost* as a heuristic?

- Would it be admissible?
- Would we save on nodes expanded?
- What's wrong with it?



- With  $A^*$ : a trade-off between quality of estimate and work per node

- As heuristics get closer to the true cost, you will expand fewer nodes but usually do more work per node to compute the heuristic itself

# Semi-Lattice of Heuristics

# Trivial Heuristics, Dominance

- Dominance:  $h_a \geq h_c$  if

$$\forall n : h_a(n) \geq h_c(n)$$

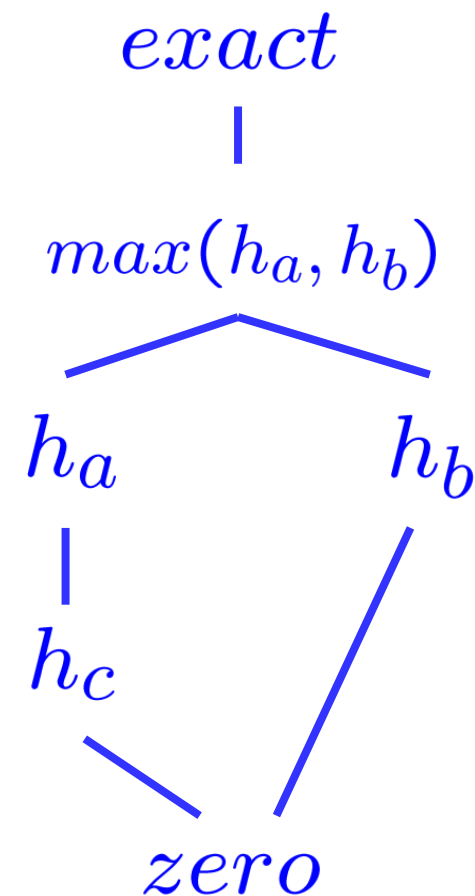
- Heuristics form a semi-lattice:

- Max of admissible heuristics is admissible

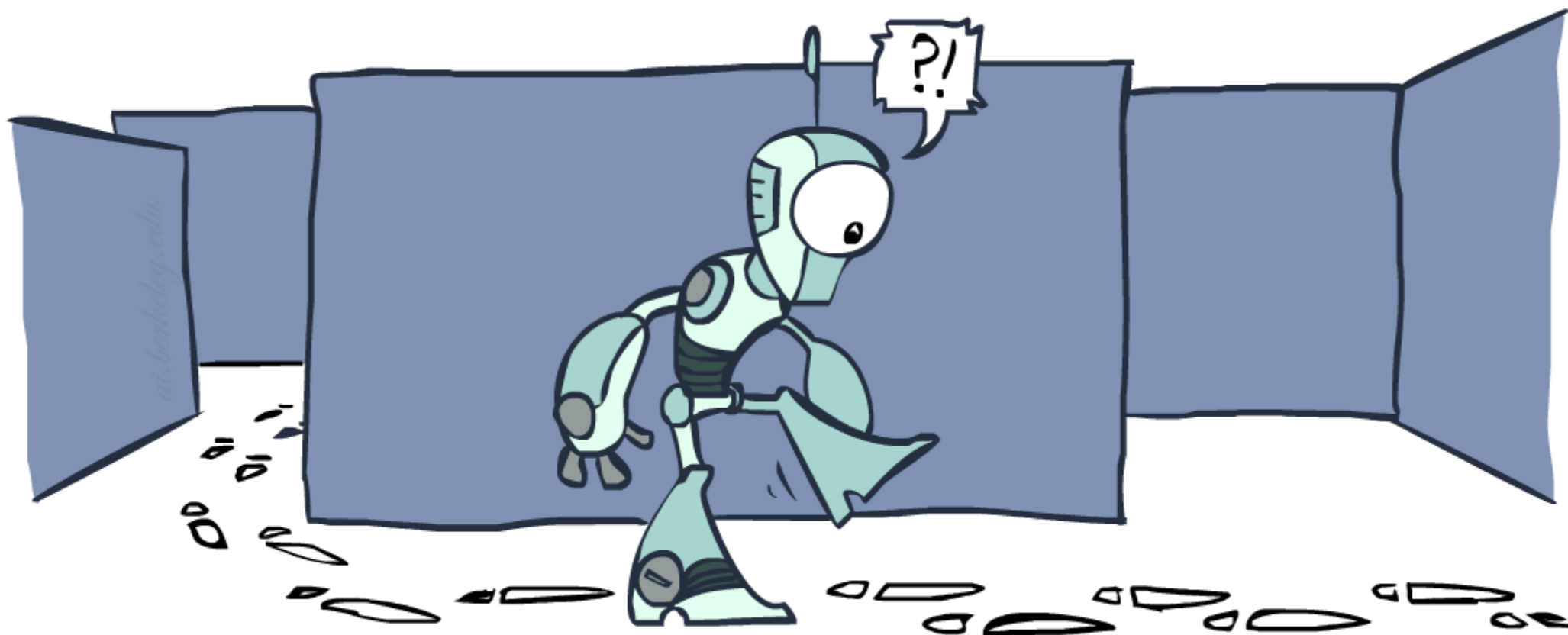
$$h(n) = \max(h_a(n), h_b(n))$$

- Trivial heuristics

- Bottom of lattice is the zero heuristic (what does this give us?)
- Top of lattice is the exact heuristic

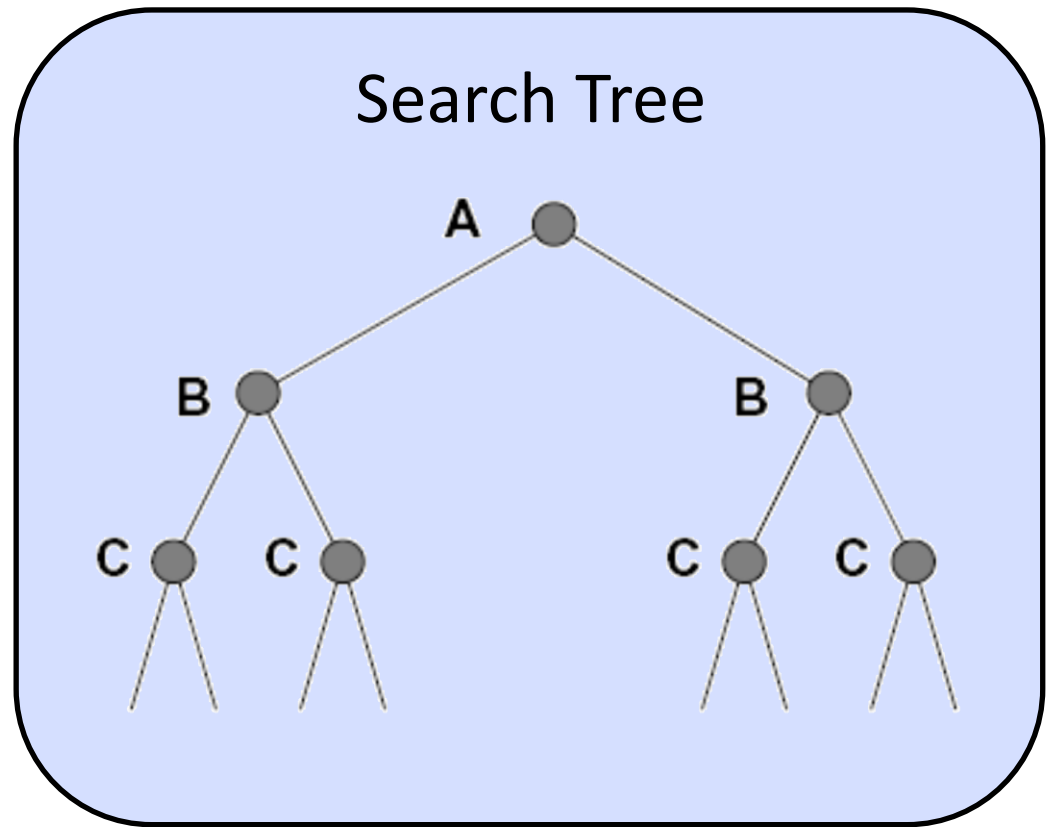
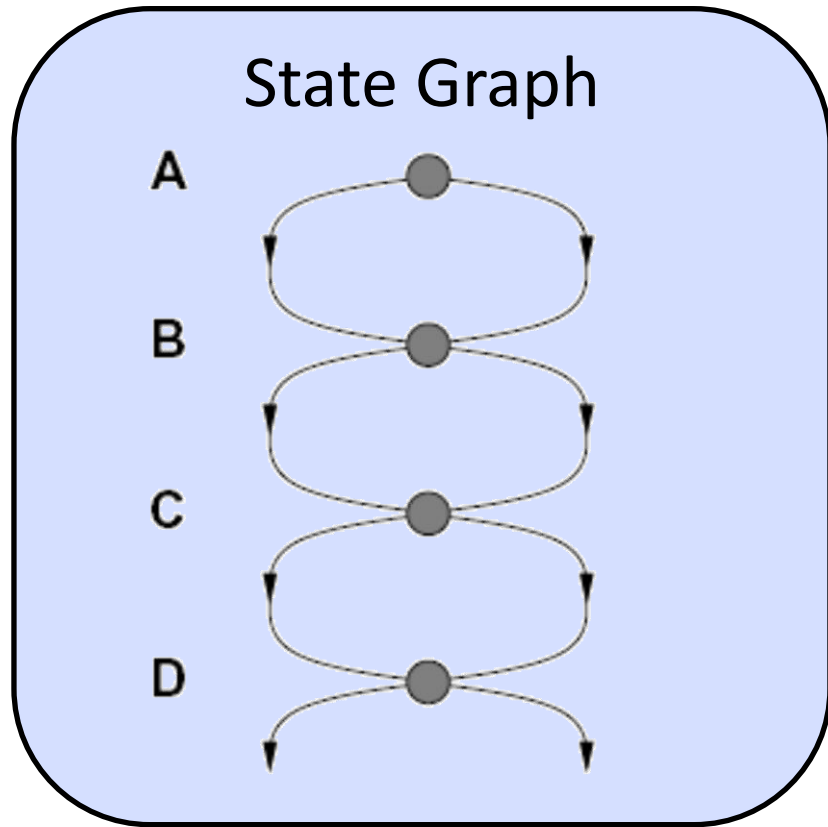


# Graph Search



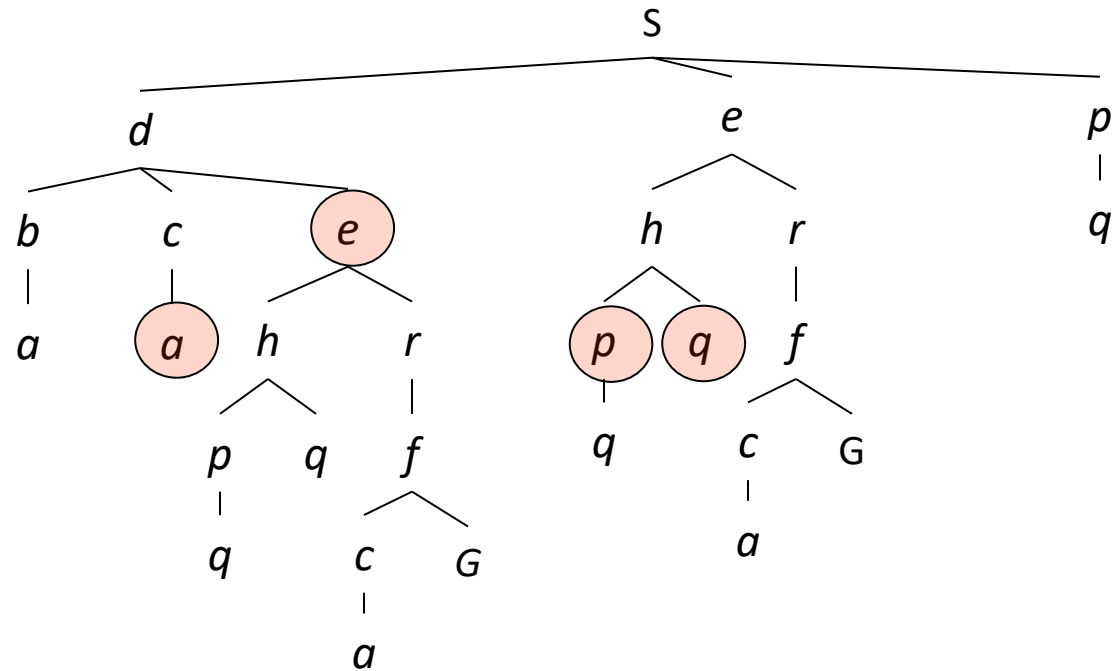
# Tree Search: Extra Work!

- Failure to detect repeated states can cause exponentially more work.



# Graph Search

- In BFS, for example, we shouldn't bother expanding the circled nodes (why?)



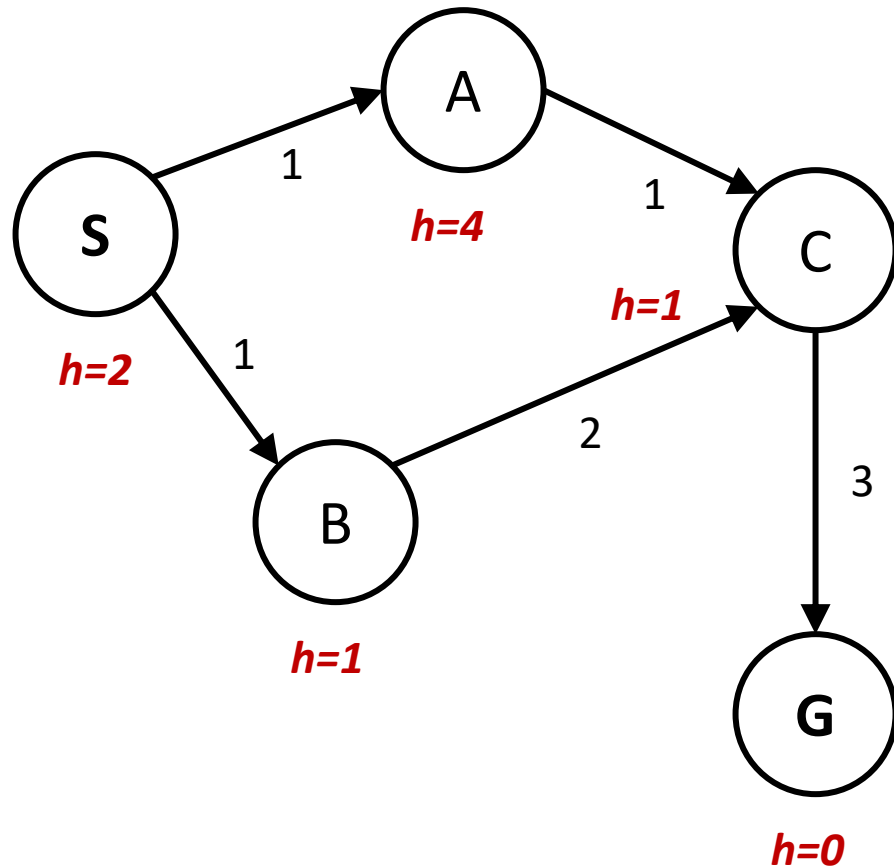
# Graph Search

---

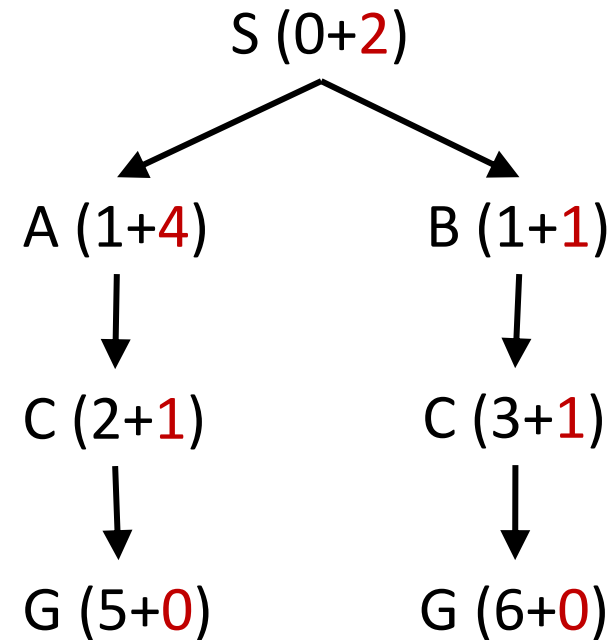
- Idea: never **expand** a state twice
- How to implement:
  - Tree search + set of expanded states (“closed set”)
  - Expand the search tree node-by-node, but...
  - Before expanding a node, check to make sure its state has never been expanded before
  - If not new, skip it, if new add to closed set
- Important: **store the closed set as a set**, not a list
- Can graph search wreck completeness? Why/why not?
- How about optimality?

# A\* Graph Search Gone Wrong?

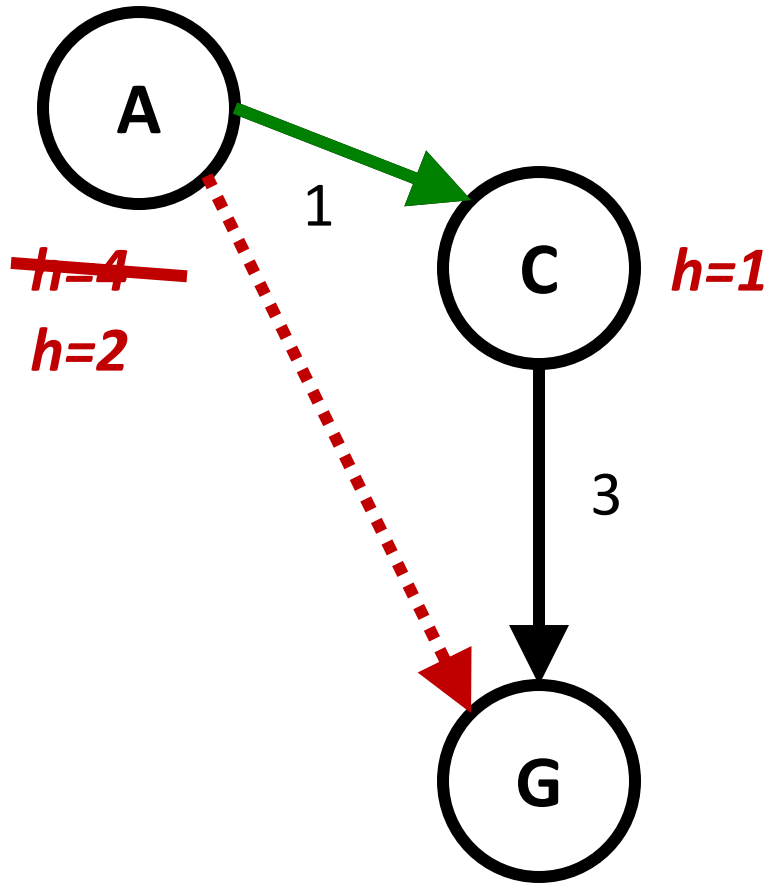
State space graph



Search tree

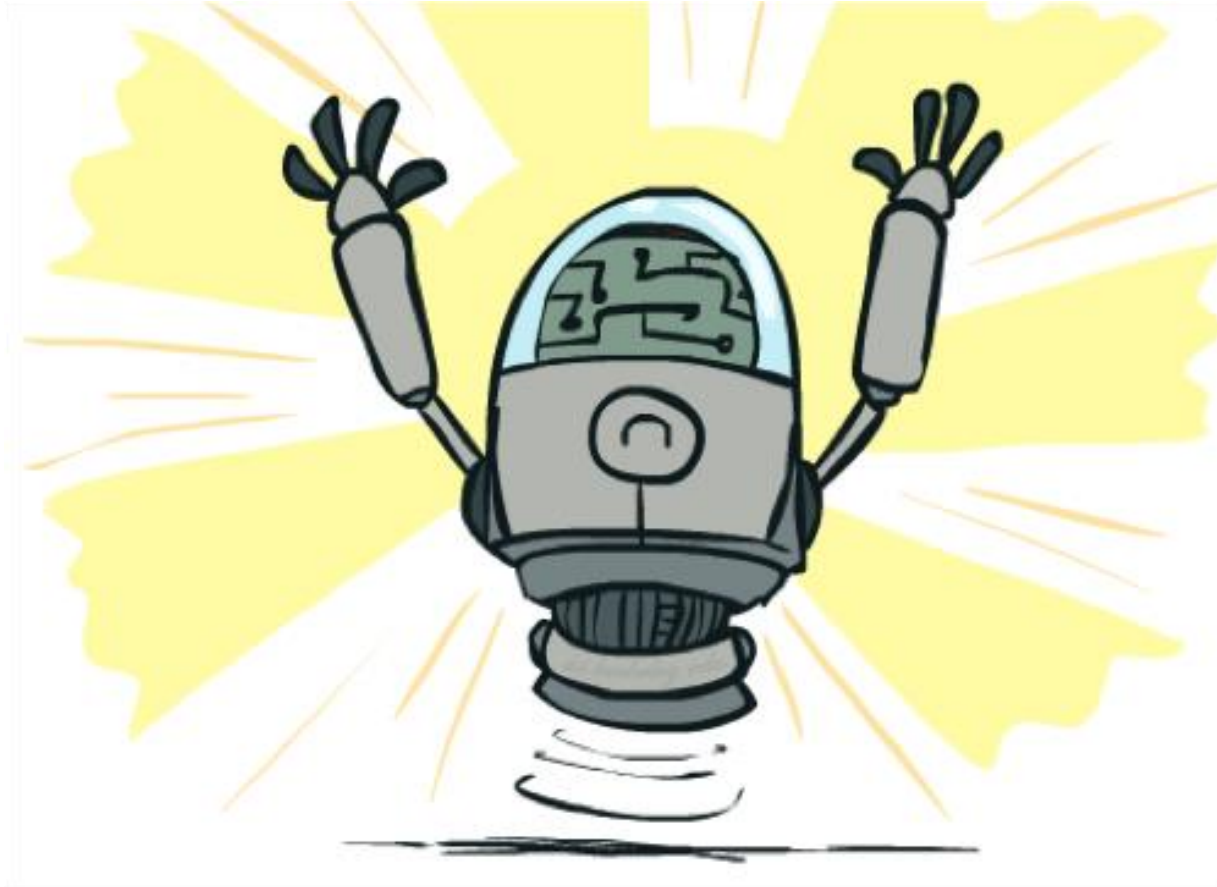


# Consistency of Heuristics



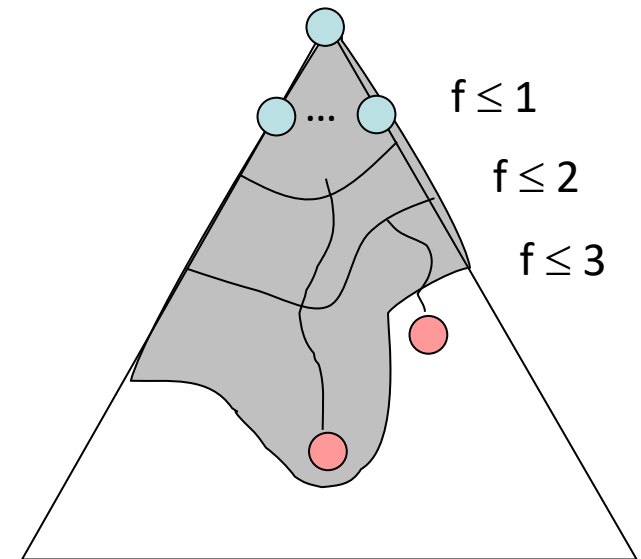
- Main idea: estimated heuristic costs  $\leq$  actual costs
  - Admissibility: heuristic cost  $\leq$  actual cost to goal
$$h(A) \leq \text{actual cost from A to G}$$
  - Consistency: heuristic “arc” cost  $\leq$  actual cost for each arc
$$h(A) - h(C) \leq \text{cost(A to C)}$$
- Consequences of consistency:
  - The f value along a path never decreases
$$h(A) \leq \text{cost(A to C)} + h(C)$$
  - A\* graph search is optimal

# Optimality of A\* Graph Search



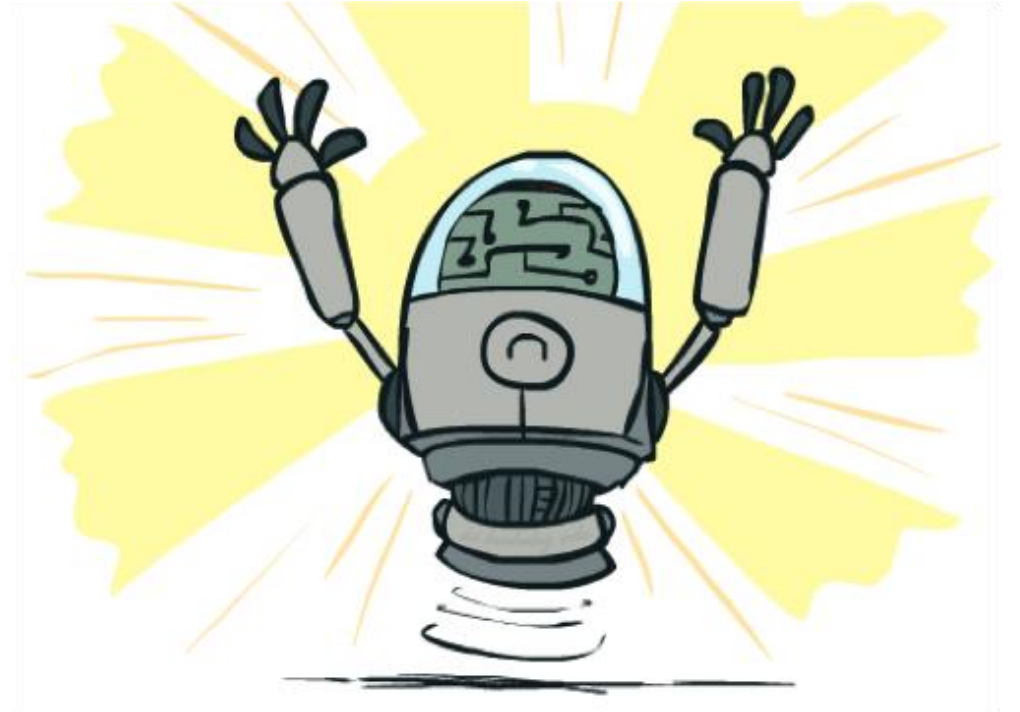
# Optimality of A\* Graph Search

- Sketch: consider what A\* does with a consistent heuristic:
  - Fact 1: In tree search, A\* expands nodes in increasing total f value (f-contours)
  - Fact 2: For every state s, nodes that reach s optimally are expanded before nodes that reach s suboptimally
  - Result: A\* graph search is optimal



# Optimality

- Tree search:
  - A\* is optimal if heuristic is admissible
  - UCS is a special case ( $h = 0$ )
- Graph search:
  - A\* optimal if heuristic is consistent
  - UCS optimal ( $h = 0$  is consistent)
- Consistency implies admissibility
- In general, most natural admissible heuristics tend to be consistent, especially if from relaxed problems



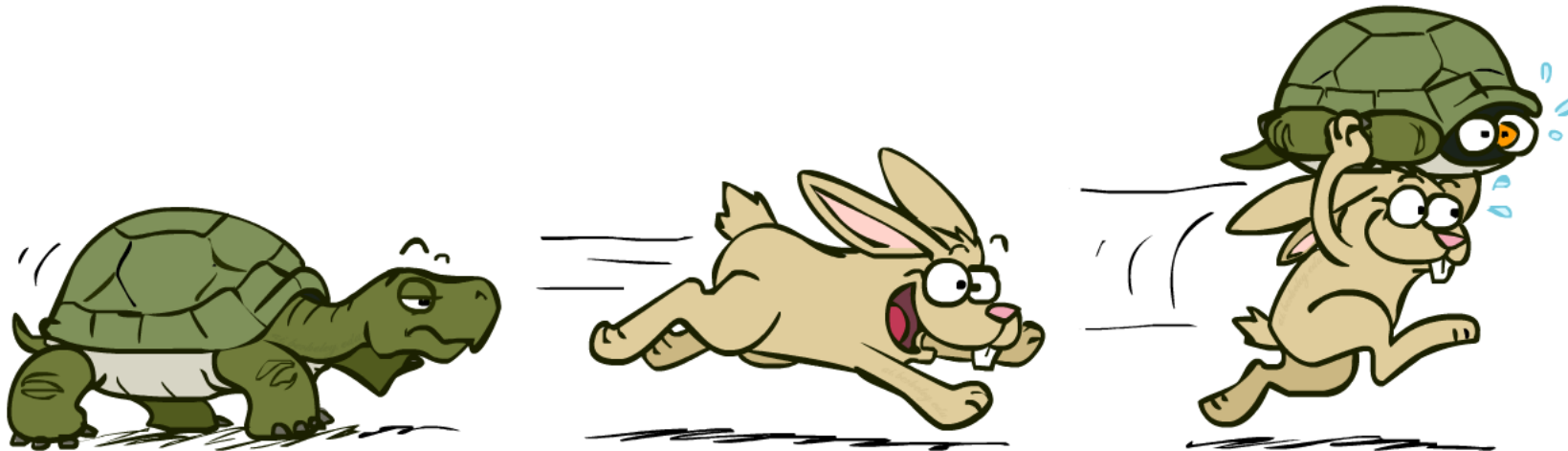
# A\*: Summary

---



# A\*: Summary

- A\* uses both backward costs and (estimates of) forward costs
- A\* is optimal with admissible / consistent heuristics
- Heuristic design is key: often use relaxed problems



# Tree Search Pseudo-Code

```
function TREE-SEARCH(problem, fringe) return a solution, or failure
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    for child-node in EXPAND(STATE[node], problem) do
      fringe ← INSERT(child-node, fringe)
    end
  end
```

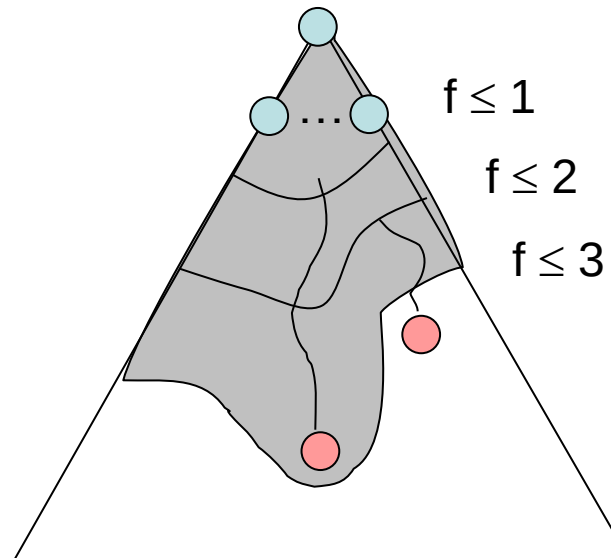
# Graph Search Pseudo-Code

```
function GRAPH-SEARCH(problem, fringe) return a solution, or failure
  closed ← an empty set
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    if STATE[node] is not in closed then
      add STATE[node] to closed
      for child-node in EXPAND(STATE[node], problem) do
        fringe ← INSERT(child-node, fringe)
      end
    end
  end
```

# Optimality of A\* Graph Search

- Consider what A\* does:
  - Expands nodes in increasing total f value (f-contours)  
Reminder:  $f(n) = g(n) + h(n) = \text{cost to } n + \text{heuristic}$
  - Proof idea: the optimal goal(s) have the lowest f value, so it must get expanded first

There's a problem with this argument. What are we assuming is true?



# Optimality of A\* Graph Search

Proof:

- New possible problem: some  $n$  on path to  $G^*$  isn't in queue when we need it, because some worse  $n'$  for the same state dequeued and expanded first (disaster!)
- Take the highest such  $n$  in tree
- Let  $p$  be the ancestor of  $n$  that was on the queue when  $n'$  was popped
- $f(p) < f(n)$  because of consistency
- $f(n) < f(n')$  because  $n'$  is suboptimal
- $p$  would have been expanded before  $n'$
- Contradiction!

